```
name: Build NowLock APK
on:
  push:
    branches: [ "main", "master" ]
  workflow_dispatch:
permissions:
  contents: write
jobs:
  build-apk:
    runs-on: ubuntu-24.04
    steps:
      - name: Generate NowLock v9.0.2 Project, Persistent Keystore & Icons
        shell: python3 {0}
        run: |
          import os, base64
          KEYSTORE_B64 = '''

MIIKzgIBAzCCCngGCSqGSIb3DQEHAaCCCmkEggplMIIKYTCCBbgGCSqGSIb3DQEHAaCCBakEggWl

MIIFoTCCBZ0GCyqGSIb3DQEMCgECoIIFQDCCBTwwZgYJKoZIhvcNAQUNMFkwOAYJKoZIhvcNAQUM

MCsEFMtDxiV6O70aAMh7qNrjJlwrW2h4AgInEAIBIDAMBggqhkiG9w0CCQUAMB0GCWCGSAFlAwQB

KgQQOOmdrxMhDUF/OuqATWpaiQSCBNCaaGGWA9cs9CjDc99DmA/ruLHU9BPod709mxK/bDOe0vJE

FCbphSCUANgU3WXaSDJ/RrBguHhbKa5D5wcxYPETMPOeqiSVUVcDJ6cprtOcW+7PwQC53HZ2mxeK

ieKSMOBFY00FYJ6iEmwM7CJTH4/CmWEwhebDJDu5y720KqZLO+kVTsaI0AdN5PRqNTlMOywDaza9

W2rbXiINS0K6NjQYd1i7NjF91QOaVS0kmO0g/ADCDlY4EDQTC6g4XRNSVKZR+hWCH3SYPF4jZ+i/

3OcSf4zBnZVV8xf9WpcUpwmZB93SlZ5J6AnUCn4qzHZI0R/wbW/qtxIpIofOJ6zvDkepdQroDJdO

Alj24XxLL1vkFEU8FvvU3nmegi3uWbSEwL0nRV4HDv28LCu0awmioXOvVa3ipTGVT2qXgvwgYImG
```

OsJZisTjaFSdtDjVWv2o0VPcLUlJeaE417as+2OSKJ5azZ/CoR0FnGU8k4awZNShh+AD4RWB
XVJE

o5tn83qZo5ijHO/7y82VSScl5RYZl3KApRUCwZeAG2Yx3rQathqV9IiL11BsXhqbBH5ZlZZIyCQo

pqP8FUjOxUaoALhhAvTjZ/E8VPDkRDbkfervSdCKdAS/tZztq+8/6sv9fLZcLHj4mISRHWVibNQC

MqWGtl/1AP14N/svlQv0gX5l+n8bIYf7UHZ8n/AOPsrFjWZOwEQri9w4uwXk9K+KF3TXrugi7NM
R

Wr2jIztf6lmaYz1z9BoV/DE8VJikF3Y9Nw15jZRFy/KeV9TS9rbGDYxY8UntaSD4Qcvq2qiauK4/

Tk7pmo295MzzXxeCBLwtX4kVipGi3XfobkGLqxDkYNlnqVv8ApbM1rwheLih2oL9AVr+iWbJl2za

Ksg5SiFWcn3t6mCnu1W4jD6XxVkFhUEsQWKFQpa1QGuPw7WtiP0E3Pe+SzxEIAfaGKcI5ZhJ
K07j

1sLCWsAYP1tHy0zMmcE2KW9Twxh+DH+5oZ8ac2ETdUHmf0t/s0gyT1W5Uz3rLRQ0AxKqcB
MXe+HL

+zlX/bIuLOgAQMi0e0djmALAXWmOrfvQNpH7Pi2aZaN1DUKoFS9YOL3YPkizJ1xMd8zoopkFf
XDZ

6NS6ofvdDJxXYkcGFHqT/PE7VvOwO11Wuc64IAX2IUpaT/NYUsXcHkZEdwOHWiIHckC+DQO
b4jb2

4Q1sOtf4/9iC+bXUNaHW8CMpQl0TpAX8q0BAcHCgInkczGfFj+v5gSdgbVEwYlImBKRcRk2KV
R0u

CeFGnoDpub7C4N+62R6OCWzSbdfQ/0wH+BguBOe06wYzJocb6FDX+4iZt9XVuMrmbeqG7Z
+4kOiX

Aq0uk6KcrPEhZGGYugHe4sWBHSY5Us6pRDiXIOrN0XHsoPlcx2fhqn+BLXEU/RX1V6d6iVuyel
iJ

UdT5+Q2lmqblUtegtLt0PRc2LO2ZiFHsvrOo0++kAt0DihuNiL7ChtB/s1Jth3CIyQQuNwhrAFCN

83z9m9RRE5n9KHzT/q5NHu1G6egr+jw781zGNRXhKuc61H79fFMsGOWUJxx/dV1UBfBq6z/P
S296

lb2Ks0onCQxo7eGz4L5ObGIo7mzXAyQDzc1A5nnl80N6DYrAX8UmyS/3VQjh6peiuryxw5xygp
W5

gDFKMCUGCSqGSIb3DQEJFDEYHhYAbgBvAHcAbABvAGMAawBfAGsAZQB5MCEGCSqGSIb3DQEJFTEU

BBJUaW1lIDE3OTMyMDMxOTI4NTUwggShBgkqhkiG9w0BBwagggSSMIIEjgIBADCCBIcGCSqGSIb3

DQEHATBmBgkqhkiG9w0BBQ0wWTA4BgkqhkiG9w0BBQwwKwQUPMSpYJxSX5Eetyj2N7uSjgTWSL4C

AicQAgEgMAwGCCqGSIb3DQIJBQAwHQYJYIZIAWUDBAEqBBDHoy4byFE/pnqUII06Z9OVgIIEEK+q

TvhNRZMn3p7zwGzqdg3ZMR7fgpmnCxi9VrELNabRJkYcee/PQeLZay7H8mmBEdGwExBLRKba12js

cnTv+XxMJVnOTBv+t557NyZempbUL/VuCb4c2N669kdWi5nHOeIqBAtQtlryng84OjVHjU7H3hXw

NN1StR9r2dCUcl6zEQlWMBigLRto3Xrnmr1+lguZ2uVAI5s3fYX9W4zYqT+PCuJGb+SPNtdmFAc/

vguiZXk8bQGDbnHpgp5YASn39T71fhpljQV1FKz2dhpSSD2I7HfBXAetUy7Qi78hufcSTX8w/EP0

dRxtzlQtmY+wdC8/PASBQexHh+azOVIB68IfZ1VO4z9zwCaAFWtmvRk0CG8NC/olEbKBszVcuJeQ

qh2TGiEiEt0t9yZEdjLNDBOLXdKTcmokHZZUO5Hgp9mByvylTDC4D1vsKsgXDBb9lxbcdahYJwtm

Kfvohw/nHoK4+wx2zfK7Sa6O4mgRUPa8euIa6TNJvOtyVi+eLtaBCUZ32cav1O2dTY/Ny15BOEcV

j6lWgq2itbV60qkInE3Vl0IGLbit5NMWZzs4+lrq/t/wE5j/0fozBVUuPIQK0xdIFGpwqm25n/dF

yLOuAmw+ICYdQ68TcHQdZTA7VzYQswQnxnUkhU4pqujIe6B6Qs89WVfdBpwTyduJ4ooXH+KUOHBG

o+bwB0jlWXoN0/4TGPcI2Qvuj1HjOnBTxBxFCEVjOe6C1ilaqc2eRMOd3t3EVQG62QU2Nfp4GcOZ

htUo/MkbArJHigk6MR60olIDfpqCo+SHc4dDikMEat5isK+dgbfng2knxDIcXPiHXYe+fFFwCT3r

XhAxwyDicp2rAWO9/GC9Fp2gSjb0ZNea1hwnvAhuJCcEL7pqvCDLSbfe0O5qMR4qOEh1G+m9O0tZ

HIOYYs8U8LX09qMAfl0+eXntaL2Jy6yYrU5qpI9/oDrGCRUkRtKvhvfW8acRItoXteNa7q7VzJsE

Gp9q3xFng589wjphvjFJ9QqN0mBq3buPFyaPSRUh5vys6tW/c8iNVFtd67k8q6pCWNdxLam6UmoQ

ZJxaPhx/N2wwG5gKDr6s6lExFxSaKYTN6VQNnpjfldVSm1KNh4uJ0q6sgq4OIPJk53YcI740NbRY

i/lQS5yf7RYWlKbtsJITiFQPK0thhbV+Q/UD/mC5/P6ux5XSXyUGFGqOyzQdh1vP2v8qSbLCTCXG

2kV9EFXE74GieqWHlLFN/Ms/IhPU6cA2F3jXDHrX/916ePDMUAyFE23S6fsMU69VfwLSOpE5mstP

B6XRsCWUGZhhTJFbnzYYnYepVo1Jw4S1Xk8uEvPSyb2qp1QziuZfN491p/GRPod9LLWLJcCARgqg

iHGKTwx23Drhi38bME0wMTANBglghkgBZQMEAgEFAAQghGHzYSPUiVv1rT82kgMfU0STFSiGgMia
n85YzpsOBAUEFBfqkIBkRqD/4cwmAo8xU5m9R/+eAgInEA==
'''
LOCK_ICON_B64 = '''

iVBORw0KGgoAAAANSUhEUgAAAMAAAADACAMAAABlApw1AAABCGlDQ1BJQ0MgUHJvZmlsZQAAeJxj

YGA8wQAELAYMDLl5JUVB7k4KEZFRCuwPGBiBEAwSk4sLGHADoKpv1yBqL+viUYcLcKakFicD6Q9A

rFlEtBxopAiQLZIOYWuA2EkQtg2IXV5SUAJkB4DYRSFBzkB2CpCtkY7ETkJiJxcUgdT3ANk2uTml

yQh3M/Ck5oUGA2kOIJZhKGYIYnBncAL5H6IkfxEDg8VXBgbmCQixpJkMDNtbGRgkbiHEVBYwMPC3

MDBsO48QQ4RJQWJRIliIBYiZ0tIYGD4tZ2DgjWRgEL7AwMAVDQsIHG5TALvNnSEfCNMZchhSgSKe

DHkMyQx6QJYRgwGDIYMZAKbWPz9HbOBQAAAAkFBMVEX///////3+///+/v79/fz8+/vx8fH94YH

z

3qD+33HX1tjV09bV0dXVy6/9zBv/xwn/xAn6wxn/wwn+wwn+wwf9wQv9wAb1rg7rpAjpngrongro

ngnongjonQnAvrrlog+8vbywr7LnmwjYlxS5iB+1ghm2gRyai2WQcDV0WCBDREgwLi4bFhgSDh
ME

BRAAAAgF+5YDAAALmklEQVR42u2di3KiShCGMVxSQUpRLhojLBgjysDw/m93umcGBMUc0X
Cxir9q

d5PaxPTH9HUYjCSNGjVq1KhRo0YNU7KsgGT5tayegN5kVVVLdiv4qYz/8woAspLbrmo6SFNz
CkV+

AQCFmav5YRTHJEnSNEIIHEehrzEkZeBOjwZqfhQnNMvShBCwHf5K0iyjSRwhhKzKAzYf/jIiQr
OU

wDUPw8BnCuDDKCZpRkIk4DIME2ECdqlhnGZwqcF0Ay3P5cOnAUAkWRqHgKkOMJoVRdLC
OKPo7iXT

SwIIYKBZHGr41cPzHjAfrq+hGxVVaHwjwDWKw6H5Ediv49UPIWtO8Q+TzjVFihJDGFMa64Miw
Muf

ZGi+YUxrxCHOARGSLAnZtw3Ffi3K0uiW+dOpwSDODH4QgR9pQyFg7gOXf6ppNwDyhZgWC
AYsArrR

MOz3wRi4wqqk6r8T4DL4+SLEGQmGQKCC+1NwHyyy0tUSYB66WIXCjyKahv0TgP0puL+h8c/
0iuki

DekVjGIRjCjtn0CVArAC3F/CpKjphfVTnkB9zJviY+NyEQZAAP6f0FBcf1kTNrJLH0TRuRuNokA/

JyIdLwho4vdJoEg6ocx/YOrKIxjNDyJo3bAdjWNsqvHjGBkKBKMgILrUW1uhqFqcRdqF/bqed3Ri

lFE1A3qgFPuMAuFMAPVAkXtzIPjx7PpLuf3MfOgUIr0Yi8W/00j0GlUCHy5BX04ECYgSg+fPwn4
/

TmkcoEWqKsZ5GO1VFdpnFbu9yMgJfFEPCO0pkBVFI5CAdBlNFPZPocCSkE1m1Y5/MsFRU2
X/nbuR

IAhTosndhwFYBA4U6VM+AnP7IS/SSLs1NcqwDLxpErlWhAE6kdL5hDOBDJoSY8oCWOMG+R
FvMuVf

p4YkEwQiDMCJUgMIup4AFDXO0IEmIoChwEJ3Y/xPmw+OZJCCQDgRpAKlcwAowTTPQML+
KCP/31/i

6IMERu5EMOv4Me2+rZPVGEqwnjsQ2k/JXf0xEmD1LjlRmMVdz2fnBYAMxPw/TO7tCrD/SALuRXkx

6HwJFIlFQNFCN0rnUECgAE6N8xJEmIi6tV9LeAqSsAToPjNBvX/9ojwMRFtHEq1TAryGMMQoYgHQ

gaAa3e3GiqwRcCIDKreR14Juy7Eix1CEWQrS2AI0NIA7kVFUMz+kcZe3EaCNTmI+xWANgAVIY7VJ

T4lVJA1Y+hW1gCQdttWTqgc1X4DcB89h3K0PQRuERUCT3mSJL0CjCOAdKkQBb0zzUtBhHgLHIYmv

Cw9iNbjxT8dEFJ59CK5BrEpypyGgqyIHgQelU7nh+quyT+OSD/lxh0HAy7AuYxuBCxA8cPXgWwlB

HyoAaHfjvYhhvGDMg0La3H9FHBl5IoXhOOwQAP0XQ0DR0YPYz5488CLRGaBZKX8agCUhVVVxJwtS

lKx+435ekQLRThSlrDMAWYohCfF9f3BhiGH9EYBpilGs5wBp3FUQQ/JEACOfbH3WiU0av4qeEH16

BkhieSJ3BMCKEL99hGmcEK15CpexodWLhhSLYVeFQACoiqrAMjwFYBRDDeTizgFkBRuJ4BmA8t5E

5wD4oW4YTwAQMi0mgh4ANJCu/yWA1i0Au439NEAxEISdA4jN3GcB8l3q7gHOO7QvDRA8C+D3CqA/

C5Df5+gNwHgWwH9xACN4cQC/ZwBuwOMARtAzgPEkQNA7QPCXAOoI0BDAEEnwpQDwZLRaNHNBDqA+

oApAgDGAR8TbvtnHt0/ESGmwo7kA8NgGHwD4haCdZheo5Z0JRVL9MIqSJC4pSfCcMVfUQFevEkW+

2ubWBN6dD2OaZRlNaUlpmj2ktPoq7IXjUFLkt3bsf8P7ApSejoefn59DSdXPftHxf0UojSS1pfM3

b2h/sp/N5g/LWa3Wq9/075DQB/Yp7/V/jaTfM9dxH5Xj2O5696tW+5Rob+2sAG5KH2euBZbY9nK5

tKxlMzkg23HWO2/r7bwLMfM/t7vVqbVtakWK0++5a9kmyLYts6E4BXz3arfbeF9fX9uvCyHWek/b

2iSdKHHyb+4sc39w3eWDgkXYfNWIA6RxOx7E9nT/ma41K2Q9aL8JBJ+3AL7bulvGAeau8/H+AXp/

f/9wzccILCTY9gNgOgsOAAgf7vJRAvQib1cDsEUApU0XQgDUUwAQy3Vr8EoAFhB4twBad6E/AFgu

zdX284UBbChrq8tU1AmA+zcAUNdYMt3Ux8DwAbgT7bbbVwawl9BUvDIAZKJNJZcOEODcxNUuwbza

UgwQwLZZ5327pfA2AwawxOjjWL/UY2+wAJZrznnTOjdd58b6QC3oDAAmmvsBbMudzxb74wl03C9m

iFDrSy7MNsVw0G4v1AQAfGf2fSJi2yQlJxylrVovw65002Uzdw+A6ZiLY0LZuwLgewJQmhwXdu30

gMNZDUDPMWDNFycw/8QFfgRrcVrMrfpq/LnzOgK4z4UcxzUXhCaEW/8D2u9PCSWLeY0bAYBXA9DG

xtDbnQC2ZcH1F5f/Z4/beIAAHnVauNc7MaYjtiiGA2C5s6Own13+A0PANTjOrsMAAbyuAO6LAXf+

naSEX/6fs/anNPmeu9cAn9vOYuBOgJlwoOP+R+z9smU4EHqa1QBsBwZg2QuScv8X3vPDCfYniOOr

KL4B8NYfgDPf8wR62JfsR4BDku6vfGh4ABjCLAJy7xEAEAWEHu8FkPsEODGAY2UBkAB86DoIOgS4

M43mAJUUJPLQawEcXhzg8LoAx1srQLAW9xnESpM0WgWAjATtEKRRa/AAeSE7XAFgIbPvApgMoZU4

7i8AGrQS/QLY83+smbuuAuwO2+ALGczDvJ0W9ffcjGIIW8PvhXBQZwPN8VBq5sRAYy+Xwwewbcde

gLk4jR1KNYAscNp8AQDLcWyIgmo3sT+myT+7bibuC+D2XUrTgpmMd9T70jyQsLv8AwJ4v7kCMBQf

6CUAlOHDzDFfAcA03flRdNQXfYTjmK8A4NR11LyTuxOgneNCF0F8E8B23AUU3WuAmkbi5gpIrfVC

c6t02OPWTY0FS0KnSit6TAGgtg7U7Qu9tQdQHLdxbxy3sXFjKLkGSOq2hXoAsC3HMi0TT1/V37Vw

3XxfYl/p5qCZhlbiToAWXcjBA2dc5g2AWZGESnWAjzN2rwDsvBD0CblgKepdSMxklX0JvqlSswKe

t+0WYPFRqP7AUzGTlXfm+Dx27XXmavd1BdDWGXClfNiDp9FlbSWGNEpwBUo6Ekyjdh3AxuumDggA

q9wL1bsQLkEqbnDkSvDIbF0rgbeYugQ4n5n7uHlmDpoh8PgLkf0cvty+bPyctbfZdFWJywDvvwDg

XLz4qSzA6WdxPQ8vLROSUIdHzpRqEP8CAF3bfFYRxK9dEy7L9a7rM3PYC338FgMMAKYaVi5c1xX/

OpZd1/ht644aTFrsRsGFSgdfbwKYSz585adVwHy77rBH/WmVVgGscy9kP3NYCK6+g2X4s2MXmlvL

v5GJx/7WO+/rVQGwXoAD3QB4GzyAubScFdj72TmAa5rPe4+N5+VWO6/7o8ffpm3/zRKA/Zsezk5/

r5znCXAkwmM2Nw9/t5hGv7dr52kXgvyzglF+5/UBsN6tV4BgW7jJZtuNnqHBE4zQ/oD5ePk9z7t+

AuLLax3ga4sIrvOoVqv1drfdbr/q1SKAxAA+ofnF56XWDwo8B83fbLoHYN0oAHjb7Sd76svzdtvd

/RJf6+ELeNf+XwVoR3Kc7nEPAdef+cBnI7HH3eD78OrfcCGPP4Y1aceDFCmih/IE+OeCl96xB+Ha

eZIPnwU9Qfm5GYBPC2Kj1UcR8R0q96tdq2IPg7b0Tub8cdzDv1WLEo/jtvRENERBSCk5tihCwf72

HkqHpQ1i9uh4W0rjoN3fcaRIE6PZk/+NFBpSy+9X+Nb276RTWn8Pc3xjjPakvNivgBw1atSoUaNG
jRo1atSoUaNGjRo1atSoUQ/pP2EdU78Lv4x8AAAAAElFTkSuQmCC
'''
CALC_ICON_B64 = '''

iVBORw0KGgoAAAANSUhEUgAAAMAAAADACAMAAABlApw1AAAAwFBMVEVZWFiZmZSpb1+7wbm7hXjL

hXJ4gn1oe4PhfGM8REnc4NJek6rXl4lIRz1+sMPY0r8iIiKXu8udwM/27NSiwtDb1seFlZn58Nf1

knWNp7Bumq3h28t5pbnPalOwtrJVZWbwjHCFm6OCrcJ9kpmnqqekvciXsrt9q8HLzMJqd3XFxrx1

iZBKV1rTc1vKeWa1lpAAAA

d3bPiU7rOGAcPZoZaeaZEWQ//sdb9gXgC8AXgC8AXwC+AHwB+ALw3woAvj3L4P/pNcuaU3ZyL6dm

Dy9N9uP19N8AIIOOuB5l+33TPLt2PG6Ox/lqPl9ZauonbQ0n5/PN8TnbN9nr6fT6/wIge/1xyqCf

c9d+2cGo2bVau3/YbfebXbvm3q7Xw09Z67DvT/8xAJkb4vmww9iP+Hf199taANlj8x8AcHpe2dEf

utG0pldttDFGS1PusP8Vz8sH+TJlNH93td2CkPaJAWRz0tqop9At7ID00kDDvuq4yacMflL+B6Ae

gnwl/JXtFsTwmhDA60qpbeg+d8jIsMnhQVcQEnXQw1QEUgVk2gvDX8n2oPpTMgCZVdtttaY+4r+R

BpDSGH+UhKC8WPiUfJqUCcefIAXV8t+6rpQ6JQJwsk6oW+X/Ov/BeKC9ptBvI1goKBO1gEyHAx4A

ChemB5sIwFyhXfnuD2zW63jQYRlFHk/povGg1chK4tEYqOMxCYBGVW40SHNktJRXJ+6uwS4Y/6cj

iYzs4udzl4qtAo/YJADmyveflCEaasOG6VWJxjroihEN82pDsAxp162ZVZES4i+nBABe1Rb7T9+r

jTajdmtYYa21lWv48rNmbTVYUAbfp2Hie04A4FmxAYDy4wjzggNtXs3j9o/Bj/FhatXoZHSgAjxh

MnPL+jwBgJWqwgzvvj3qwD/GPf27jQaD3wddsncYwe8BWAdAuu+s9GfjyMdQMYat723vXuu67/Fl

Mqld67qamjvqTttlDkEjt0kA/GsdViDff+wYdsCYyZTad/j3vfiDNpuF9+4LzMTCKCjvIFXr06cB

vK4rS7MivM7/Uc1tZ6bTqA+zuBufbDPjEKCHAt6fk33zaQCZGwZxuTQIoC/S9fdG6514vedRqSwF

AOV9RlDU7/fpxhXK2ZXS/EQEzgjYN0kigSYAQAvof9WDWaRXgwODg78COptWAICVtvr9QvDHAMzs

54M++/l4z7y1xB+7geh7RdMQ2oH9vTd0HwBSSQQwnf1kzIurEzPptW8xSsETI3EfmM5pGsKolR0A

FMEAwGw2nIVm0TDfbOHTAQv+GAhi9p0A6GQA9s4jYdfYAagEwCyAGKj7jREPAGbF+Cyj9gJzrRIA

TuzrBACeeSEG7xElEBQ+HlX++zfGnWAUs1Ix45zXwaBbAMCwy3uHM/RbAMflk1B+FpUhnV0trKGf

AxueBSBDs5mNV0IBABBSABhL4PvIeuNffF94eAVPEel8cb2GD08wAJyI1utVEgnoKwCxEgwm9ZnX

pOJXFh3Widl4PeBZiDiNRCpEAFCFqu9FbL7eVAeT/dB4ryYe6e+siC71COYsAZi609iAlQgdANib

i9dsMKmOh7mIbLgYzErDj8UqpNmIU6jQWoggY50ERjrsx/Nq5MdTZ2TTsys4VwAoWE4DwBIAWolt

MRusYtfTf1GMh382do0Ga+EYQVV5Cag0ACoPYC0qdOXDzGbF7Wk+CKiYjbUnyG5gAwMAn7eBDU2j

qELrYAPjMb5yE26pWWQR0UIykoCllRj4jyQS2KitMGUEYBavuUMNGcxLV6Y8VJ7B6jw2Yu2N2KZe

iato+Q/We+0ejyaZgYoV4kXQLyOdBAAi8yoZAD0A8NPAfByfzEZmfEsqYwnMEYBmYmiVVAJrP40G

N0AW1Gja/DX7MEYwdM9dPFCBCiGJppKo0HYIYDqaA6+myiig+VUQPLsdXBIAxakR+/s8zd8GUMxm

P+tPWJ//gBYa8CqKOfz1OrUNVPPvsxvqPpvdQUzMrvt+47J6CCBVPCBBfTWOie+jq/7gc9YF9UpL

zjKFEa9jd7oaeXNp24wEYH2uIAmAbUUJOxiWuXNU6uJGxFuM3R8/9UfqNItcodnA/Z4xO2r6ecXU

Io6aM+Ls8xLY0sIIimnh+6veeDLXv/6u7fAFGyQxJj49QjR1T2UT8PXztVY+banSRWSUTa8iYp0S

HL4Jzc9Mf3VnZmCYJYDyi5ADVDaFEa99fgO9k4r+0LjT86j/gwOcvKgq6eB1pmnusxvuDXNCHJFt

UwBYq5C3hgTTOuriaByrm+kP7vwtmJVPiKyj9KykEpNMo5sgARXyj3+jFkWP6xAGuXIaHaN89wnA

1qazAa+aw0wlp/20oNMh/R41TtrjKW2Mvg2QR4YFbiBDk86Zo24OUqoDYQwKN4blG/LGsHqPCkKM

TyAbEwoUDGVoEs1C2gMYlMdQ5l5530vpW0l4n/FiH1lJbjw6LclyxdIhoSWJBzZIqxjKcoeiAB2n

p7V0hwMpOsDjGpRC66HxXBUfeADIICQDUEnFkq9l8EUeokJBv7QYe1R0IP1ExjwqVPB1LlyDoUSV

pGYlWUSm4wIOX20SGeiggsbE9SixIcSFNaEszQMNFRVKDKlKuBJrb4WmU12n3H/5676kJlRPeE3Q

sb1HE5SvUcOXDr6s42/kZIRRqQBYpYPFdpx+X1Gz1qsPIuulReMNbzsleXssJu1xkiFfKP/4yL99

e3y8XB4eHx+/fdSGAMMfTQaA1xll+k05bA6DmIFd4YHBGU2c6mp01WalCEH3cTm/D9ulNZ1OOAsd

JUeG0qX+W5vnOJzYYUvmZ49jbOWqozVOUPuLHDqFyvMRdf/M7y81/zEHYJuG3NUicPjjk13sJtfu

SGfgE66Xtq53uwO0SV23gBQtHMbfhst2uxpPgQAeXIdflsu35dvbEhuA+NDKA5gnAQBfhqbpunIo

FtxcX9yr60wPpx221v0+XVBo4t7sXDdxBi7hFH56ilfhqRV0snb9dX2PGiBwADqdzljnIAGuu+2c

vuAgum7n+Q77dMBxdgA2JXXPDX3NH8rxFADYLVhaLUmhBdjatE5x3obtxQGgxV2l8YU2wZWArtQ4

8HnTnJr9BEpkCIAxKwDgfs+brGmOMs4WSoxcv+HXsoFTex4ABJA7BeKRvzg1WoIyvb9/67iqLuk0

CtrcE4BF7fpRZs1+x9rgAHTlpmRoucNQLgSAC2xLktvTaZ+X+ybHb9iA4gEA1P+3t4fs4c0DMOyR

JLIBK3WV2k2iKIG9G8eidV0tUAKrTsEwW+hz1pRFUeI4RwBAcRaHegqnEMABJOAmoQDg9PC2RBm8

vz92vG6qbYKAZr5e+5WRAIAAatf1J+zLlCSgWL3bEiU0ALDxNryr903JAEC7PIC/nAT+QlV6AwBc

nK221TyVEeMsxABy6l952oPWE4A+9BIkdBwCYKt2VsA2sIsBPD48PGRZ5l4BgQeQaBoVANqrUEEA

ijzbQ39hIdBgHq1M9K6f9dAGBIAz4gzfowopAvAGvaf28EZGTOU2WtltOhVCAGUkgSInCdijk4AW

S8V5CNXkGgBMsWA+izEAlkD2cHljI+by3RQ5svnWasP+vyUALelBTrMlGjFgs6Qce2fcNOBTWiLs

ZoMCgVbUTXZY4DRqBQCYLs1Cb6xCYR1IAWBdKcNbGBjAAYwYJhyYhUYADhlMTgUuuCQBt8QdQbsO

ubvoFgA03XgafTTM4qQBgOsAOZwwIfI0Wu+OTXNAh2EDKuRcIcvj/9y2uEzDKYv2DQCK8tSUdb6n

CaCOVUgkgKJwAP7yWzxSrMQMQHt/hxYyaO3CL2QKl1s3wE4ATXZqTiVh6zUA2AC2w56uwrnXfZMi

ALwSvz08QvdRhXIj/EeVJE+8lm0Dpqfl1s324BOQV0GuhAOwQl/ouKeGquaGmb1pcu7ccbZn0DsC

cEYNeiM5uHZBX0gkoDaJplHeION6uaMJpq5dn/4N0jgSgNWqnJCvyRbs1mo3zLCGl+SoLgo+MS1q

EJsD4Jw51qGlOKXgTruIJp0E5rh7gAE4dzo/YDfATtnDBEVH+8DRXXinGU4hAIgH2sOOVzln3BQP

uO9s3XifB/60O/DeClmRxp1eV7JrCX1mB6FtWyz7blvo9abswMAp2LR8qqZTFlm3XuI4d5YjMmcd

4MI6b875/+cLN4rJ3DpmUtrA2irh5FxA4PqycS2Ob3uDnI+lePjoIsv8mMNHNpYCftOvKFjGoBOv

xVMQkn0bh8TvD60JvNB6kwgAk1pGIR8xiOkV8ygdMRUlvbpTwD0wq9v1gchgLkPI4BZJiQdpwErI

Vh2TxIiRWmTSMyJze3xFJkeoLgxyFR9nYoFW1MDJd0D+dCiY8GUdNtqLAMeYQEIb2CRaB4gXvcWc

C+fF9KjfbihKi1yjuBg3cPRCcXmiMt4sR/5oupWY6D4h1DxhK1s/cZaK91IGAn64xUkPzjK7HtPw

hqlh+IhCFdJqtJswYgijrZXRdkQeUdmKO7hK0gU+NzCkJT3vXiUoPWYJSMeMIpKwx000PQ2jZ75i

8rxDkl/TFKV6ugBe3Ck2AkDROcfpQ9pfbW3i3YxJAMAuMmGUjYJ51E2kOJduaBYS5pRneJme3KnO

cGy74nl34+cuZoa7j8twEj3/1XaSzzLJXImQDihvNEq6uEVuVTK76LAdceLv0ca7iDH15CggUN3H

w/utdYAtOw2ArVViiegv2MOBMu47IBBLckZ5ubW8DtfELLpTGq9albaWq3YHZh0d6BaWsctS2gsu

xS4c6HjDRQoVgoVMkm/GrVAtuEDstC0oJO6ZmcvIIDGPAK4HAHgVOkILjHQW09ZBcoEveqPeFQJ/

DpjFS61l13oSG1gryd9pcDl3i6lnH+BnG7hRom8PMP4H5g+RdSRmDv27vGVSYgVXdRgPeAEs2Rtt

fSInDYA1b4MDABK57471QlhSdKeRWgRYTxS2AIIJxY0u1smR7YWQ5oTx/pSE0xEztyRumqSwBHea

04SJZiFJwmDg21K/j6cnUgYhdwnbYrFzkQ60XaBNOVxeuEi02VMYBBEZSCBQi16NEACv4IlsYM3V

L55W8dzcNHBsinsJZ6a7A3bYB/UYiBZAF0m8446BBL6N2XUMKT+U8ir0nEiFKMtohZ12ApjWk2Kx

CAAwqHdRWLYPgUsAQFRAWUiCoEUVMgBgeQ0gCimfkyxkkmYVADjMGTJUQwAL4HWP+5x4oCnlYZiM

cRc97ffHFkUYAXgjZgsaUuwEQNaBYyoAtH+AAeSnvetPtvOshPESKE8N3Hmn5dA9ApBnTQacBZBJ

EEkHFXr4p1CLj14CCX0hD0BLVyZN0xZtk02LgQSQcdnlZV0S+eMAQEzJV4EJ7Oua2KRpSwAeEcAj

jX/28LBcRgDQnd4nWQdUBIBI5uMRppTc0yoMYCH8YYPGTsQW0WFw1aFY7PYZmH8LRqyNN+JlIFeI

FxIATQJ3Woo1IAdwAu4KskwMQIVZaIHL8x4B7JgbJQB504DR7E/AbI1noWDKEQCdRgJbK2U2Rnjm

tsxbZ612grPQxmrJ0DiteaqBX88mQwBTZFRLsn8HIL8CsGQREAC/n/g5CQAdAKB1uvmzyLMndIlG

AJxY0FIDaaftJm/J9Bu4i9WeeOsSorhvfiWW8QdiqDVSR5QGAPNCGtdUzmIcFq1bCsSV0JRLxV+B

Ad1TCpaSAFqSNyCYJjvyGg0pZE7yeSEAlDMQW1IblcQGtr70GFbiXDzRHWkT+0JI2u1ghnfO3GFH

7l4NDC452lPx8w64eNTIjQKvFeWJhdu9pAUw3/q9lBCZ+GQLp+p3RLIhf4gEKJCOIR3P2WW0fZqk

eIGg5PLHe6CnxYTfHzodbGCfBoDU5wG1mGNoMuWIBqIrjFp6IB3dCLtwZwoFEzURoIpm33LV4ik4

gxeRD95CBHN+eXmBaOblhSpXPqTsNSEAX9C2uo4oNxQ3Em86qldBPsmo6zMbKtlBHRrFxO/

faiZb

dJrbMogNUFzZWY7qMaR3/1ccr2kohUJ4myPHxaWL3Mn4lTuzKjfCBjhkPTNX5uPhfD4HfvTy8NG5

YKBjwiyhEQvZZDq6FYrcN0XqJKUAkM/6mi0uLYMatVDM1UmRnXvT1W3bfny03Gq+I4NJaMRWyQ3L

fE0I36us64QKGtCF0efj239pE92vypfNcu5kVIEpie5ks5C/q5qiwuPBDY18QSNzPTFJF3oT2DFh

6w19k2YezN+XS/sYPA2AtZIKYV+KGxduE3sY1WOq6CZcoeRYxyWZKtzBLS4EV9qzxXT3qxRG7ABo

LmXlQe+6wB52JtwLjzWLT3vqWht/kXvpkFgM/eaiy66riWTnqkst9UJZGleCWVH30kuegtjDle2F

3sVpKE5i9LwXRs5wJSAQi72n3+uPb98eL6F9yzuZslVSAFw3Z6+qLkvKg4HHMEg8wYxpUe26TrJL

ZZ7DPzf5unkUWd/8mlr86NjOTNJZiDY3AknoXGmgriacyVtxaaIaJPkw/7faYEUmZSmJcJww67jC

UkddP1C95flyPst6cG69uSeSgJVtWMjMOUeTqUP4D/Vac3G1iZpDZ4gysCtYik3noE3Yf+JT7GWg

L+S5RfRG3VL8EXYSJDJiBqAw0X1gb/TAP0pyOc1K+r9jYnExzcGX0LETLqeoTk1KDYiZC8UqHZce

gwROiQD4eiGppMmz2juWPdf00u8Qz5Qx+dOzC7t7gminZcZoBeRu7qtVfHPuREcLXqp14F9WVqKo

LOvIRU1cAWrA6aeQksIurOioMW70ZMwebndMESVFZMwL+WASpXAGALQRIqERMwAJKafHTABQ4YwR

bLt9Y0EKTxy2MIADlkk1rTufYdWic12VxMSiPITDAajllqRpVIgloAlAjfyhG8tTDEAz9TDdZU1d

FPb0NF1ItAlV4QDAoSqk3m4MwNsxADi3kt9MZgMmAoB04lOdBwARewW1mHsojCP6jQGQBMrmqQDq

EZl3WzKAYanHkoJ62u6R0Ijxy7QnCWE+PDKAaZ0Te7VBQrSgWqi9VLZbCshQhbKmnFhOHdjNigHE

6gNW8PL+nsveonUqZ45WTS8BShCUBAXqhTDFR9hKym+UbAMWUh9c+80VWxlJYACAinbxxwuuZEkB

VBRzeZqWzVgAUBk3A6izJt8dmNhys5ClciGkgaflfn98Jl4IJaAZAGdoiF4EAEbubppGhaySoGss

gah8Txjc7Ij8IcyjC1ahVUmrH1ZsYVHvAkvOBgC8Ib8wtciuRAIAmKHx68BAAiHJxwyuA+AGuHD
d

bBkbqdCB81FY1FsQgCABrPl7iAHIrrokKgR1o3owjdJsw0aM9YeG0zCQBMgPh7xpJoSNPMBygtTi

Pt/T1gJyJYIE3t4es39eAgCJkJKFlDIivFqBQj+dWnElcDMJZ/J2aKmnLBdsITHlLjk1DfGRE0qP

Bwk8Zlg3SkVzOd7TjtzpNLSK7GrrQ8b3QEsy0m+d+EKwkkFp65P1G2VUoIQPwHz55LIdA7h4cveD

I2+4+/cpiQQ4Ikd3upYshtcg2o+0WZF0FuCnYjKvpaJkTy3C9iDK8h9KpLRV94AAYA+ck8Dj4yPv

odG8PTMxAKWYWqwn0GSrFZFsFKvZGpldZBYtFUS4SzHUaSWcoTMb8E+6CwB49CkykIJzJXLZepps

HcARQTsob5KEGACuNiPecQX79QB6f00twtqhdH0BV+Ixi/cPnLn0WNEN5D8P4MjMHO/ItatRRNzL

PtSOmcXNERnEElNPqAmdu2p4GVyIO9MhgDenO6hCl8e3JTpzDABW4gQRmeypN3jb5uGGT6ZVkArp

gDrphXjsVWCv9PgeAnRVByExO3M4C6ERnzlDw+vAKZkKKaMH99rXw23dA64qVGcyFTl6UoFmQri+

0Cy0jKZRBiC3bU5Eq8iTG1S83/nGvv/4oRp+3HFWj7fYa96fwQCWCEAWMgKgEy5kdF8JQ3Vgg4LR

znNxxK5dFZRy4aviS/WgKjQAWC4jV+LMvEoyI2YAUp2rou3QYY/zyh8KR63ydy5Qanw3c3omBAII

aUqOidGdxittwnVAJqFQfbgJHNzmeOR8h5Byfs8w1UnxoU1c8+cB+Iies5TMbOn/AABmFrGOXhIS

dTtqKBOgEbnmD56Lw7yijVcPAHAmG4hT9ZxmVSyBLJkKcckZk2wLn6jkLcTkLocdHLtdS5k82oJi

D9E26omlUwRgyfEYs1tniciAmEu3kJEG9bgzeDG89RE6Pe10MTrIbhJVGeWj0zl5GS1LYLkM9R7n

90urAoDPqxDeb5TZVr8rMu7qYX+CR2EdFuMTE94o4+u8/E5wyo4zAM9reWaultyIA/D6eQDryi+q

wC+Q0kRDvW+y/Njg5uKREAIAPMeuHG0+A/vQLc1Co0z3peZ9awAgTTygwk6+XTSW5NqfsrpY8Obi

UFGKIRjttSIAFOo02Wkv2CDPjbNQoOYYgC8XSqNClSygEN1685XNSiVyhUV+OnrlF/lY6CWmN0AC

T1iO+fRUTiUmNiSBsAwsAwB0p6vKpgOAt75wYePC1xdTL5+I7MyZzIqreIsIaDwANzH5mQsBDFZi

X7ZFKsSuUJXgGRxoA7xjwfott7EJlAVWcT1d3ZrNUl24pY18VF1xoJLSFVBekS8U+F2g17UUbG1T

AeBEtevR1SSUA5ErOK4AgORA8cjYaS8l7zTWVxIQG+i0t4EEAJ7X9LQeHRlxDKCGdAYQKYdQ0s7v

OKRBbhT2ueJG0SZUCupuAAB/vPh7ApAEPj8LwaOkxClGXujfo3E+nnBg7QjZvxcT2iTg+bwDNaF9

DQb1nB/w1euSpgQJrO/YUn8XAEkQYO3V4UrVgc8lIihuBWbyWG6wfvuGAkBnCIq/z5EB0FbKXB6G

Vd2xgeP3AE7wPZwgQF+I8nTT0KFiktfTwmcuF0yduFWAd6ds8KpwcxisksK8LWylPIcdHHhrmIfa

lzvdUbh7x4N0bFXJk22cM4R+JjijddhtMqnHrbU537hHRCBuLBcV0RZe3AExqnciT0gANJ8H8GO1

3iq/tcuuyjvbqlxZv/VmfJUveOo+BrtBz5dvbedvEl1VfZYAwFFVgd41qpc9n3FfQ6f9Wwy7/M3F

+mFA5oM11dWt30WWO7l2oTanumcWvQNA00c6pHV8m8E+etvTr/3oNnJ+EdSypVTS/tHNzsyNWzjq

+2z4nkfabegmmqF6aXirx/BMwVt3+dPjG/7J0xjU4PGB5vpRodU9vuhdAOBxUlX45uGdB39zZ8vh

7s/r213+5CGDOIfesZX1x72PdUQZ/O6ZhuNf9A2uaPQ8yBvXyWOrwAJeEwGAh8JVlTzLMGK16BYo

4UanUYGcjsvLIiXS4fGzQSWV31DKN//EG6mqdI82JQRVNXpkZrgdofb7jvxRv5kt3qqrBxdGO3wH

Z/Fvre19XbvvUz+OvftaeD6rvVaecZ9G70eKon/5fOJqW3H3j/d17P7H+x6h69st/oEtNXhOMR5Z

4xOf1zefW8zH5JU/hZdWePl26+++K0+AtdXx7qdE3wsAFoTnTRj/dQxgWwVY1EM4yN3bundbO
l0N

O4tH8IMBeW/nx+wPOvWnz+gGl/64kYdz9+qTDR7n6Lyj+Xy1OT4/PzfwNPUff9ijH59or9nra3Y6

uX/DBgdcX2DbGEZh8BPbCf77T7n2+uOzLfvxP96+AHwB+ALwBeALwBeALwBfAD7R/g9pGhP
DfViu

NgAAAABJRU5ErkJggg==

'''

FINGERPRINT_ICON_B64 = '''

iVBORw0KGgoAAAANSUhEUgAAAKAAAACgCAYAAACLz2ctAAAXNklEQVR42u2debwVxZXH
v+y7PhAR

wUgUEaNCQEAlGjGKIi4wQWUUNz6JohONS4wbRh0XRhw3VCJxiY5bXOJGzChiFBMN4gIobo
gCKoso

+gQeKCBS+ePUG9/ge111qrvv7Xtv/z6f+li9z3dt/rXVafO2sgYQ44cxULjfAhy5ATMkRMwR46c

gDlyAubIaBvgTeDvgTL0/wQeBTJ8HcNGQF1ZgZgObaBfAL7AL7AHy7AL2bbBTJ6n6uB
F4Hn

zgnYEehhyfYzoC/wQ6AT0Dzla68DvgTeWYCr9/XmpJWhFoVGGG6J7AvwH7AL2BbTJ6n6uB
F4Hn

gCnAmzkBSxP9LeEOAvqV+G95BXgGmGxny5yAGURb4ADgBGB4mc/mjwI3A/8AvskJWFzsD5
xmZ7tK

xKPALcDUnICFw/bA6cAZ+V7y/7AeuBWYBLyTEzAdjADGFlGn2wisBWoQ08paoAliiqlrXagC2g
HN

inSfLwKXZ31WLCUCHglcDXRL8RofAZ8Cc4HXgfnACmAZYrdbi95+18YSsRWwBfADxNxTuwv
vipiB

0sJiu1I8lhMwDCOBPwDtE5b7tjVxzADmAQvtrFYMtETskT+25BwG7JDwNWqAk4H7cwL6oTvwt
P2b

BJ61y9GziOG3FNAHsVkeBgxOSOZ7QK+s7JyzSsC7gONjylgFPALcBMwuk01GHzuLHWOX9V
C8BPwk

J2D9mAIMibHM3AHcDrxV5jvePsCJwH8QFlSyg9VxcwLWwV529xayvF4P/C+ViQOBcxF7qC8OsipO

UZG1cKy9lOePB7a2+lGlkg+r2w62u+lrPL8zlws3njUCzvU87ylkWuUCayLJlfgUOAcx+VwYcd6d

ZCQcLIs64AcRO99rgUsQA3AON1ral/SsOpuWmxC7IDkB60cXxEZXVef/PWIV7hU5p4LR0c56mQpc

yLIdcLDdqU0FFhTpHroDW1o98wdINPTmfBcN3cqqAi0QF90G+/kG+BzxnCwHPrafD+2/a/L3IfsE

LATaIS6xHvazE7A7hQlUnQe8AbyGuMtep0QCCHICBv5WJEB1H2AosKOd1bKEjXYjNgV4HvHYLM0J

WLroC4xC7GN9S/Q3fIKE5j9oSVmTEzDbu75/RyKif1amL9V0JKDgPiSpKSdgBpbWo4FTyYh/s4B4

G/EA3WmX75yABV5eTwdG53tJQExVlwFzcgKmi58D/03y8XJarLZLYA2S49sEMck0QyKityb93OL6

MB/4HfBATsBkMQYYhxhU08I6YBaSIP45Yn9cZh/qB4hdLwRtEJvidkji+85ITF5PpPpCWjgFSVbK

CRgDJwM3pjCbzEUc8s9Yc8cSO6sVGlXAtnanvh+S67J1gvI3Aucj6Qw5ARUYBtyD1GZJAjPszvEl

xAC8IcMvXTfEIH4oEg2dRDrCCuCXSApnTsAI9LSDtHNMOcss4Z5E7GiljF2RIN0T7NIdB7OQBK8F

OQG/jwnEy/NdZnWePyGurnJEZ+A4JMIlzII9Efh1TkBBP+AFxLkfgocQe9gMKgsD7Y734MDvfwXs

SRGLH2UhIPUaxCGvJd9GJAy9FeL9qDTyYfXZQ6yePD7g+60Ru+GVlTgDdgGmIUEBGnwN/BYpzJPj

+xPK2YidVItXgEGEm5pKioCDEMe6FqcBv8955oWLgUsDVpX+FDCNtRhL8KkB5BuL+Hxz8vnjMqRc

3UQlH2YhvvWynAHvRZKqfTHNDsanRXiAHZF4wTZIyd4uSHR0p3pe3tWIS+4z+1ll/73S/vvzIpOx

N2JT7a34ztnAdeVEwMmIcdkXx1nCFgJbILm1e9md5Q4kZwDHknERUvt5DpL7/JJ9sQqpcx0H3

K04

fxLwq3Ig4FSkcqkPZlkypF2oeygS2HAoybq+tJhtZ/q/23FKm5Bdgb8h6Qc+uAPxoKQDY0zan6nG

H5elfC+HGGMeMNnGXGPMOGPMnimPxQTFPd2c1n2kTb7Jih85JKV7aGuJXaqYYIzpkdLYHK24j4vS

uIc0l+BJSDiQD9IolLMdUiH0GMoDnyDenkkkG7XTC/8g1pHAn0tBBxwPnOdx3ud2d5lksnRH4AYk

GaIc8T+I9yIpf3d7pG7glp6EfSvLBDwKvyqc84BdSC4sqpU1G5yS4oNfhxQCX23/trZmmmYUpxb0

X5AaMEkQoq2dZds6zvva/maTRQJu77mUvmPJlxSORiJgksACxDn/BlL2bRVSzWC5JV1DqELC8psh

Uc6tkEoKvYABSOL7dsQrLBlFxDEJ2Eub2VXJZYJ6xloqMkXARnaGcM0Ey+sYc+OiDfBXYN8YMqqR

GtQvWvNE2rVTWiHxfX2RmMdfesw6vrgQ+K+YMlraWc6FY5J46ZMkoI+tb50d7CSW3b2REK4QfGv1

xIIlcfJio721Sx5KfDdYNRKe9XLMDZxPwGrn2LNuQtvpkZ5b+XYJXW9coEnjbWPMgQWwfcb99DPG

TDTGfBXDfHNtAjZTF57Lgh2wjeeA9EvgWo2MMa8EPIzpxpjeJUC8hogwPZCEc4wx28S49i0e19i7

2AR8yuMmL0jgOp2NMWuVD+A1Y8xeJUq8TT+drEciBEfFuO5ih+xFxSTggR4/fmoCg99fOeAbjDHH

lQnxNv1sYYy5MYCEZwder4eH7MOKRcAax419lcCAD1YO9K12qabMP+0D/Nq3BF7rQQ8Vp+CuuAFI

GHcU4rYCGImUJfPFoVRetfy+1g7oW1RzMvq2tlW4K3E1Crn5OBHRLoPq/THJt4+CfO8glRMqsVXD

bCRw1jd4dDj6xoUrgKsijr9RDDtgZ+u6SfSNsOiJf8uG+4BjC/Cg21n72K5ITF1H6/HY3BrEm9jz

vkECUN9HIqI/sL/lQ4cnJQkMtl4KHzwKHK60VVY3cGwEod04Y+oh1zegE/w8hsytFDrNpSmGcB1o

bXH/tJuaJPCF1VGPMMZ0Tene2xhjFipCvTSyBxpjlm8iY0yxzTAX17mZj40xQ2PIamaM+cxz8E5

N

YXc5xhjzfAFj/RbaI6h7CkR8xvMeTguQPcAYs68le6YDUrUfX4PrqASvOdRa9LMQCX1UwuM5yf
Pa

+xTrmWeJfOMLTL7RxpilGYyA/tIYc3qC43q153WbF+O5Z6U2zHDgcY/zTgZujXmt/ZF+xF0zvrtd

ZwMTHktA1kQkHzsKLwI/LfSPzAIB2+AXYh431Ki5tZcNobTwvI1u+TqmHJ+c7ILkAmeNgC8jxRij

EDc10DdKW4uvkBZc6+oxQW3L/+93Fxfhpo7vMAt3v5Ru9jdVBAHH4K5h/La1vYXiNqTRYShqk
Mpb

r9iXZS5Sh9CnYUwjJEB3AHCEfdHitJK4ATgzxve3wF2l4U10FRRKloBRhs262JLw0havAz8O+N
6r

SAbaa9agnCSaIjX5TgKOD1yS4zTh8TFWJxLtXAhDdJzPwx47s5GBspt6hBFtipU2bKxjgcdhtDV
Q

azDfGNMqxjWv8rhGs3I2w/iEVz0Sw4uxTPEwq40xIzJghhqtjHdcHtN04rrWxeVshvkYd6fK1oE7

v+X49xIp+K7PA1cjBTh9UI1EwYSMU1+7KYlCi7T918WoD3iUB/mGBA7qG57km2F10KyRD+Ac
pEjk

tx7ndkCqbIVgNu6WDRPKcROyge8iR+rDs1ZRTsPYClK+9jyyj0Z2M9TPc6c/JuAaVaQU55fVGfB
E

B/kgrJbL/p7kO7xEyAdSeaA/fo1lTkLKF2uxArjJcc6lqb5lBZ4BFyK90pI0OHfCLzf1ACTxvBTx

lBId7sKOAWajxh7LfWqzYCFnwGEO8kGYt+MJj3MOLmHygbSh8KlK9UqA7I0es+AR5TADzkdq
xzSE

c9E31BuB9Mktxk53K8Sz0AKJlv62zgNda2flpQlfcxruMiQhOq5rFZlDmEE/MwT0qUEXMs27TC4
h

CTgNoSfSHHugfRhVHt9ZhVSumo0UCX85gftYj7v+Tog/11VaJX4Zjno13clYWSelYPQc42GsTcp
A

PD+hWL8NtqxI2xj309vjOlNSSH89q5QN0a6LtLc7siRlDvPUDxvCKCThKS1cZ9WDEJzmobcNQ
HzZ

Sc2uC5Cm2yW3CXHpLI8HkM8V3TljJvnuTZl8AL9BImr2DPjuRGCx45wbAuRGBftuj2QFlhwBx6
Qw

UFeksJvGvv2LKVxd6baIJyOkqqur58pP0HfRfNhxfL9S3IQYh5K+uVLeHkR3xpwWOFDtrOJeRXEQ

EvH9mGOTFVLJNOp5PY1UuyiZGXCA4/gdATJds8UFgff6XhHJBzDO7rKTHIsDkCLwGkTFCiaezpA2

AV0+3b8GyIx6SIsCTR1PoOuW9JLdCAyxOtwO1kzTH+nfO9rOZtqSFXeii0b+FLeB+lzlPdzmsQKV

jBlmesJmEldjlTMCZB6rqGoQEiDb3BhzijHmA8/rrFHK/5FD3vqEy7GdlyRH0pwBG1mjbchU3xAO

cxy/K0DmPR7nnIR4PR4KNBz/wc6SPvpTa3QdzN8lOiSrGbomka4uB7uVyhLsanoyJUDmoRHH5gWY

c8Z5nDMIuD2hMXkasXkucZx3viW8L5L05W4kOlB191IhoMtmpHUVdSe6JNwtAb99rOOcg4F/JDwu

K5CUzY2O885RyLzfIe845T1G6dE/JMGmPGkS0GVg1baFH+o4Plkp70SPWeWplMZmI9DHcc7pSpmu

PsC9FLKmO47vUgoEdA2wtjnh4Y4HqpXnmv3OTNlC8CbSI7ghtEJnSH7AcVzTO8+VK7JTKRCwZ8Sx

jwLk7eZ4mBp0s5+GcKvHEpkEXL7gXyj1yyjso5D1kcf4ZZ6AXRLU/9oS3b9MG2zqsmXdQWEwk++X

9agLrUfno4Tsd2scxzuXAgFbRxz7JEEygz7qw1Wa9lUKh6i61u2RJoe+uDniWBMkiNYXUQWjOpQC

AasSnAFdSq82FD2qtMWLBVp+a+GK2tlZqVdGQWPamR9jQig6AZs6turaWi87Oo5/qZDVjGgbZajZ

ZXM7Y2nxnuO4ppjRspgrie8z6pR1ArZwHNdm27sS2WsUsly785kBG5ppiH2vGikepNGRXKuBRnf7

LEHdbW3EsTZJzlRpwEXsbxMk4BJ07V+7xSREXbREIoXr/t5BVkYH/ApvLrHj0VC+dFJ6G+iqwkbl

bzfP+gzoIoRWx4qa8rUlPKocx1cqZF3SwBg2wx006ztzaTqs1zjGtrVC1pYUAGkR0NV1XOvK6ZyQ

/uejHlQrZEVVG9XsXqOaZndUyNIItD9ck9XWI8ZSn/kZUDuFR+kca5SyXKVB1iU0fjslRMAWyuf0

+4hjjyvkRNldF2SdgK5ltlOCstYl/HJochRWJfQb33Ec19jdLkbKGm+K4/FPlHepKfOTIknTFAn4

RYQeoXXlrI0xo2kJqBmTKIN6W4Ucl/1uO3Smq12BI5F8kYVIlLPG/dnfcfyjUiDgZxEE3EopK2p3

p01qcs2YLRUbEVdqZFvPnbBrRjkAvXfmz/jVk6kPwx3HE6ubneYSvDzi2DZKWUsDNyja+9Iudy7
i

+G5EXEvw2RQWru6j00qBgFGk2VYp65MY+oqWgD9SyHK1lB2gkDXd8VIcViDy7e8Y06dJ0FW
ZJgHf

dVxXs3QuijjWTvk75nnoT754y3FcU/XAVYjyLwUi4PmO43clebE0CehyaWlmGpe/VLPjXEm08Xp
g

grPzIIWcaz3OmZIy+UbhTqV9pFQI+KHHzs4XLqVX23gwSufaWynrhYhj2yt11Msdx4dY/SuN57Y
H

7no4E0i4an6xdEDQVVpyEVBb4OdJx85V83K4+rcNVcjyKc2xL9Kj7pAEn9UpRJc7qUXi9aLTJO
BK

hwmir0LWF0RHvGhLyP7TcVyzdLoSl85QyFqLlON1oQVSVWlhcEKM5zjcqjeTPM69EH3aqxNpF
yd6

joaDPz9H5/COKk+7EZ1BehvHxkZbhOdtogNHd3ZsyjbF3ehTKd+wpHzO6rjVdhJYg9h7O1l1YHf
7

gg3G7RevxTJ0pUv8kXJpjomOMg9VClkXOWR1V97bxwmWDRmbQtuxaRnq4t4tLY6kXZzI5WL
SdH10

7QC1lZtcZg3NZsRV43AEuuiY2rGZRfExkARdb4XUAcGd76CxublcUdoly1VuQ5MSuQZ3e9N7A
8av

H4XL0GtIF56R5gWKXaDyeeUs+Lxjg9AEnZXeeLygvgPUE7dnZDhhBuVRdqOwWYGItxjx4ixL+
0KF

KNE7xWFS0ASnumZU7SzoqoylKdX7Hu6SFpMJa0fxJ8RzNLYAz+saJAViWSGYXggCuhzX/RW
yXO4q

rdP+Vsfxi5TyjvU4J06vkCvtLH8h+khwF65DfM7nUEgUoEdIP8cOa4JS3vsOedsp5VU75A1UyrvG

Y1c5LqGx3dMYM94YM8sY85VyZ1ttjHnCGDPKGNOyWI26s9An5B101ZZ+Q7TfdCy6Ao8XAZd
FHJ8O

7KWQ1xR3TgxI0cvbEx7nHZFIo+ZIAlIbq+JsQFxoNYjv+kP0udkluwkBKT8RVelpK/wTXboSHQiq

rbzfHHeQ6p7KpXM4fvkXIc1kygqF6pbpcnJrXGlLiK4tuBI+rU1rsd5jJrpN+Xsn4xeN/CoJVhko

RRRqBuxIdCCothvjMKILUs4jujzcpuiCu2xuiAmlBr/cEM0KkBMwEK52rVXoksLXEZ3eOQhdnZeH

ia6a9S36HJruwAee525PdHpmvgTHxPWO479QynPZxCYq5Z3pON7Ew2xT30s32PPcBYS3GCtdFHDL

3d5hFvhMKa+Rh6lhf6XMmzxkDgv47acqzCMzjTH7FcssUq5mmFq4wpa2RmeBv4XoZoiLcVfWqos2

+KVRtvM8ry4uRhfQOdsahx9AV3wpX4Ij4OpnoS1+/VvH8W2Uy9oa/DpXTg/47ZcBVynO74u4Cr9B

wv5Ho88mzDch9a36Ecdao692NQF31LHW//oC7nCsR9BHYgP8J1JVKxTfIDbJmXa3PwfJzluhnHhq

y8etrRQd0NXv7cpAec099Ko/KmW29NTXJgXe88gUg0fXG2PWGWNW2f52y40xKyPOn26MaVcsHbBY

yucIOzi1PslfxZT3O48H00spc5jnA78i8J63NcYsykjE89xK2YRsimb4+U3jLu0hxmmAG4Ffe5x3

ud1khODSGN9NEj9FCrSX9SakPn0mKbjsiDuiy1ADaZflUxToIg87Z0O4BEkWurvIz+KgYly02DNg

0piJu51oZ3SVQrFmEJ+su2nom8vURXskpnEMBSqRWweh0do5AeugB+7aL2+i60oOfr7iWlQjUS5x

q4juiuT8nlAAMi5BX7GsLJbgpPE+ElIehV7ABUq5SxU2yg6ICy6uW+0tJDq5E+JTPsTaEv+Ovips

FD5E10kznwE9Xqr1Hkvm3rgrJGwKn8SjupiChIbVpPRbuyORRI0QL1IHu4y3QLw6rZAkrbX286W1

/a1EUi3fIsWUy0olIEihHVc64Ub7gLTFdjqj73V3PjovSMWgcZn+rpdxF/ppjLvFaX1YhsT4LVJ8

Z7wI7ZE55SpjBqzFx7iDEa7CXZSxIfwNqSiqVfjPROIP8xmwzH+fTwWu8xBHfwgGA+OU3+mKhOsv

B04lwb5r+QyYTYwEHvRU6ENNJ72RAIbQygVPIgG0T1UaARtXwG98CL9I5vmE98Gdg2Ti3Rf4/YMt

CQ0S3ZPPgGWIObjtXV8juSlxytD2AaYSz3h8D9LZKCdgGcEn/xf0ifIN4WzcRvEo7I6+OU2+BGc

Y

65GIDxd2xj+RKArXInbG8YHfH1AJD6WSCAgSbnS0x3m7JXS9tYjbr12AbrcuJ2B54gHcdr+5CV9z

NXCWHe8TPeR/Avwx1wHLG1dTf1LT1+g6i4eiJ3AuEu3SZJPr98Ed1ZMTsAxwBVJrrxar7PI7v8D3

sQeS8VYNPFtJD6DSCYjVz3ZBMudeIkdOwByVg8b5EOTICZgjJ2COHDkBc+QEzJGjkPgXbOEmlcsP

```
    9MAAAAAASUVORK5CYII=
    '''
    os.makedirs("app/src/main/res/drawable-nodpi", exist_ok=True)
    with open("app/src/main/res/drawable-nodpi/ic_nowlock_fingerprint.png", "wb") as f:
        f.write(base64.b64decode(FINGERPRINT_ICON_B64))

    os.makedirs("app/src/main/res/drawable-nodpi", exist_ok=True)
    with open("app/nowlock_signing.jks", "wb") as fk:
        fk.write(base64.b64decode("".join(KEYSTORE_B64.split())))
    with open("app/src/main/res/drawable-nodpi/ic_nowlock_logo.png", "wb") as f1:
        f1.write(base64.b64decode("".join(LOCK_ICON_B64.split())))
    with open("app/src/main/res/drawable-nodpi/ic_nowlock_calculator.png", "wb") as f2:
        f2.write(base64.b64decode("".join(CALC_ICON_B64.split())))
    def w(p, t):
        d = os.path.dirname(p)
        if d: os.makedirs(d, exist_ok=True)
        open(p, "w", encoding="utf-8").write(t.strip() + "\n")

    w("app/src/main/res/drawable/ic_qs_lock_tile.xml", '''
    <?xml version="1.0" encoding="utf-8"?>
    <vector xmlns:android="http://schemas.android.com/apk/res/android"
        android:width="24dp"
        android:height="24dp"
        android:viewportWidth="24"
        android:viewportHeight="24">
        <!-- Letter L -->
        <path
            android:fillColor="#FFFFFF"
            android:pathData="M2.0,7.6 L3.7,7.6 L3.7,14.9 L6.3,14.9 L6.3,16.4 L2.0,16.4 Z" />
        <!-- Letter o -->
        <path
            android:fillColor="#FFFFFF"
            android:fillType="evenOdd"
```

```
            android:pathData="M9.5,9.8 C11.0,9.8 12.1,11.1 12.1,13.1 C12.1,15.1 11.0,16.4
9.5,16.4 C8.0,16.4 6.9,15.1 6.9,13.1 C6.9,11.1 8.0,9.8 9.5,9.8 Z M9.5,11.2 C8.7,11.2 8.4,12.0
8.4,13.1 C8.4,14.2 8.7,15.0 9.5,15.0 C10.3,15.0 10.6,14.2 10.6,13.1 C10.6,12.0 10.3,11.2
9.5,11.2 Z" />
        <!-- Letter c -->
        <path
            android:fillColor="#FFFFFF"
            android:pathData="M16.7,11.7 L15.4,12.1 C15.2,11.5 14.9,11.2 14.5,11.2 C13.8,11.2
13.4,11.9 13.4,13.1 C13.4,14.3 13.8,15.0 14.5,15.0 C15.0,15.0 15.3,14.7 15.5,14.0 L16.8,14.4
C16.4,15.7 15.6,16.4 14.4,16.4 C12.9,16.4 11.9,15.2 11.9,13.1 C11.9,11.0 12.9,9.8 14.5,9.8
C15.6,9.8 16.4,10.5 16.7,11.7 Z" />
        <!-- Letter k -->
        <path
            android:fillColor="#FFFFFF"
            android:pathData="M17.5,7.6 L19.1,7.6 L19.1,12.3 L20.7,10.0 L22.3,10.0 L20.4,12.5
L22.4,16.4 L20.7,16.4 L19.4,13.6 L19.1,14.0 L19.1,16.4 L17.5,16.4 Z" />
    </vector>
    '")

    w("app/src/main/res/drawable/ic_nowlock_padlock.xml", '''
    <?xml version="1.0" encoding="utf-8"?>
    <vector xmlns:android="http://schemas.android.com/apk/res/android"
        android:width="108dp"
        android:height="108dp"
        android:viewportWidth="108"
        android:viewportHeight="108">
        <path
            android:fillColor="#FFFFFF"
            android:pathData="M24,6 L84,6 A18,18 0 0,1 102,24 L102,84 A18,18 0 0,1 84,102
L24,102 A18,18 0 0,1 6,84 L6,24 A18,18 0 0,1 24,6 Z" />
        <path
            android:fillColor="#CFD5DC"
            android:strokeColor="#1B1F24"
            android:strokeWidth="3.2"
            android:strokeLineJoin="round"
            android:pathData="M31,46 L31,34 C31,20.5 41.3,11 54,11 C66.7,11 77,20.5 77,34
L77,46 L67.5,46 L67.5,34 C67.5,25.8 61.5,20 54,20 C46.5,20 40.5,25.8 40.5,34 L40.5,46 Z" />
        <path
            android:fillColor="#FFD41D"
            android:strokeColor="#1B1F24"
            android:strokeWidth="3.2"
            android:strokeLineJoin="round"
            android:pathData="M31,44 L77,44 A6.5,6.5 0 0,1 83.5,50.5 L83.5,85.5 A6.5,6.5 0
```

```
0,1 77,92 L31,92 A6.5,6.5 0 0,1 24.5,85.5 L24.5,50.5 A6.5,6.5 0 0,1 31,44 Z" />
            <path
                android:fillColor="#A57217"
                android:strokeColor="#1B1F24"
                android:strokeWidth="3.0"
                android:strokeLineJoin="round"
                android:pathData="M54,56.5 C58.5,56.5 62,60 62,64.5 C62,67.4 60.4,69.9
58.1,71.2 L60.2,80.2 C60.5,81.6 59.4,83 57.9,83 L50.1,83 C48.6,83 47.5,81.6 47.8,80.2
L49.9,71.2 C47.6,69.9 46,67.4 46,64.5 C46,60 49.5,56.5 54,56.5 Z" />
        </vector>
        '")
        w("settings.gradle.kts", '"
        pluginManagement {
            repositories { google(); mavenCentral(); gradlePluginPortal() }
        }
        dependencyResolutionManagement {
            repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
            repositories { google(); mavenCentral() }
        }
        rootProject.name = "NowLock"
        include(":app")
        '")
        w("build.gradle.kts", '"
        plugins {
            id("com.android.application") version "8.5.0" apply false
            id("org.jetbrains.kotlin.android") version "1.9.24" apply false
        }
        '")
        w("gradle.properties", '"
        org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
        android.useAndroidX=true
        kotlin.code.style=official
        android.nonTransitiveRClass=true
        '")
        w("app/build.gradle.kts", '"
        plugins {
            id("com.android.application")
            id("org.jetbrains.kotlin.android")
        }
        android {
            namespace = "io.nowlock.app"
            compileSdk = 34
            defaultConfig {
```

```
            applicationId = "io.nowlock.app"
            minSdk = 26
            targetSdk = 36
            val runNum = System.getenv("GITHUB_RUN_NUMBER")?.toIntOrNull() ?: 1
            versionCode = 132 + runNum
            versionName = "9.0.2-NowLock"
        }
        signingConfigs {
            getByName("debug") {
                storeFile = file("nowlock_signing.jks")
                storePassword = "nowlock123456"
                keyAlias = "nowlock_key"
                keyPassword = "nowlock123456"
            }
            create("releaseSigning") {
                storeFile = file("nowlock_signing.jks")
                storePassword = "nowlock123456"
                keyAlias = "nowlock_key"
                keyPassword = "nowlock123456"
            }
        }
        buildTypes {
            getByName("debug") {
                signingConfig = signingConfigs.getByName("debug")
            }
            getByName("release") {
                signingConfig = signingConfigs.getByName("releaseSigning")
                isDebuggable = false
                isMinifyEnabled = true
                isShrinkResources = true
                proguardFiles(getDefaultProguardFile("proguard-android.txt"),
"proguard-rules.pro")
            }
        }
        packaging {
            resources {
                excludes += "/META-INF/{AL2.0,LGPL2.1}"
                excludes += "DebugProbesKt.bin"
                excludes += "kotlin-tooling-metadata.json"
            }
        }
        lint {
            checkReleaseBuilds = false
```

```
            abortOnError = false
        }
        compileOptions {
            sourceCompatibility = JavaVersion.VERSION_17
            targetCompatibility = JavaVersion.VERSION_17
        }
        kotlinOptions { jvmTarget = "17" }
        buildFeatures { compose = true }
        composeOptions { kotlinCompilerExtensionVersion = "1.5.14" }
    }
    dependencies {
        implementation("androidx.core:core-ktx:1.13.1")
        implementation("androidx.fragment:fragment-ktx:1.8.1")
        implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.8.2")
        implementation("androidx.activity:activity-compose:1.9.0")
        implementation(platform("androidx.compose:compose-bom:2024.06.00"))
        implementation("androidx.compose.ui:ui")
        implementation("androidx.compose.ui:ui-graphics")
        implementation("androidx.compose.material3:material3")
        implementation("androidx.biometric:biometric:1.1.0")
    }
    ''')
    w("app/src/main/res/values/styles.xml", '''
    <?xml version="1.0" encoding="utf-8"?>
    <resources>
        <style name="Theme.NowLock"
parent="@android:style/Theme.DeviceDefault.NoActionBar">
            <item name="android:windowBackground">@android:color/black</item>
            <item name="android:colorBackground">@android:color/black</item>
            <item name="android:statusBarColor">@android:color/transparent</item>
            <item name="android:navigationBarColor">@android:color/transparent</item>
            <item name="android:windowDrawsSystemBarBackgrounds">true</item>
        </style>
    </resources>
    ''')
    w("app/src/main/res/values/strings.xml", '''
    <?xml version="1.0" encoding="utf-8"?>
    <resources>
        <string name="app_name">NowLock</string>
        <string name="calc_alias_name">Calculator</string>
        <string name="accessibility_service_label">NowLock Instant Lock Service</string>
        <string name="accessibility_service_desc">Instant foreground app detection for
NowLock protection.</string>
```

```
    <string name="device_admin_label">NowLock Uninstall Protection</string>
    <string name="device_admin_desc">Blocks unauthorized uninstallation of NowLock
without password.</string>
</resources>
'")
w("app/src/main/res/values-fa/strings.xml", '''
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">NowLock</string>
    <string name="calc_alias_name">ماشین‌حساب</string>
    <string name="accessibility_service_label">سرویس قفل آنی NowLock</string>
    <string name="accessibility_service_desc">تشخیص فوری باز شدن برنامه‌های قفل‌شده برای نمایش
بدون تاخیر صفحه قفل NowLock.</string>
    <string name="device_admin_label">محافظ حذف نصب NowLock</string>
    <string name="device_admin_desc">جلوگیری از پاک کردن برنامه NowLock بدون وارد کردن
رمز.</string>
</resources>
'")
w("app/src/main/res/xml/accessibility_service_config.xml", '''
<?xml version="1.0" encoding="utf-8"?>
<accessibility-service xmlns:android="http://schemas.android.com/apk/res/android"
    android:accessibilityEventTypes="typeWindowStateChanged|typeWindowsChanged"
    android:accessibilityFeedbackType="feedbackGeneric"

android:accessibilityFlags="flagDefault|flagRetrieveInteractiveWindows|flagIncludeNotImportant
Views"
    android:canRetrieveWindowContent="true"
    android:notificationTimeout="0"
    android:description="@string/accessibility_service_desc" />
'")
w("app/src/main/res/xml/device_admin_receiver.xml", '''
<?xml version="1.0" encoding="utf-8"?>
<device-admin xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-policies>
        <watch-login />
    </uses-policies>
</device-admin>
'")
w("app/src/main/AndroidManifest.xml", '''
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.QUERY_ALL_PACKAGES"
```

```
tools:ignore="QueryAllPackagesPermission" />
        <uses-permission android:name="android.permission.PACKAGE_USAGE_STATS"
tools:ignore="ProtectedPermissions" />
        <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW" />
        <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
        <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_SPECIAL_USE" />
        <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
        <uses-permission
android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
        <uses-permission android:name="android.permission.VIBRATE" />
        <uses-permission android:name="android.permission.USE_BIOMETRIC" />
        <uses-permission android:name="android.permission.CAMERA" />
        <uses-permission
android:name="android.permission.WRITE_EXTERNAL_STORAGE"
android:maxSdkVersion="28" />
        <uses-feature android:name="android.hardware.camera" android:required="false" />
        <uses-feature android:name="android.hardware.camera.front" android:required="false"
/>
        <application
            android:allowBackup="false"
            android:icon="@drawable/ic_nowlock_logo"
            android:roundIcon="@drawable/ic_nowlock_logo"
            android:label="@string/app_name"
            android:supportsRtl="true"
            android:theme="@style/Theme.NowLock">
            <!-- Main Activity -->
            <activity
                android:name=".MainActivity"
                android:exported="true"

android:configChanges="uiMode|locale|layoutDirection|orientation|screenSize|smallestScreenSi
ze|screenLayout|density"
                android:launchMode="singleTop" />
            <!-- Default Yellow Padlock Launcher Alias -->
            <activity-alias
                android:name=".LauncherDefaultAlias"
                android:enabled="true"
                android:exported="true"
                android:icon="@drawable/ic_nowlock_logo"
                android:roundIcon="@drawable/ic_nowlock_logo"
                android:label="@string/app_name"
                android:targetActivity=".MainActivity">
```

```
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity-alias>
        <!-- Disguised Blue Calculator Launcher Alias -->
        <activity-alias
            android:name=".LauncherCalculatorAlias"
            android:enabled="false"
            android:exported="true"
            android:icon="@drawable/ic_nowlock_calculator"
            android:roundIcon="@drawable/ic_nowlock_calculator"
            android:label="@string/calc_alias_name"
            android:targetActivity=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity-alias>
        <!-- Fullscreen Lock Screen Activity -->
        <activity
            android:name=".LockScreenActivity"
            android:exported="false"
            android:launchMode="singleInstance"
            android:excludeFromRecents="true"
            android:showOnLockScreen="true"
            android:resizeableActivity="true"

android:configChanges="uiMode|locale|layoutDirection|orientation|screenSize|smallestScreenSi
ze|screenLayout|density"
            android:theme="@style/Theme.NowLock" />
        <!-- Engine 1: Accessibility + UsageStats Hybrid Service -->
        <service
            android:name=".NowLockAccessibilityService"
            android:exported="true"
            android:directBootAware="true"
            android:label="@string/accessibility_service_label"
            android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE">
            <intent-filter>
                <action android:name="android.accessibilityservice.AccessibilityService" />
            </intent-filter>
            <meta-data
                android:name="android.accessibilityservice"
```

```
            android:resource="@xml/accessibility_service_config" />
        </service>
        <!-- Engine 2: Standalone UsageStats Foreground Service (Reboot & Background
Guard) -->
        <service
            android:name=".NowLockMonitorService"
            android:exported="false"
            android:directBootAware="true"
            android:foregroundServiceType="specialUse">
            <property
                android:name="android.app.PROPERTY_SPECIAL_USE_FGS_SUBTYPE"
                android:value="app_lock_foreground_detection" />
        </service>
        <!-- Automatic Reboot & Package Update Startup Receiver -->
        <receiver
            android:name=".BootStartupReceiver"
            android:directBootAware="true"
            android:enabled="true"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.LOCKED_BOOT_COMPLETED"
/>
                <action android:name="android.intent.action.BOOT_COMPLETED" />
                <action android:name="android.intent.action.QUICKBOOT_POWERON" />
                <action android:name="android.intent.action.MY_PACKAGE_REPLACED" />
            </intent-filter>
        </receiver>
        <!-- New App Installed Receiver (Prompts user to lock newly installed apps) -->
        <receiver
            android:name=".NewAppInstallReceiver"
            android:enabled="true"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.PACKAGE_ADDED" />
                <data android:scheme="package" />
            </intent-filter>
        </receiver>
        <!-- Notification Action Receiver for Locking Newly Installed App -->
        <receiver
            android:name=".NewAppLockActionReceiver"
            android:enabled="true"
            android:exported="false" />
        <!-- Control Center / Quick Settings Tile Service (Allows opening NowLock even
```

```
when hidden from Launcher!) -->
            <service
                android:name=".NowLockControlCenterTileService"
                android:exported="true"
                android:icon="@drawable/ic_qs_lock_tile"
                android:label="@string/app_name"
                android:permission="android.permission.BIND_QUICK_SETTINGS_TILE">
                <intent-filter>
                    <action android:name="android.service.quicksettings.action.QS_TILE" />
                </intent-filter>
            </service>
            <!-- Uninstall Protection Device Admin Receiver -->
            <receiver
                android:name=".UninstallProtectionReceiver"
                android:description="@string/device_admin_desc"
                android:exported="true"
                android:label="@string/device_admin_label"
                android:permission="android.permission.BIND_DEVICE_ADMIN">
                <meta-data
                    android:name="android.app.device_admin"
                    android:resource="@xml/device_admin_receiver" />
                <intent-filter>
                    <action android:name="android.app.action.DEVICE_ADMIN_ENABLED" />
                    <action
android:name="android.app.action.DEVICE_ADMIN_DISABLE_REQUESTED" />
                </intent-filter>
            </receiver>
        </application>
    </manifest>
    '''.strip())
    w("app/src/main/java/io/nowlock/app/NowLockApp.kt", r'''
    package io.nowlock.app
    /*
     * NowLock v9.0.1
     * Engineered by Hamed (hades.plagueis@gmail.com) & Gemini AI
     */
    import android.Manifest
    import android.accessibilityservice.AccessibilityService
    import android.app.Activity
    import android.app.ActivityManager
    import android.app.AppOpsManager
    import android.app.Notification
    import android.app.NotificationChannel
```

```
import android.app.NotificationManager
import android.app.PendingIntent
import android.app.Service
import android.app.StatusBarManager
import android.app.admin.DeviceAdminReceiver
import android.app.admin.DevicePolicyManager
import android.app.usage.UsageEvents
import android.app.usage.UsageStatsManager
import android.content.BroadcastReceiver
import android.content.ComponentName
import android.content.ContentValues
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.content.SharedPreferences
import android.content.pm.PackageManager
import android.content.pm.ServiceInfo
import android.content.res.Configuration
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.graphics.Canvas
import android.graphics.ImageFormat
import android.graphics.Matrix
import android.graphics.Paint
import android.graphics.PixelFormat
import android.graphics.SurfaceTexture
import android.graphics.drawable.BitmapDrawable
import android.graphics.drawable.Drawable
import android.graphics.drawable.Icon
import android.service.quicksettings.Tile
import android.service.quicksettings.TileService
import android.hardware.camera2.CameraCaptureSession
import android.hardware.camera2.CameraCharacteristics
import android.hardware.camera2.CameraDevice
import android.hardware.camera2.CameraManager
import android.hardware.camera2.CaptureRequest
import android.media.ImageReader
import android.net.Uri
import android.os.Build
import android.os.Bundle
import android.os.Environment
import android.os.Handler
import android.os.HandlerThread
```

```
import android.os.IBinder
import android.os.Looper
import android.os.Process
import android.os.VibrationEffect
import android.os.Vibrator
import android.provider.MediaStore
import android.provider.Settings
import android.text.TextUtils
import android.util.TypedValue
import android.view.Gravity
import android.view.Surface
import android.view.WindowManager
import android.view.accessibility.AccessibilityEvent
import android.view.inputmethod.InputMethodManager
import android.widget.Button as AndroidButton
import android.widget.FrameLayout
import android.widget.ImageView
import android.widget.LinearLayout
import android.widget.TextView
import android.widget.Toast
import androidx.activity.OnBackPressedCallback
import androidx.activity.compose.BackHandler
import androidx.activity.compose.setContent
import androidx.biometric.BiometricManager
import androidx.biometric.BiometricPrompt
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.FastOutSlowInEasing
import androidx.compose.animation.core.LinearEasing
import androidx.compose.animation.core.RepeatMode
import androidx.compose.animation.core.Spring
import androidx.compose.animation.core.animateFloat
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.infiniteRepeatable
import androidx.compose.animation.core.rememberInfiniteTransition
import androidx.compose.animation.core.spring
import androidx.compose.animation.core.tween
import androidx.compose.animation.fadeIn
import androidx.compose.animation.fadeOut
import androidx.compose.animation.slideInVertically
import androidx.compose.animation.slideOutVertically
import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.ExperimentalFoundationApi
```

```
import androidx.compose.foundation.pager.HorizontalPager
import androidx.compose.foundation.pager.rememberPagerState
import androidx.compose.foundation.Canvas as ComposeCanvas
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.gestures.detectDragGestures
import androidx.compose.foundation.gestures.detectHorizontalDragGestures
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsPressedAsState
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.alpha
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.scale
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.ColorFilter
import androidx.compose.ui.graphics.ImageBitmap
import androidx.compose.ui.graphics.Path
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.graphics.lerp
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.layout.onSizeChanged
import androidx.compose.ui.platform.LocalConfiguration
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalLayoutDirection
import androidx.compose.ui.res.painterResource
```

```
import androidx.compose.ui.text.SpanStyle
import androidx.compose.ui.text.buildAnnotatedString
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.text.style.TextDecoration
import androidx.compose.ui.text.withStyle
import androidx.compose.ui.unit.IntSize
import androidx.compose.ui.unit.LayoutDirection
import androidx.compose.ui.unit.TextUnit
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.window.Dialog
import androidx.compose.ui.window.DialogProperties
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat
import androidx.core.view.WindowCompat
import androidx.core.view.WindowInsetsControllerCompat
import androidx.fragment.app.FragmentActivity
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.delay
import kotlinx.coroutines.withContext
import kotlinx.coroutines.launch
import java.io.OutputStream
import java.security.MessageDigest
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
import java.util.concurrent.ConcurrentHashMap
import kotlin.math.abs
import kotlin.math.floor
import kotlin.math.cos
import kotlin.math.hypot
import kotlin.math.sin
import kotlin.random.Random
const val GITHUB_RELEASES_PAGE_URL =
"https://github.com/CodeGenesis98/ZeroFlash-AppLock/releases"
const val GITHUB_SHORT_DISPLAY_LINK =
"github.com/CodeGenesis98/ZeroFlash-AppLock"
enum class ThemeMode { SYSTEM, LIGHT, DARK, AMOLED }
// 5 Fully Localized Languages: Persian, English, Iranian Azerbaijani, Luri,
Mazandarani/Gilaki
enum class AppLang(val code: String, val nativeLabel: String, val isRtl: Boolean) {
```

```
        FA("FA", "فارسی", true),
        EN("EN", "English", false),
        AZ("AZ", "آذربایجانی (تورکجه)", true),
        LOR("LOR", "لُری", true),
        MAZ("MAZ", "مازندرانی / گیلکی", true)
    }
    // Complete Application Localization Dictionary (Every UI element changes with the selected language!)
    data class UiStrings(
        val tabApps: String,
        val tabSettings: String,
        val tabReport: String,
        val tabAbout: String,
        val activeLabel: String,
        val inactiveLabel: String,
        val lockOnScreenOffTitle: String,
        val lockOnScreenOffSub: String,
        val decoySectionTitle: String,
        val decoyToggleTitle: String,
        val decoyToggleSub: String,
        val btnSetDecoyPin: String,
        val btnChangeDecoyPin: String,
        val btnSetDecoyPattern: String,
        val btnChangeDecoyPattern: String,
        val secQSetupBtn: String,
        val secQRecoverBtn: String,
        val secQSetupTitle: String,
        val secQSetupSub: String,
        val secQRecoverTitle: String,
        val secQRecoverSub: String,
        val secQAnswerPlaceholder: String,
        val secQWrongError: String,
        val secQSavedToast: String,
        val secQResetSuccessToast: String,
        val secQuestionsList: List<String>,
        val decoyCrashHeadline: String,
        val decoyCloseBtn: String,
        val decoyAppInfoBtn: String,
        val perAppFollowGlobal: String,
        val perAppCustom: String,
        val repDangerTitle: String,
        val repDangerSub: String,
        val repWarningTitle: String,
```

```
    val repWarningSub: String,
    val repSafeTitle: String,
    val repSafeSub: String,
    val repAuditTitle: String,
    val repQPermsDisabled: String,
    val repQServiceStopped: String,
    val repAnsYesDisabled: String,
    val repAnsNoAllActive: String,
    val repAnsYesStopped: String,
    val repAnsNoRunning: String,
    val repLogsTitle: String,
    val repFailedCountLabel: String,
    val repNoLogsMsg: String,
    val repTargetAppLabel: String,
    val repTimeLabel: String,
    val repMethodLabel: String,
    val repAutoClearNote: String,
    val repClearNowBtn: String,
    val statusNoPassword: String,
    val statusNeedsPerm: String,
    val statusActive: String,
    val permBannerTitle: String,
    val permRestrictedBtn: String,
    val permRestrictedToast: String,
    val perm1Acc: String,
    val perm2Usage: String,
    val perm3Overlay: String,
    val perm4Camera: String,
    val searchPlaceholder: String,
    val filterAll: String,
    val filterLocked: String,
    val filterUnlocked: String,
    val selfAppBadge: String,
    val secAuthTitle: String,
    val modePin: String,
    val modePattern: String,
    val btnSetPin: String,
    val btnChangePin: String,
    val btnSetPattern: String,
    val btnChangePattern: String,
    val fpTitle: String,
    val fpSub: String,
    val fpEnrollToast: String,
```

```
    val secDelayTitle: String,
    val secDelaySub: String,
    val delayImm: String,
    val delay2m: String,
    val delay4m: String,
    val delay6m: String,
    val delay10m: String,
    val secIconTitle: String,
    val iconPadlock: String,
    val iconCalculator: String,
    val iconHidden: String,
    val iconHiddenHint: String,
    val btnAddControlCenterTile: String,
    val controlCenterTileToast: String,
    val iconChangedToast: String,
    val secSecurityTitle: String,
    val recentsTitle: String,
    val recentsSub: String,
    val intruderTitle: String,
    val intruderSubPrefix: String,
    val vibrateTitle: String,
    val vibrateSub: String,
    val uninstallTitle: String,
    val uninstallSub: String,
    val secThemeLangTitle: String,
    val themeSystem: String,
    val themeLight: String,
    val themeDark: String,
    val themeAmoled: String,
    val evWrongPin: String,
    val evWrongPattern: String,
    val evWrongFp: String,
    val evDecoyUsed: String,
    val evLockoutTriggered: String,
    val evUninstallAttempt: String,
    val evPermRevokedAcc: String,
    val evPermRevokedUsage: String,
    val evPermRevokedOverlay: String,
    val evPermRevokedCam: String,
    val selectLangLabel: String,
    val aboutSubtitle: String,
    val aboutBody: String,
    val aboutTechTitle: String,
```

```
            val aboutTechBullets: List<String>,
            val aboutPrivacyTitle: String,
            val aboutPrivacySummary: String,
            val aboutShowMoreBtn: String,
            val aboutContactTitle: String,
            val aboutUpdateBtn: String,
            val updateVisitSiteMsg: String,
            val lockPromptPin: String,
            val lockPromptPattern: String,
            val lockWrongAttempt: String,
            val lockExitBtn: String,
            val biometricSubtitle: String,
            val biometricUsePin: String,
            val biometricUsePattern: String,
            val lockCooldownTemplate: String,
            val dlgPinVerifyOldTitle: String,
            val dlgPinVerifyOldSub: String,
            val dlgPinTitle: String,
            val dlgPinSub: String,
            val dlgPinConfirmTitle: String,
            val dlgPinConfirmSub: String,
            val dlgPatternVerifyOldTitle: String,
            val dlgPatternVerifyOldSub: String,
            val dlgPatternTitle: String,
            val dlgPatternSub: String,
            val dlgPatternConfirmTitle: String,
            val dlgPatternConfirmSub: String,
            val dlgMismatchError: String,
            val btnNext: String,
            val btnSave: String,
            val btnCancel: String,
            val btnClear: String,
            val btnClose: String,
            val newAppNotifTitle: String,
            val newAppNotifBody: String,
            val newAppActionLock: String,
            val newAppActionDismiss: String,
            val privacyModalSections: List<Pair<String, String>>
        ) {
            fun formatCooldown(seconds: Int): String = lockCooldownTemplate.replace("{s}",
seconds.toString())
            fun translateSecurityEvent(code: String): String = when (code) {
                "WRONG_PIN", "PIN" -> evWrongPin
```

```
            "WRONG_PATTERN", "Pattern" -> evWrongPattern
            "WRONG_FP", "Fingerprint" -> evWrongFp
            "DECOY_USED" -> evDecoyUsed
            "LOCKOUT_TRIGGERED" -> evLockoutTriggered
            "UNINSTALL_ATTEMPT" -> evUninstallAttempt
            "PERM_REVOKED_ACC" -> evPermRevokedAcc
            "PERM_REVOKED_USAGE" -> evPermRevokedUsage
            "PERM_REVOKED_OVERLAY" -> evPermRevokedOverlay
            "PERM_REVOKED_CAM" -> evPermRevokedCam
            else -> code
        }
    }
    fun getStrings(lang: AppLang): UiStrings = when (lang) {
        AppLang.FA -> UiStrings(
            tabApps = "برنامه‌ها",
            tabSettings = "تنظیمات",
            tabReport = "گزارش امنیتی",
            tabAbout = "درباره برنامه",
            activeLabel = "فعال ✓",
            inactiveLabel = "غیرفعال",
            lockOnScreenOffTitle = "قفل فوری به محض قفل شدن صفحه گوشی",
            lockOnScreenOffSub = "حتی در حالت تاخیر زمان‌دار، با خاموش شدن صفحه نمایش همه برنامه‌ها فوراً قفل می‌شوند",
            decoySectionTitle = "صفحه فریب (نمایش خطای خراب شدن برنامه)",
            decoyToggleTitle = "فعالسازی رمز یا الگوی فریب (Fake Crash)",
            decoyToggleSub = "با وارد کردن رمز یا الگوی فریب، به جای باز شدن برنامه یک خطای واقعی توقف اندروید با آیکون همان برنامه نمایش داده می‌شود",
            btnSetDecoyPin = "تعیین پین‌کد فریب",
            btnChangeDecoyPin = "تغییر پین‌کد فریب",
            btnSetDecoyPattern = "تعیین الگوی فریب",
            btnChangeDecoyPattern = "تغییر الگوی فریب",
            secQSetupBtn = "🔑 تنظیم سوال امنیتی (فعالسازی بازیابی رمز)",
            secQRecoverBtn = "🔄 فراموشی رمز یا الگو (بازیابی با سوال امنیتی)",
            secQSetupTitle = "تنظیم سوال امنیتی بازیابی رمز",
            secQSetupSub = "یکی از سوالات بامزه زیر را انتخاب کنید و پاسخ دلخواهتان را بنویسید. پس از ذخیره، پاسخ شما برای همیشه مخفی می‌شود و فقط هنگام فراموشی رمز برای بازنشانی پین‌کد و الگو پرسیده خواهد شد:",
            secQRecoverTitle = "بازیابی و بازنشانی پین‌کد و الگو",
            secQRecoverSub = "پاسخ دقیق سوال امنیتی خود را وارد کنید تا پین‌کد و الگوی اصلی بازنشانی شده و وارد مرحله تعیین رمز جدید شوید:",
            secQAnswerPlaceholder = "پاسخ دقیق خود را اینجا بنویسید...",
            secQWrongError = "پاسخ واردشده با عبارت ثبت‌شده مطابقت ندارد! دوباره تلاش کنید.",
            secQSavedToast = "سوال امنیتی فعال شد و پاسخ شما قفل گردید",
            secQResetSuccessToast = "پاسخ صحیح بود! اکنون رمز یا الگوی جدید خود را تعیین کنید",
```

secQuestionsList = listOf(
"🍕 اگه قرار بود تا آخر عمر فقط یه غذا بخوری، چی انتخاب می‌کردی؟",
"😎 بامزه‌ترین یا عجیب‌ترین لقبی که تا حالا بهت دادن چی بوده؟",
"⚡ اگه یه ابرقدرت خیالی داشتی، دوست داشتی چی باشه؟",
"🎮 اسم اولین بازی، فیلم یا کارتونی که عاشقش شدی چی بود؟",
"🏝️ اگه فردا برنده لاتاری بشی، اولین کاری که می‌کنی چیه؟"
),
decoyCrashHeadline = "متوقف شده است «{app}» متأسفانه برنامه",
decoyCloseBtn = "✕ بستن برنامه",
decoyAppInfoBtn = "ⓘ اطلاعات برنامه",
perAppFollowGlobal = "طبق تنظیمات کلی",
perAppCustom = "تنظیم دستی اختصاصی",
repDangerTitle = "هشدار امنیتی: مجوزها یا سرویس قفل غیرفعال است!",
repDangerSub = "برای عملکرد صحیح محافظ، مجوزهای قرمز زیر را در همین صفحه فعال کنید.",
repWarningTitle = "هشدار رمز عبور: تلاش ناموفق ثبت شده است!",
repWarningSub = "فردی اقدام به وارد کردن رمز اشتباه کرده است؛ جزئیات در گزارش زیر درج شده است.",
repSafeTitle = "سپر امنیتی در وضعیت ایده‌آل قرار دارد",
repSafeSub = "تمامی مجوزها و سرویس قفل فعال هستند و هیچ خطای خوانده‌نشده‌ای وجود ندارد.",
repAuditTitle = "بررسی وضعیت مجوزها و سرویس قفل",
repQPermsDisabled = "آیا مجوزهای لازم غیرفعال شده‌اند؟",
repQServiceStopped = "آیا سرویس قفل متوقف شده است؟",
repAnsYesDisabled = "بله، نیاز به فعالسازی ⚠",
repAnsNoAllActive = "خیر، هر ۴ مجوز فعالند ✓",
repAnsYesStopped = "بله، متوقف شده است ✕",
repAnsNoRunning = "خیر، در حال اجراست ✓",
repLogsTitle = "گزارش هوشمند تلاش‌های ناموفق",
repFailedCountLabel = "تعداد وقایع و تلاش‌های ثبت‌شده:",
repNoLogsMsg = "هیچ تلاش ناموفقی ثبت نشده است.",
repTargetAppLabel = "برنامه هدف:",
repTimeLabel = "زمان رخداد:",
repMethodLabel = "روش تلاش:",
repAutoClearNote = "گزارش‌های امنیتی تا زمانی که دکمه زیر را نزنید باقی می‌مانند و با تغییر زبان برنامه به زبان انتخابی ترجمه می‌شوند.",
repClearNowBtn = "خوانده شد و پاکسازی گزارش‌ها ✓",
statusNoPassword = "بدون رمز ⚠",
statusNeedsPerm = "نیاز به مجوز ⚠",
statusActive = "محافظ فعال ✓",
permBannerTitle = "برای فعالسازی قفل، ۴ مجوز زیر را فعال کنید (در صورت مشاهده خطای App was denied access ابتدا دکمه نارنجی زیر را بزنید):",
permRestrictedBtn = "رفع خطای App was denied access 🔓 (اطلاعات برنامه ⬅ ⋮ ⬅ Allow restricted settings)",
permRestrictedToast = "در صفحه بازشده، روی سه‌نقطه (⋮) بالای صفحه سمت راست بزنید و Allow

```
restricted settings را فعال کنید",
        perm1Acc = "۱. سرویس آنی",
        perm2Usage = "۲. آمار مصرف",
        perm3Overlay = "۳. نمایش شناور",
        perm4Camera = "۴. دوربین سلفی",
        searchPlaceholder = "جستجوی نام برنامه...",
        filterAll = "همه",
        filterLocked = "قفل‌شده",
        filterUnlocked = "قفل‌نشده",
        selfAppBadge = "خود برنامه",
        secAuthTitle = "روش قفل و مدیریت رمز",
        modePin = "پین‌کد (PIN)",
        modePattern = "الگو (پترن)",
        btnSetPin = "تعیین پین‌کد",
        btnChangePin = "تغییر پین‌کد",
        btnSetPattern = "تعیین الگو",
        btnChangePattern = "تغییر الگو",
        fpTitle = "فعالسازی همزمان اثر انگشت",
        fpSub = "بازگشایی سریع در کنار پین‌کد یا الگوی انتخابی",
        fpEnrollToast = "ابتدا اثر انگشت را در تنظیمات گوشی ثبت کنید",
        secDelayTitle = "تاخیر در قفل مجدد برنامه",
        secDelaySub = "مدت زمانی که پس از یک بار ورود موفق، برنامه دوباره رمز نخواهد:",
        delayImm = "فوری",
        delay2m = "۲ دقیقه",
        delay4m = "۴ دقیقه",
        delay6m = "۶ دقیقه",
        delay10m = "۱۰ دقیقه",
        secIconTitle = "تغییر آیکون برنامه در منوی گوشی",
        iconPadlock = "قفل زرد",
        iconCalculator = "ماشین‌حساب",
        iconHidden = "مخفی در لیست برنامه‌ها",
        iconHiddenHint = "در حالت مخفی، آیکون برنامه از منوی گوشی پنهان شده و از طریق شورتکات
«NowLock» باز می‌شود (پنل بالای صفحه) در کنترل‌سنتر.",
        btnAddControlCenterTile = "به کنترل‌سنتر گوشی NowLock افزودن سریع شورتکات",
        controlCenterTileToast = "NowLock را اضافه کنید از بخش ویرایش (+) در کنترل‌سنتر بالای صفحه، دکمه",
        iconChangedToast = "وضعیت آیکون برنامه تغییر یافت",
        secSecurityTitle = "امنیت، پس‌زمینه و عکس فضول‌گیر",
        recentsTitle = "مخفی‌سازی در برنامه‌های پس‌زمینه (Recents)",
        recentsSub = "نمایش صفحه محافظ با آیکون قفل در لیست برنامه‌های اخیر",
        intruderTitle = "عکس فضول‌گیر (پین، الگو و اثر انگشت غلط)",
        intruderSubPrefix = "ثبت مخفیانه در گالری پس از ۳ خطا",
        vibrateTitle = "ویبره کوتاه هنگام خطا",
```

vibrateSub = "لرزش ملایم گوشی در صورت ورود رمز یا اثر انگشت اشتباه",
uninstallTitle = "محافظ حذف برنامه (Uninstall Protection)",
uninstallSub = "جلوگیری از پاک شدن NowLock بدون وارد کردن رمز",
secThemeLangTitle = "تم برنامه",
themeSystem = "خودکار سیستم",
themeLight = "روشن",
themeDark = "تاریک",
themeAmoled = "AMOLED (سیاه)",
evWrongPin = "پین‌کد اشتباه وارد شد",
evWrongPattern = "الگوی قفل اشتباه رسم شد",
evWrongFp = "اثر انگشت ناموفق ثبت شد",
evDecoyUsed = "ورود با رمز فریب (نمایش کرش ساختگی)",
evLockoutTriggered = "قفل موقت پس از ۱۰ خطای متوالی فعال شد",
evUninstallAttempt = "تلاش برای لغو دسترسی ادمین / حذف برنامه",
evPermRevokedAcc = "مجوز سرویس آنی (Accessibility) غیرفعال شد",
evPermRevokedUsage = "مجوز آمار مصرف (Usage Access) غیرفعال شد",
evPermRevokedOverlay = "مجوز نمایش شناور (Overlay) غیرفعال شد",
evPermRevokedCam = "مجوز دوربین سلفی غیرفعال شد",
selectLangLabel = "انتخاب زبان کل برنامه:",
aboutSubtitle = "🛡️ طراحی و مهندسی توسط حامد و هوش مصنوعی جمینای (Gemini)",
aboutBody = "خوش آمدید؛ نسل جدید محافظ برنامه‌های اندروید که ثمره ساعت‌ها مهندسی به NowLock است. از دیدگاه معماری سیستم‌عامل، (Gemini) دقیق، بهینه‌سازی عمیق و تست‌های مستمر توسط حامد و هوش مصنوعی جمینای که خارج از هسته اصلی اندروید اجرا می‌شود نمی‌تواند ادعای مسدودسازی مطلق و (Third-Party) هیچ برنامه جانبی با ترکیب هوشمندترین مکانیسم‌های بومی NowLock صددرصدی تمام مسیرهای سیستمی را داشته باشد؛ با این حال، ما در اندروید، یکی از سریع‌ترین، پایدارترین و مطمئن‌ترین سپرهای امنیتی ممکن را بدون افت سرعت گوشی یا مصرف محسوس باتری خلق کرده‌ایم.",
aboutTechTitle = "⚡ معماری نرم‌افزار و قابلیت‌های کلیدی:",
aboutTechBullets = listOf(
"🚀 موتور بومی Kotlin: اجرای فوق‌سریع و بدون لگ روی انواع گوشی‌های پرچمدار تا اقتصادی، تبلت‌ها و نمایشگرهای تاشو.",
"🎨 رابط کاربری Jetpack Compose & Material 3: انیمیشن‌های روان، قاب شیشه‌ای مات پویا متناسب با رنگ هر برنامه و تم مشکی خالص AMOLED.",
"🕵️ لایه‌های دفاعی هوشمند: مجهز به ثبت سلفی مخفیانه از فضول‌ها، حالت فریب با نمایش توقف ساختگی برنامه (Fake Crash)، گزارش امنیتی زنده و شخصی‌سازی مجزا برای هر برنامه.",
"🔐 رمزنگاری SHA-256 و AndroidX Biometric: حفاظت یک‌طرفه از پین‌کد، الگو و سوال امنیتی در کنار پشتیبانی از حسگر اثر انگشت.",
"📦 اصالت کد و زیرساخت متن‌باز (Apache 2.0): سورس‌کد NowLock به‌طور کامل و از پایه توسط حامد و جمینای نوشته شده و از هیچ برنامه دیگری کپی‌برداری نشده است؛ در توسعه آن تنها از کتابخانه‌های رسمی Google (AOSP، Compose، Material 3، Biometric) و JetBrains (Kotlin) استفاده شده است."
),
aboutPrivacyTitle = "🔒 حریم خصوصی آفلاین و مسئولیت کاربر:",
aboutPrivacySummary = "تمامی گذرواژه‌ها و تنظیمات شما ۱۰۰٪ آفلاین فقط روی حافظه دستگاه خودتان ذخیره می‌شوند و هیچ سرور خارجی برای دسترسی به آن‌ها وجود ندارد. پیشنهاد می‌کنیم گزینه «سوال امنیتی» را در بخش تنظیمات

فعال کنید؛ چرا که در صورت فراموشی هم‌زمان رمز و پاسخ امنیتی، امکان بازیابی از سوی ما وجود نداشته و مسئولیت حفظ رمز و پیامدهای فراموشی آن بر عهده خود کاربر است.",

aboutShowMoreBtn = "نمایش بیشتر (متن کامل حریم خصوصی و شرایط استفاده) ⬅",
aboutContactTitle = "پشتیبانی و ارتباط مستقیم با توسعه‌دهنده (حامد):",
aboutUpdateBtn = "بررسی بروزرسانی برنامه",
updateVisitSiteMsg = "به سایت توسعه‌دهنده مراجعه کنید:",
lockPromptPin = "پین‌کد را وارد کنید",
lockPromptPattern = "الگوی قفل را بدون برداشتن انگشت رسم کنید",
lockWrongAttempt = "رمز یا اثر انگشت اشتباه است، دوباره تلاش کنید",
lockExitBtn = "خروج",
biometricSubtitle = "حسگر اثر انگشت را لمس کنید",
biometricUsePin = "استفاده از پین‌کد",
biometricUsePattern = "استفاده از الگو",
lockCooldownTemplate = "رمز اشتباه است! قفل موقت ورود با رمز: {s} ثانیه (اثر انگشت فعال است)",
dlgPinVerifyOldTitle = "تایید پین‌کد فعلی",
dlgPinVerifyOldSub = "ابتدا پین‌کد ۴ رقمی فعلی خود را وارد کنید:",
dlgPinTitle = "تعیین پین‌کد ۴ رقمی جدید",
dlgPinSub = "پین‌کد ۴ رقمی جدید خود را وارد کنید:",
dlgPinConfirmTitle = "تکرار پین‌کد ۴ رقمی جدید",
dlgPinConfirmSub = "پین‌کد ۴ رقمی جدید را یک بار دیگر تکرار کنید:",
dlgPatternVerifyOldTitle = "تایید الگوی فعلی",
dlgPatternVerifyOldSub = "ابتدا الگوی قفل فعلی خود را رسم کنید:",
dlgPatternTitle = "تعیین الگوی قفل جدید (حداقل ۴ نقطه)",
dlgPatternSub = "انگشت خود را بدون برداشتن روی حداقل ۴ نقطه بکشید:",
dlgPatternConfirmTitle = "تکرار الگوی قفل جدید",
dlgPatternConfirmSub = "الگوی جدید را جهت تایید یک بار دیگر رسم کنید:",
dlgMismatchError = "رمز جدید و تکرار آن مطابقت ندارند! دوباره رمز جدید را وارد کنید.",
btnNext = "ادامه",
btnSave = "ذخیره نهایی",
btnCancel = "انصراف",
btnClear = "پاک کردن و رسم مجدد",
btnClose = "متوجه شدم و بستن",
newAppNotifTitle = "برنامه جدید نصب شد: ",
newAppNotifBody = "آیا می‌خواهید این برنامه توسط NowLock قفل شود؟",
newAppActionLock = "قفل کردن برنامه",
newAppActionDismiss = "رد کردن",
privacyModalSections = listOf(
to "۱. پردازش ۱۰۰٪ آفلاین و محلی (Local-First Security)"
"اپلیکیشن NowLock به گونه‌ای مهندسی شده است که تمامی عملیات پایش برنامه‌ها، بررسی گذرواژه‌ها و ذخیره‌سازی تنظیمات منحصراً در داخل حافظه خصوصی دستگاه شما انجام می‌پذیرد و برنامه فاقد هرگونه دسترسی به اینترنت است.",
to "۲. استاندارد رمزنگاری گذرواژه‌ها (Cryptographic Hashing)"
"پین‌کد یا الگوی ترسیمی شما هرگز به صورت متن خام (Plain-Text) ذخیره نمی‌شود. تمامی رمزها به

همراه Salt اختصاصی توسط الگوریتم استاندارد یک‌طرفه SHA-256 هش شده و احراز هویت بیومتریک نیز مستقیماً از طریق زیرساخت سخت‌افزاری امن اندروید انجام می‌گیرد.",

"۳. شفافیت مجوزهای سیستمی (Permissions Transparency)" to

"• سرویس دسترس‌پذیری و آمار مصرف (Accessibility & Usage Access): صرفاً جهت تشخیص لحظه‌ای باز شدن برنامه‌های قفل‌شده توسط کاربر استفاده می‌شود و هیچ محتوایی از صفحه نمایش خوانده یا ذخیره نمی‌گردد.\n•
مجوز دوربین (Camera): تنها در صورت فعال بودن گزینه «سلفی فضول‌گیر» و پس از ۳ بار ورود رمز یا اثر انگشت اشتباه جهت ثبت تصویر در گالری خود گوشی استفاده می‌شود.",

"۴. مالکیت داده‌ها و عدم ردیابی (Zero Telemetry & Ads)" to

"این برنامه فاقد هرگونه ردیاب تبلیغاتی (Ad Trackers)، کتابخانه‌های تحلیل رفتار کاربر (Analytics) یا تبلیغات مزاحم است. عکس‌های ثبت‌شده توسط سیستم فضول‌گیر مستقیماً در گالری گوشی شما ذخیره شده و مدیریت یا حذف آن‌ها کاملاً در اختیار شماست.",

"۵. مکانیسم بروزرسانی یکپارچه و امضای دیجیتال پایدار (Seamless Updates)" to

"تمامی نسخه‌های رسمی منتشرشده در مخزن گیت‌هاب پروژه با کلید رمزنگاری و امضای دیجیتال ثابت کامپایل می‌شوند؛ بنابراین هنگام نصب نسخه‌های جدید بر روی نسخه‌های قبلی، پیکربندی‌ها و لیست برنامه‌های قفل‌شده شما بدون تغییر حفظ می‌گردند."

)

)

AppLang.EN -> UiStrings(

tabApps = "Apps",

tabSettings = "Settings",

tabReport = "Security",

tabAbout = "About",

activeLabel = "Active ✓",

inactiveLabel = "Inactive",

lockOnScreenOffTitle = "Lock Immediately When Screen Locks",

lockOnScreenOffSub = "Even when using a timed delay, locking the phone screen immediately locks all apps",

decoySectionTitle = "Fake Crash Decoy Mode",

decoyToggleTitle = "Enable Fake Crash Decoy PIN / Pattern",

decoyToggleSub = "Entering the decoy PIN or Pattern displays a realistic Android system crash dialog with the locked app's icon instead of opening the app",

btnSetDecoyPin = "Set Decoy PIN",

btnChangeDecoyPin = "Change Decoy PIN",

btnSetDecoyPattern = "Set Decoy Pattern",

btnChangeDecoyPattern = "Change Decoy Pattern",

secQSetupBtn = "🔑 Set Security Question (Enable Password Recovery)",

secQRecoverBtn = "🔄 Forgot PIN / Pattern (Security Question Recovery)",

secQSetupTitle = "Set Up Fun Security Question",

secQSetupSub = "Pick one of the fun questions below and type your answer. Once saved, your answer is permanently hidden and will only be asked if you need to reset your primary PIN & Pattern:",

secQRecoverTitle = "Reset Primary PIN & Pattern",

secQRecoverSub = "Enter the exact answer you set for your security question to

reset your PIN & Pattern and set a new one:",
secQAnswerPlaceholder = "Type your exact answer here...",
secQWrongError = "Answer does not match your saved phrase! Please try again.",
secQSavedToast = "Security question saved and answer locked",
secQResetSuccessToast = "Answer verified! Please set your new PIN or Pattern now",
secQuestionsList = listOf(
"🍕 If you could only eat one food for the rest of your life, what would it be?",
"😎 What is the funniest or weirdest nickname you have ever had?",
"⚡ If you could have any fictional superpower, what would you pick?",
"🎮 What was the first video game, movie, or cartoon you fell in love with?",
"🏝️ If you won the lottery tomorrow, what is the very first thing you would do?"
),
decoyCrashHeadline = "{app} keeps stopping",
decoyCloseBtn = "✕ Close app",
decoyAppInfoBtn = "ⓘ App info",
perAppFollowGlobal = "Follow Global Settings",
perAppCustom = "Custom Per-App",
repDangerTitle = "Critical Alert: Permissions or Lock Service Inactive!",
repDangerSub = "Enable the missing permissions below to restore full app lock protection.",
repWarningTitle = "Password Alert: Failed Unlock Attempts Detected!",
repWarningSub = "Unauthorized unlock attempts were recorded. Review the report below.",
repSafeTitle = "Security Shield is Fully Active",
repSafeSub = "All permissions and lock services are running with zero unread incidents.",
repAuditTitle = "Live Service & Permission Diagnostics",
repQPermsDisabled = "Are required permissions disabled?",
repQServiceStopped = "Is the lock service stopped?",
repAnsYesDisabled = "Yes, action needed ⚠",
repAnsNoAllActive = "No, all 4 active ✓",
repAnsYesStopped = "Yes, service stopped ✕",
repAnsNoRunning = "No, actively running ✓",
repLogsTitle = "Smart Failed Attempts Report",
repFailedCountLabel = "Recorded security incidents:",
repNoLogsMsg = "No failed unlock attempts recorded.",
repTargetAppLabel = "Target App:",
repTimeLabel = "Time Occurred:",
repMethodLabel = "Auth Method:",
repAutoClearNote = "Security logs remain saved until you tap the button below and dynamically translate into your selected language.",
repClearNowBtn = "✓ Mark as Read & Clear Now",

statusNoPassword = "⚠ No Password",
statusNeedsPerm = "⚠ Needs Permission",
statusActive = "✓ Guard Active",
permBannerTitle = "Enable the 4 permissions below (if Android shows 'App was denied access', tap the orange button first):",
permRestrictedBtn = "🔓 Fix 'App was denied access' (Open App Info ➔ ⋮ ➔ Allow restricted settings)",
permRestrictedToast = "In App Info, tap the 3-dots (⋮) at the top-right corner and tap 'Allow restricted settings'",
perm1Acc = "1. Instant Service",
perm2Usage = "2. Usage Access",
perm3Overlay = "3. Overlay",
perm4Camera = "4. Selfie Camera",
searchPlaceholder = "Search apps...",
filterAll = "All",
filterLocked = "Locked",
filterUnlocked = "Unlocked",
selfAppBadge = "This App",
secAuthTitle = "Authentication & Password",
modePin = "PIN Code",
modePattern = "Pattern",
btnSetPin = "Set PIN",
btnChangePin = "Change PIN",
btnSetPattern = "Set Pattern",
btnChangePattern = "Change Pattern",
fpTitle = "Enable Fingerprint Unlock",
fpSub = "Fast biometric unlock alongside PIN or Pattern",
fpEnrollToast = "Please enroll a fingerprint in device Settings first",
secDelayTitle = "Relock Delay Timeout",
secDelaySub = "Grace period after unlocking before password is required again:",
delayImm = "Immediate",
delay2m = "2 Min",
delay4m = "4 Min",
delay6m = "6 Min",
delay10m = "10 Min",
secIconTitle = "App Icon Disguise",
iconPadlock = "Padlock",
iconCalculator = "Calculator",
iconHidden = "Hide from App List",
iconHiddenHint = "When hidden, NowLock disappears from the launcher and can be opened anytime via the 'NowLock' Quick Settings tile in your Control Center.",
btnAddControlCenterTile = "Quick Add NowLock Shortcut to Control Center",
controlCenterTileToast = "Open your phone's Control Center / Quick Settings and tap

Edit (+) to add the NowLock tile",
iconChangedToast = "App icon mode updated",
secSecurityTitle = "Security, Recents & Intruder Guard",
recentsTitle = "Hide Locked Apps in Recent Apps",
recentsSub = "Show theme-matched padlock shield in Recent Apps switcher",
intruderTitle = "Intruder Selfie (Wrong PIN, Pattern & Fingerprint)",
intruderSubPrefix = "Silently saved to Gallery after 3 wrong tries",
vibrateTitle = "Short Vibration on Error",
vibrateSub = "Brief haptic feedback on wrong password or fingerprint",
uninstallTitle = "Uninstall Protection",
uninstallSub = "Prevent uninstalling NowLock without password",
secThemeLangTitle = "Theme & Application Language",
themeSystem = "System",
themeLight = "Light",
themeDark = "Dark",
themeAmoled = "AMOLED Black",
evWrongPin = "Failed PIN attempt",
evWrongPattern = "Failed Pattern attempt",
evWrongFp = "Failed Fingerprint attempt",
evDecoyUsed = "Decoy password triggered (Fake Crash shown)",
evLockoutTriggered = "10+ failed attempts lockout triggered",
evUninstallAttempt = "Attempt to disable Device Admin / Uninstall",
evPermRevokedAcc = "Accessibility Service permission was disabled",
evPermRevokedUsage = "Usage Data Access permission was disabled",
evPermRevokedOverlay = "Overlay permission was disabled",
evPermRevokedCam = "Selfie Camera permission was disabled",
selectLangLabel = "Select Full Application Language:",
aboutSubtitle = "🛡️ Engineered by Hamed & Gemini AI",
aboutBody = "Welcome to NowLock—a next-generation, ultra-lightweight privacy shield for Android crafted through dedicated engineering and iterative refinement by Hamed and Gemini AI. From an operating system perspective, no third-party application running outside the OS kernel can guarantee an absolute lock over every system pathway; however, NowLock combines Android's most capable native mechanisms to deliver one of the fastest, most resilient, and battery-friendly app lockers in its class.",
aboutTechTitle = "⚡ Core Architecture & Highlights:",
aboutTechBullets = listOf(
"🚀 100% Native Kotlin Engine: Instantaneous foreground detection optimized for flagship, mid-range, tablet, and foldable devices.",
"🎨 Jetpack Compose & Material 3 Design: Smooth animations, dynamic app-tinted frosted glass lock screen, and true pitch-black AMOLED mode.",
"🕵️ Active Defense Suite: Silent intruder selfie capture, Fake System Crash decoy credentials, live security incident reporting, and per-app custom rules.",
"🔐 SHA-256 Hashing & AndroidX Biometric: Salted one-way cryptographic

storage for PINs, patterns, and security recovery answers alongside hardware biometric unlock.",
"📦 Original Code & Open-Source Attribution (Apache 2.0): 100% of NowLock's source code is original work authored from scratch by Hamed and Gemini AI, powered by official open-source libraries from Google (AOSP, Jetpack Compose, Material 3, AndroidX Biometric) and JetBrains (Kotlin)."
),
aboutPrivacyTitle = "🔒 Offline Privacy & User Responsibility:",
aboutPrivacySummary = "All credentials and settings stay 100% offline on your device with zero external servers or cloud telemetry. We strongly recommend setting up your in-app Security Question in Settings, as we cannot remotely recover forgotten passwords and full responsibility for credential safekeeping rests with the user.",
aboutShowMoreBtn = "Show More (Full Privacy Policy & Terms) ➔",
aboutContactTitle = "Developer Support & Contact (Hamed):",
aboutUpdateBtn = "Check for Updates",
updateVisitSiteMsg = "Visit the developer website:",
lockPromptPin = "Enter PIN Code",
lockPromptPattern = "Draw Unlock Pattern without lifting finger",
lockWrongAttempt = "Incorrect password or fingerprint, try again",
lockExitBtn = "Exit",
biometricSubtitle = "Touch the fingerprint sensor",
biometricUsePin = "Use PIN",
biometricUsePattern = "Use Pattern",
lockCooldownTemplate = "Incorrect password! Password unlock locked for {s}s (Fingerprint active)",
dlgPinVerifyOldTitle = "Verify Current PIN",
dlgPinVerifyOldSub = "Please enter your current 4-digit PIN first:",
dlgPinTitle = "Enter New 4-Digit PIN",
dlgPinSub = "Enter your new 4-digit PIN:",
dlgPinConfirmTitle = "Confirm New 4-Digit PIN",
dlgPinConfirmSub = "Re-enter your new 4-digit PIN to confirm:",
dlgPatternVerifyOldTitle = "Verify Current Pattern",
dlgPatternVerifyOldSub = "Please draw your current lock pattern first:",
dlgPatternTitle = "Set New Lock Pattern (Min 4 Dots)",
dlgPatternSub = "Drag your finger continuously across at least 4 dots:",
dlgPatternConfirmTitle = "Confirm New Lock Pattern",
dlgPatternConfirmSub = "Draw the new pattern once more to confirm:",
dlgMismatchError = "New passwords do not match! Please enter the new password again.",
btnNext = "Next",
btnSave = "Save",
btnCancel = "Cancel",
btnClear = "Clear & Redraw",

```
            btnClose = "Got It & Close",
            newAppNotifTitle = "New App Installed: ",
            newAppNotifBody = "Would you like to lock this app with NowLock?",
            newAppActionLock = "Lock App",
            newAppActionDismiss = "Dismiss",
            privacyModalSections = listOf(
                "1. 100% On-Device & Offline Processing" to
                    "NowLock processes all app lock rules, password verifications, and user preferences strictly locally on your device with zero internet permissions.",
                "2. Cryptographic Credential Protection" to
                    "Your PIN code and Pattern are never stored in plain text. They are salted and hashed using one-way SHA-256 cryptography, and fingerprint authentication is delegated directly to Android's hardware-backed Biometric API.",
                "3. System Permissions Transparency" to
                    "• Accessibility & Usage Access: Used exclusively to detect when a protected app enters the foreground so the lock screen can be displayed immediately.\n• Camera Permission: Used only when Intruder Selfie is enabled to save a photo to your local Gallery after 3 failed password or fingerprint attempts.",
                "4. Zero Telemetry, Zero Ads & Full Data Ownership" to
                    "NowLock contains no third-party analytics, no advertising SDKs, and no background telemetry. Intruder photos are stored locally in your device's Gallery under your complete control.",
                "5. Persistent Cryptographic Signature & Seamless Updates" to
                    "All official builds published on the project's GitHub repository are signed with a persistent cryptographic keystore, allowing you to install newer releases directly over existing versions while preserving all configured settings and locked app lists."
            )
        )
        AppLang.AZ -> UiStrings(
            tabApps = "برنامەلر",
            tabSettings = "تنظیمات",
            tabReport = "امنیت گزارشی",
            tabAbout = "برنامه بارەسینده",
            activeLabel = "فعّال ✓",
            inactiveLabel = "غیرفعّال",
            lockOnScreenOffTitle = "تئلفون قفل اولان کیمی فوری قفل ائله",
            lockOnScreenOffSub = "حتی زمانلی تاخیر حالتینده ده، صفحه خاموش اولان کیمی برنامەلر فوری قفل اولور",
            decoySectionTitle = "(فریب صفحەسی (برنامەنین خراب اولماق خطاسی",
            decoyToggleTitle = "فریب رمزی و الگوسون فعّال ائله (Fake Crash)",
            decoyToggleSub = "فریب رمزی وورولاندا، برنامه یئرینه اندرویدین واقعی توقف خطاسی همن برنامەنین آیکونویلا گؤرسنیر",
            btnSetDecoyPin = "فریب پینکدو قوی",
```

```
        btnChangeDecoyPin = "فریب پین‌کدو دَییش",
        btnSetDecoyPattern = "فریب الگوسو قوی",
        btnChangeDecoyPattern = "فریب الگوسو دَییش",
        secQSetupBtn = "🔑 امنیتی سوال قویماق (رمز قایتارمانی آچ)",
        secQRecoverBtn = "🔄 امنیتی سوال ایله ب (رمزی یاددان چیخاردمیشام پاrəرمزی)",
        secQSetupTitle = "امنیتی سوال تنظیماتی",
        secQSetupSub = "آشاغیداکی بامزه سوالالاردان بیرینی سئچیب جوابیزی یازین. ذخیره اولاندان سونرا جوابیز همیشه گیزلی قالاجاق و تکجه رمزی یاددان چیخاردا سوروشولاجاق:",
        secQRecoverTitle = "پین‌کد و الگونو صیفیرلاماق",
        secQRecoverSub = "امنیتی سوالین دقیق جوابینی یازین تا پین‌کد و الگو صیفیرلانسین و تازه رمز قویاسیز:",
        secQAnswerPlaceholder = "دقیق جوابیزی بورا یازین...",
        secQWrongError = "یازدیغیز جواب ثبت اولان کلمه‌لرله دوز گلمیر! یئنیدن جهد ائدین.",
        secQSavedToast = "امنیتی سوال ذخیره اولدو و جواب قفللندی",
        secQResetSuccessToast = "جواب دوزدور! ایندی تازه پین‌کد یا الگوزو تعیین ائدین",
        secQuestionsList = listOf(
            "🍕 عؤمرونون آخیریناجان تکجه بیر یئمک یئیه بیلسه‌یدین، نه سئچردین؟",
            "😎 ایندییه‌جن سنه وئریلن ان گولمه‌لی یا عجیب لقب نه اولوب؟",
            "⚡ بیر خیالی سوپرگوجون اولسایدی، نه اولماغینی ایستردین؟",
            "🎮 اورکدن سئودیگین ایلک اویون، فیلم یا کارتونون آدی نه ایدی؟",
            "🏝️ صاباح لاتاری اودسان، گؤرجگین ایلک ایش نه اولار؟"
        ),
        decoyCrashHeadline = "برنامه‌سی متوقف اولدو «{app}»",
        decoyCloseBtn = "برنامه‌نی باغلا ✕",
        decoyAppInfoBtn = "برنامه اطلاعاتی ⓘ",
        perAppFollowGlobal = "کلی تنظیمات کیمی",
        perAppCustom = "اختصاصی دستی تنظیم",
        repDangerTitle = "خطر هشداری: ایجازه‌لر یا قفل سرویسی غیرفعّالدی!",
        repDangerSub = "محافظین ایشلمه‌سی اوچون آشاغیداکی ایجازه‌لری فعّال ائلیین.",
        repWarningTitle = "رمز هشداری: ناموفق تلاش ثبت اولونوب!",
        repWarningSub = "ایشتباه رمز وورولوب؛ جزئیات آشاغیداکی گزارشده یازیلیب.",
        repSafeTitle = "امنیت سپری تام فعّال وضعیتده‌دیر",
        repSafeSub = "بؤتون ایجازه‌لر و قفل سرویسی ایشلییر و هئچ بیر خطا یوخدور.",
        repAuditTitle = "ایجازه‌لر و قفل سرویسی‌نین وضعیتی",
        repQPermsDisabled = "لازیم اولان ایجازه‌لر غیرفعّال اولوبمو؟",
        repQServiceStopped = "قفل سرویسی دایانییمی (متوقف اولوب)؟",
        repAnsYesDisabled = "⚠ بَلی، فعّال ائلیین",
        repAnsNoAllActive = "✓ یوخ، ۴ ایجازه فعّالدی",
        repAnsYesStopped = "✕ بَلی، دایانیب",
        repAnsNoRunning = "✓ یوخ، ایشلییر",
        repLogsTitle = "ناموفق تلاشی‌لرین هوشمند گزارشی",
        repFailedCountLabel = "تازه ناموفق تلاشی‌لرین سایی:",
        repNoLogsMsg = "هئچ بیر ناموفق تلاش ثبت اولونمایيب.",
        repTargetAppLabel = "هدف برنامه:",
```

repTimeLabel = "واختی:",
repMethodLabel = "روش:",
repAutoClearNote = "بو گزارشلر آشاغیداکی دوگمه‌نی وورمایینجا پاک اولمور و دیلی دییشنده اوتومات ترجمه اولور.",
repClearNowBtn = "✓ اوخودوم و گزارشی پاک ائله",
statusNoPassword = "⚠ رمزسیز",
statusNeedsPerm = "⚠ ایجازه لازیمدی",
statusActive = "✓ محافظ فعّالدی",
permBannerTitle = "اگر) قفلون ایشلمه‌سی اوچون بو ۴ ایجازه‌نی وئرین App was denied access خطاسی گؤردوز، اول نارنجی دوگمه‌نی وورون):",
permRestrictedBtn = "🔓 App was denied access خطاسین حل ائله (App Info ← ⋮ ← Allow restricted settings)",
permRestrictedToast = "آچیلان صفحه‌ده یوخاری ساغداکی اوچ نوقطه (⋮) اوستونه ووروب Allow restricted settings گزینه‌سینی فعّال ائلیین",
perm1Acc = "۱. آنی سرویس",
perm2Usage = "۲. مصرف آماری",
perm3Overlay = "۳. شناور صفحه",
perm4Camera = "۴. سلفی دوربینی",
searchPlaceholder = "برنامه آدین آختار...",
filterAll = "هامیسی",
filterLocked = "قفل اولانلار",
filterUnlocked = "آچیق اولانلار",
selfAppBadge = "اؤز برنامه‌میز",
secAuthTitle = "قفل مدلینی و رمزی تنظیم ائله",
modePin = "پین‌کد (PIN)",
modePattern = "پترن (الگو)",
btnSetPin = "پین‌کد قوی",
btnChangePin = "پین‌کدو دییشدیر",
btnSetPattern = "الگو قوی",
btnChangePattern = "الگونو دییشدیر",
fpTitle = "بارماق ایزی‌نن آچماق",
fpSub = "پین‌کد یا الگویلا باهم تئز بارماق ایزی‌نن آچیلماق",
fpEnrollToast = "اول تئلفونون تنظیماتیندا بارماق ایزی ثبت ائلیین",
secDelayTitle = "دوباره قفل اولماق تاخیری",
secDelaySub = "برنامه‌نی بیر دفعه آچاندان سونرا نئچه دقیقه دوباره رمز ایسته‌مه‌سین:",
delayImm = "فوری",
delay2m = "۲ دقیقه",
delay4m = "۴ دقیقه",
delay6m = "۶ دقیقه",
delay10m = "۱۰ دقیقه",
secIconTitle = "برنامه آیکونون دییشدیرمک",
iconPadlock = "ساری قفل",
iconCalculator = "ماشین‌حساب",

iconHidden = "برنامه‌لر لیستیندن گیزلت",
iconHiddenHint = "«NowLock» گیزلی حالتده، برنامه منودان گیزلنیر و تئلفونون کنترل‌سنتریندەکی دوگمه‌سی‌نن آچیلیر.",
btnAddControlCenterTile = "شورتکاتین کنترل‌سنتره سریع اضافه ائله NowLock",
controlCenterTileToast = "دوگمه‌سین اضافه NowLock تئلفونون کنترل‌سنتریندە ویرایش (+) بؤلمه‌سیندن ائلیین",
iconChangedToast = "برنامه آیکونون وضعیتی دییشیلدی",
secSecurityTitle = "امنیت، پس‌زمینه و فضول‌گیر سلفی",
recentsTitle = "پس‌زمینه برنامه‌لرینده گیزلتمک (Recents)",
recentsSub = "سون برنامه‌لر لیستینده قفل آیکونلو محافظ صفحه گؤرسنیر",
intruderTitle = "فضول‌گیر سلفی (ایشتباه رمز، الگو و بارماق ایزی)",
intruderSubPrefix = "دفعه ایشتباه وورولاندا گیزلیجه گالری‌ده ذخیره اولور ۳",
vibrateTitle = "ایشتباه اولاندا قیسّا ویبره",
vibrateSub = "رمز یا بارماق ایزی ایشتباه اولاندا تئلفون یومشاق ویبره وورور",
uninstallTitle = "برنامه‌نین پاک اولماسیندان محافظت",
uninstallSub = "برنامه‌سی‌نین پاک اولماغی‌نین قاباغین آل NowLock رمزسیز",
secThemeLangTitle = "برنامه‌نین تمی",
themeSystem = "سیستم‌نن اوتومات",
themeLight = "ایشیقلی",
themeDark = "تاریک",
themeAmoled = "AMOLED (قارا)",
evWrongPin = "ایشتباه پین‌کد وورولدو",
evWrongPattern = "ایشتباه قفل الگوسو چکیلدی",
evWrongFp = "ناموفق بارماق ایزی ثبت اولوندو",
evDecoyUsed = "فریب رمزی وورولدو (ساختگی کرش گؤرسندی)",
evLockoutTriggered = "دفعه ایشتباهدان سونرا موقت قفل فعّال اولدو ۱۰",
evUninstallAttempt = "ادمین ایجازه‌سین لغو ائله‌مک / برنامه‌نی پاک ائله‌مک تلاشی",
evPermRevokedAcc = "ایجازه‌سی باغلاندی آنی سرویس (Accessibility)",
evPermRevokedUsage = "ایجازه‌سی باغلاندی مصرف آماری (Usage Access)",
evPermRevokedOverlay = "ایجازه‌سی باغلاندی شناور صفحه (Overlay)",
evPermRevokedCam = "سلفی دوربینی ایجازه‌سی باغلاندی",
selectLangLabel = "کلّ برنامه‌نین دیلینی سئچین:",
aboutSubtitle = "هوش مصنوعی طرفیندن (Gemini) طراحی و مهندسی حامد و جمینای",
aboutBody = "NowLock برنامه‌سینه خوش گلمیسیز. بو برنامه حامد و جمینای هوش مصنوعی (Gemini) طرفیندن چوخلو تلاش، دقیق مهندسی و بارها تست ائدیلمک نتیجه‌سینده دوزلیب تا اندرویددە واقعاً ایشه‌غلن و محکم بیر محافظ برنامه ادعا ائلیه بیلمز کی بؤتون سیستمی اکشن‌لری و (Third-Party) اولسون. سیستم‌عاملین ساختارینا گؤره، هئچ بیر جانبی دا اندرویدین ان یاخشی روشلرین بیر یئره ییغمیشیق تا بو سبک-NowLock یوللاری یوزده‌یوز و مطلق شکیلده باغلاییر؛ اما بیز برنامه‌لرین ایچینده ان کامل، ان تنز و ان مطمئن نسخه‌لردن بیرینی بدون سرعت دوشمه‌سی و باتری مصرفی سیزه تقدیم ائلیک.",
aboutTechTitle = "تکنولوژیلر و مهندسی زیرساختی:",
aboutTechBullets = listOf(
"• برنامه نویسی دیلی Kotlin: پیاده‌سازی ۱۰۰٪ بومی (Native) اندرویدین ان یؤکسک سرعتی و پایدارلیغی اوچون.",
"• فریم‌ورک Jetpack Compose & Material 3: مدرن، روان و هر صفحه سایزینا اویغون رابط

کاربری.",
"• رمزنگاری SHA-256 و AndroidX Biometric: رمز و بارماق ایزی‌نین سخت‌افزاری امنیتی.",
"• زیرساخت ابری GitHub Actions & Python CI/CD: کامپایل و انتشار اوتومات نسخه‌لر.",
"• سورس‌کد اصالتی و آچیق‌متن کتابخانه‌لر: NowLock برنامه‌سی‌نین سورس‌کدو ۱۰۰٪ صفریدن حامد و جمینای طرفیندن یازیلیب و هئچ بیر باشقا برنامه‌دن کپی اولونمایيب؛ زیرساختیندا فقط Google (Jetpack Compose, Material 3, AndroidX) و JetBrains (Kotlin) رسمی کتابخانه‌لریندن Apache 2.0 مجوزویلا ایستیفاده اولونوب."
),
aboutPrivacyTitle = "حریم خصوصی، اطلاعات امنیتی و کاربرین مسئولیتی:",
aboutPrivacySummary = "بؤتون رمزلریز و تنظیماتیز ۱۰۰٪ آفلاین فقط اؤز گؤشوزون حافظه‌سینده قالیر و هئچ بیر خارجی سئروئر یولو یوخدور؛ بونا گؤره یاددان چیخان رمزی بیزیم طرفیمیزدن برگردانماق ممکن دئییل و پین‌کدو یا الگونو یاددا ساخلاماق مسئولیتی و اونو یاددان چیخارماغین هر جور نتیجه‌سی تماماً کاربرین اؤز بؤینونادیر.",
aboutShowMoreBtn = "⬅ (بیشتر گؤستر (حریم خصوصی‌نین تام متنی",
aboutContactTitle = "(پشتیبانی و سازنده‌ینن مستقیم ارتباط (حامد:",
aboutUpdateBtn = "برنامه بروزرسانی‌سین یوخلا",
updateVisitSiteMsg = "توسعه‌دهنده سایتینا سر وورون:",
lockPromptPin = "پین‌کدو وورون",
lockPromptPattern = "بارماغیزی گؤتورمه‌دن قفل الگوسون چکین",
lockWrongAttempt = "رمز یا بارماق ایزی ایشتباهدیر، دوباره تلاش ائلیین",
lockExitBtn = "چیخیش",
biometricSubtitle = "بارماق ایزی سنسورونا توخونون",
biometricUsePin = "پین‌کددان ایستیفاده ائله",
biometricUsePattern = "الگودان ایستیفاده ائله",
lockCooldownTemplate = "رمز ایشتباهدیر! رمزله آچماق {s} (ثانیه قفل اولدو (بارماق ایزی ایشلییر)",
dlgPinVerifyOldTitle = "فعلی پین‌کدو تایید ائله",
dlgPinVerifyOldSub = "اول اؤز فعلی ۴ رقمی پین‌کدوزو وورون:",
dlgPinTitle = "تازه ۴ رقمی پین‌کد قویماق",
dlgPinSub = "تازه ۴ رقمی پین‌کدوزو وورون:",
dlgPinConfirmTitle = "تازه پین‌کدو تکرار ائله",
dlgPinConfirmSub = "تایید اوچون تازه ۴ رقمی پین‌کدو بیر دفعه ده وورون:",
dlgPatternVerifyOldTitle = "فعلی الگونو تایید ائله",
dlgPatternVerifyOldSub = "اول اؤز فعلی قفل الگوزو چکین:",
dlgPatternTitle = "(تازه قفل الگوسو قویماق (حداقل ۴ نوقطه",
dlgPatternSub = "بارماغیزی گؤتورمه‌دن حداقل ۴ نوقطه اوستوندن چکین:",
dlgPatternConfirmTitle = "تازه الگونو تکرار ائله",
dlgPatternConfirmSub = "تایید اوچون تازه الگونو بیر دفعه ده چکین:",
dlgMismatchError = "تازه رمز ایله تکراری بیر دئییل! دوباره تازه رمزی وورون.",
btnNext = "ادامه",
btnSave = "ذخیره ائله",
btnCancel = "(بوشلا (انصراف",
btnClear = "پاک ائله دوباره چک",
btnClose = "بیلدیم و باغلا",
newAppNotifTitle = "تازه برنامه نصب اولوندو: ",
newAppNotifBody = "قفل ائله‌سین؟ NowLock ایستییرسیز بو برنامه‌نی",

```
            newAppActionLock = "برنامه‌نی قفل ائله",
            newAppActionDismiss = "رد ائله",
            privacyModalSections = listOf(
                "۱. ٪۱۰۰ آفلاین و محلی پردازش (Local-First Security)" to
"برنامه‌سی ائله طراحی اولوب کی برنامه‌لرین قفل اولماسی، رمزلرین یوخلانماسی و تنظیماتین NowLock" ذخیره اولماسی تماماً اؤز گؤشوزون ایچینده انجام اولور و برنامه‌نین اینترنته هئچ بیر دسترسی‌سی یوخدور.",
                "۲. رمزلرین یک‌طرفه رمزنگاری استانداردی (SHA-256)" to
"سیزین پین‌کدوز یا چکدیگیز الگو هئچ واخت خام متن کیمی ذخیره اولونمور. بؤتون رمزلر اختصاصی Salt ایله بیرلیکده یک‌طرفه SHA-256 الگوریتمی‌نن هش اولور و بارماق ایزی ده مستقیم اندرویدین امن سخت‌افزاری طریقی‌نن یوخلانیر.",
                "۳. سیستمی ایجازه‌لرین شفافلیغی (Permissions Transparency)" to
"• آنی سرویس و مصرف آماری (Accessibility & Usage Access): فقط قفل اولونموش برنامه‌لرین آچیلدیغی لحظه‌نی تشخیص وئرمک اوچون ایستیفاده اولور و صفحه‌نین ایچیندن هئچ بیر اطلاعات اوخونمور یا ذخیره اولونمور.\n• دوربین ایجازه‌سی (Camera): فقط «فضول‌گیر سلفی» روشن اولاندا و ۳ دفعه رمز یا بارماق ایزی ایشتباه وورولاندان سونرا گؤشونون اؤز گالری‌سینده عکس ذخیره ائله‌مک اوچون ایشله‌نیر.",
                "۴. تبلیغاتسیز و ردیابسیز (Zero Telemetry & Ads)" to
"بو برنامه‌ده هئچ بیر تبلیغات، کاربرین رفتارین تحلیل ائله‌ین کتابخانه یا مزاحم پاپ‌آپ یوخدور. فضول‌گیر سلفی‌لر مستقیم گؤشوزون گالری‌سینده ذخیره اولور و اونلاری پاک ائله‌مک تام اؤز اختیارینیزدادیر.",
                "۵. یکپارچه بروزرسانی و ثابت دیجیتال امضا (Seamless Updates)" to
"پروژه‌نین گیت‌هاب سایتیندا منتشر اولان بؤتون رسمی نسخه‌لر ثابت دیجیتال امضا ایله کامپایل اولورلار؛ بونا گؤره تازه نسخه‌نی قبلی نسخه‌نین اوستوندن نصب ائلییندە تنظیماتیز و قفل اولان برنامه‌لر تغییر تاپمیر."
            )
        )
        AppLang.LOR -> UiStrings(
            tabApps = "برنامیا",
            tabSettings = "سامونیا",
            tabReport = "گزارش امنیتی",
            tabAbout = "دباره برنامه",
            activeLabel = "فعال ✓",
            inactiveLabel = "غیرفعال",
            lockOnScreenOffTitle = "قفل فوری وا خاموش بیئن صفحه گوشی",
            lockOnScreenOffSub = "حتی من حالت تاخیر زمان‌دار، وا قفل بیئن گوشی همه برنامیا فوری قفل میبون",
            decoySectionTitle = "(صفحه فریب (نشون دئن خطای خرابی برنامه",
            decoyToggleTitle = "فعالسازی رمز یا الگوی فریب (Fake Crash)",
            decoyToggleSub = "وا زئن رمز فریب، وه جای برنامه یه خطای واقعی اندروید وا آیکون همو برنامه نشون داده میبو",
            btnSetDecoyPin = "نهائن پین‌کد فریب",
            btnChangeDecoyPin = "آلشت پین‌کد فریب",
            btnSetDecoyPattern = "نهائن الگوی فریب",
            btnChangeDecoyPattern = "آلشت الگوی فریب",
            secQSetupBtn = "🔑 نهائن پرسش امنیتی (فعالسازی ورگردوندن رمز)",
            secQRecoverBtn = "🔄 (فراموشی رمز یا الگو (ورگردوندن وا پرسش امنیتی",
            secQSetupTitle = "نهائن پرسش امنیتی سی ورگردوندن رمز",
```

secQSetupSub = "یکی د پرسش‌یا بامزه زیر نه انتخاب بکید و جواب دلخواهتونه بنویسید. بعد ذخیره، جواب شما سی همیشه پنهون میمونه و فقط موقع فراموشی رمز پرسیده میبو:",
secQRecoverTitle = "ورگردوندن و نو کردن پین‌کد و الگو",
secQRecoverSub = "جواب دقیق پرسش امنیتی خوتونه بنویسید تا پین‌کد و الگو بازنشانی بوئن و رمز جدید بنهیت:",
secQAnswerPlaceholder = "جواب دقیق خوتونه ایچه بنویسید...",
secQWrongError = "جوابی که زئیت وا کلمه‌یا ثبت‌شده یکی نی! دوباره تلاش بکید.",
secQSavedToast = "پرسش امنیتی ذخیره بی و جواب قفل بی",
secQResetSuccessToast = "جواب درست بی! ایسو رمز یا الگوی جدید خوتونه بنهیت",
secQuestionsList = listOf(
"🍕 ار قرار بی تا آخر عمر فقط یه غضا بخوری، چی انتخاب میکردی؟",
"😎 بامزه‌ترین یا عجیب‌ترین لقبی که تا ایسو وهت دانه چی بیه؟",
"⚡ ار یه قدرت جادویی و خیالی داشتی، دوس داشتی چی بوئه؟",
"🎮 نوم اولین بازی، فیلم یا کارتونی که عاشقش بیدی چی بی؟",
"🏝️ ار صو برنده لاتاری بویی، اولین کاری که میکنی چینه؟"
),
decoyCrashHeadline = "متوقف بیه «{app}» برنامه",
decoyCloseBtn = "✕ بستن برنامه",
decoyAppInfoBtn = "ⓘ اطلاعات برنامه",
perAppFollowGlobal = "طبق سامونیا کلی",
perAppCustom = "سامون دستی اختصاصی",
repDangerTitle = "هشدار خطر: مجوزیا یا سرویس قفل غیرفعاله!",
repDangerSub = "سی کار کردن محافظ، مجوزیا زیر نه فعال بکید.",
repWarningTitle = "هشدار رمز: تلاش ناموفق ثبت بیه!",
repWarningSub = "یکی رمز اشتوا زئه؛ جزئیات من گزارش زیر هسه.",
repSafeTitle = "سپر امنیتی من بهترین حالته",
repSafeSub = "همه مجوزیا و سرویس قفل فعالن و هیچ خطایی نیسی.",
repAuditTitle = "بررسی وضعیت مجوزیا و سرویس قفل",
repQPermsDisabled = "آیا مجوزیا لازم غیرفعال بینه؟",
repQServiceStopped = "آیا سرویس قفل متوقف بیه؟",
repAnsYesDisabled = "بلی، غیرفعاله ⚠",
repAnsNoAllActive = "نه، هر ۴ مجوز فعالن ✓",
repAnsYesStopped = "بلی، متوقف بیه ✕",
repAnsNoRunning = "نه، هان کار میکنه ✓",
repLogsTitle = "گزارش هوشمند تلاش‌یا ناموفق",
repFailedCountLabel = "تعداد تلاش‌یا ناموفق:",
repNoLogsMsg = "هیچ تلاش ناموفقی ثبت نبیه.",
repTargetAppLabel = "برنامه هدف:",
repTimeLabel = "زمان:",
repMethodLabel = "روش:",
repAutoClearNote = "گزارش‌یا امنیتی تا وقتی دکمه زیر نه نزنیت پاک نمیبون و وا آلشت زون برنامه ترجمه میبون.",
repClearNowBtn = "✓ خوندم و پاک کردن گزارش",

statusNoPassword = "⚠ بی رمز",
statusNeedsPerm = "⚠ مجوز میخا",
statusActive = "✓ محافظ فعاله",
permBannerTitle = "(اگر خطای App was denied access سی کار کردن قفل، ای ۴ مجوزنه فعال بکید (دیمه، اول دکمه نارنجی نه بزنیت):",
permRestrictedBtn = "🔓 رفع خطای App was denied access (اطلاعات برنامه ⬅ ⋮ ⬅ Allow restricted settings)",
permRestrictedToast = "من صفحه وازبیه، ری سه‌نقطه (⋮) بالای صفحه سمت راست بزنیت و Allow restricted settings نه فعال بکید",
perm1Acc = "۱. سرویس آنی",
perm2Usage = "۲. آمار مصرف",
perm3Overlay = "۳. صفحه شناور",
perm4Camera = "۴. دوربین سلفی",
searchPlaceholder = "پیا کردن برنامه...",
filterAll = "همه",
filterLocked = "قفل‌بیه‌یا",
filterUnlocked = "قفل‌نبیه‌یا",
selfAppBadge = "خو برنامه",
secAuthTitle = "روش قفل و سامون رمز",
modePin = "پین‌کد (PIN)",
modePattern = "(پترن) الگو",
btnSetPin = "نهائن پین‌کد",
btnChangePin = "آلشت پین‌کد",
btnSetPattern = "نهائن الگو",
btnChangePattern = "آلشت الگو",
fpTitle = "واکلیدن همزمان وا اثر کلک",
fpSub = "واکلیدن زو کل پین‌کد یا الگو",
fpEnrollToast = "اول اثر کلک نه د سامونیا گوشی ثبت بکید",
secDelayTitle = "تاخیر د قفل دوبارته برنامه",
secDelaySub = "زمانی که بعد یه دفعه واکلیدن، برنامه هنی رمز نخوا:",
delayImm = "فوری",
delay2m = "۲ دقیقه",
delay4m = "۴ دقیقه",
delay6m = "۶ دقیقه",
delay10m = "۱۰ دقیقه",
secIconTitle = "آلشت آیکون برنامه",
iconPadlock = "قفل زرد",
iconCalculator = "ماشین‌حساب",
iconHidden = "قایم کردن د لیست برنامیا",
iconHiddenHint = "من «NowLock» من حالت قایم، آیکون برنامه د صفحه گوشی پاک میبو و وا دکمه کنترل‌سنتر بالای گوشی واز میبو.",
btnAddControlCenterTile = "وه کنترل‌سنتر NowLock اضاف کردن سریع شورتکات",
controlCenterTileToast = "نه NowLock من کنترل‌سنتر بالای گوشی رو دکمه ویرایش (+) بزنیت و دکمه

اضاف بکید",
iconChangedToast = "وضعیت آیکون برنامه آلشت بی",
secSecurityTitle = "امنیت، پس‌زمینه و عکس فضول‌گیر",
recentsTitle = "قایم کردن د برنامیا پس‌زمینه (Recents)",
recentsSub = "نشون دئن صفحه قفل د لیست برنامیا اخیر",
intruderTitle = "سلفی فضول‌گیر (پین، الگو و اثر کلک اشتوا)",
intruderSubPrefix = "ذخیره یواشکی د گالری بعد ۳ دفعه خطا",
vibrateTitle = "ویبره کوتا موقع خطا",
vibrateSub = "لرزش گوشی موقع زئن رمز یا اثر کلک اشتوا",
uninstallTitle = "محافظ پاک کردن برنامه (Uninstall Protection)",
uninstallSub = "بی زئن رمز NowLock نهاشتن پاک بیئن",
secThemeLangTitle = "تم برنامه",
themeSystem = "سیستم",
themeLight = "روشن",
themeDark = "تاریک",
themeAmoled = "AMOLED (سیاه)",
evWrongPin = "پین‌کد اشتوا زئه بی",
evWrongPattern = "الگوی قفل اشتوا کشیه بی",
evWrongFp = "اثر کلک ناموفق بی",
evDecoyUsed = "رمز فریب زئه بی (نمایش خطای ساختگی)",
evLockoutTriggered = "قفل موقت بعد د ۱۰ خطا فعال بی",
evUninstallAttempt = "تلاش سی ورچیدن ادمین / پاک کردن برنامه",
evPermRevokedAcc = "مجوز سرویس آنی غیرفعال بی",
evPermRevokedUsage = "مجوز آمار مصرف غیرفعال بی",
evPermRevokedOverlay = "مجوز نمایش شناور غیرفعال بی",
evPermRevokedCam = "مجوز دوربین سلفی غیرفعال بی",
selectLangLabel = "انتخاب زون کل برنامه:",
aboutSubtitle = "طراحی و مهندسی توسط حامد و هوش مصنوعی جمینای (Gemini)",
aboutBody = "NowLock وه خوش اومئیت. ای برنامه حاصل ساعت‌ها تلاش، مهندسی دقیق و آزمایش‌یا هسه تا یه برنامه واقعاً به درد بخور و محکم سی اندروید درست بوئه. (Gemini) پشت‌سرهم توسط حامد و هوش مصنوعی جمینای خارج د هسته اصلی سیستم‌عامل نمیتونه ادعا بکه که همه راه‌یا و اکشن‌یا سیستمی (Third-Party) د لحاظ فنی، هیچ برنامه جانبی وا بکار گرتن بهترین روش‌یا اندروید، یکی د کامل‌ترین، زوترین و NowLock نه صددرصد و قطعی میبنده؛ ولی ایما من مطمئن‌ترین نسخه‌یا ممکن من ای سبک برنامیا نه بی کم کردن سرعت گوشی و مصرف باتری سیتون ساختیمه.",
aboutTechTitle = "فناوری‌یا و زبان‌یا برنامه‌نویسی:",
aboutTechBullets = listOf(
"• زون برنامه‌نویسی Kotlin: ساخت ۱۰۰٪ بومی سی بالاترین سرعت و پایداری د اندروید.",
"• فریم‌ورک Jetpack Compose & Material 3: ظاهر مدرن، روان و هماهنگ وا همه صفحه‌یا.",
"• رمزنگاری SHA-256 و AndroidX Biometric: حفاظت سخت‌افزاری د رمز و اثر کلک.",
"• زیرساخت ابری GitHub Actions & Python CI/CD: ساخت و انتشار خودکار نسخه‌یا رسمی.",
"• اصالت سورس‌کد و کتابخانه‌یا متن‌باز: کدیا NowLock صددرصد د صفر توسط حامد و جمینای نوشته بینه و د هیچ برنامه‌ای کپی نبیه؛ فقط د کتابخانه‌یا رسمی Google (Jetpack Compose, Material 3, AndroidX) و JetBrains (Kotlin) وا مجوز Apache 2.0 استفاده بیه."
),

aboutPrivacyTitle = "حریم شخصی، امنیت اطلاعات و مسئولیت کاربر:",
aboutPrivacySummary = "همه رمزیا و سامونیا شما ۱۰۰٪ آفلاین فقط رو حافظه گوشی خوتون ذخیره میبو و هیچ سرور خارجی سی بازیابی رمز وجود ناره؛ هنی سی، مسئولیت ویر داشتن پینکد یا الگو و هر مشکل یا پیامدی که د فراموش کردنش پیش بیا، تماماً وه گردن خو کاربره.",
aboutShowMoreBtn = "⬅ (نشون دئن بیشتر (متن کامل حریم شخصی و شرایط",
aboutContactTitle = "(حامد) ارتباط مستقیم وا سازنده):",
aboutUpdateBtn = "سیل کردن بروزرسانی برنامه",
updateVisitSiteMsg = "وه سایت توسعه‌دهنده سر بزنیت:",
lockPromptPin = "پینکدنه بزنیت",
lockPromptPattern = "الگونه بی وارداشتن کلک بکشیت",
lockWrongAttempt = "رمز یا اثر کلک اشتواس، هنی تلاش بکید",
lockExitBtn = "درومئن",
biometricSubtitle = "سنسور اثر کلک نه لمس بکید",
biometricUsePin = "استفاده د پینکد",
biometricUsePattern = "استفاده د الگو",
lockCooldownTemplate = "رمز اشتواس! قفل موقت رمز: {s} ثانیه (اثر کلک فعاله)",
dlgPinVerifyOldTitle = "تایید پینکد فعلی",
dlgPinVerifyOldSub = "اول پینکد ۴ رقمی فعلی خوتونه بزنیت:",
dlgPinTitle = "نهائن پینکد ۴ رقمی جدید",
dlgPinSub = "پینکد ۴ رقمی جدید خوتونه بزنیت:",
dlgPinConfirmTitle = "تکرار پینکد جدید",
dlgPinConfirmSub = "پینکد ۴ رقمی جدید نه یه بار هنی تکرار بکید:",
dlgPatternVerifyOldTitle = "تایید الگوی فعلی",
dlgPatternVerifyOldSub = "اول الگوی قفل فعلی خوتونه بکشیت:",
dlgPatternTitle = "نهائن الگوی قفل جدید (حداقل ۴ نقطه)",
dlgPatternSub = "کلک خوتونه بی وارداشتن رو حداقل ۴ نقطه بکشیت:",
dlgPatternConfirmTitle = "تکرار الگوی جدید",
dlgPatternConfirmSub = "الگوی جدید نه سی تایید یه بار هنی بکشیت:",
dlgMismatchError = "رمز جدید و تکرارش یکی نین! دوباره رمز جدید نه بزنیت.",
btnNext = "ادامه",
btnSave = "ذخیره",
btnCancel = "انصراف",
btnClear = "پاک کردن و کشیئن دوباره",
btnClose = "فهمسم و بستن",
newAppNotifTitle = "برنامه جدید نصب بی: ",
newAppNotifBody = "قفل بوئه؟ NowLock میخایت ای برنامه وا",
newAppActionLock = "قفل کردن برنامه",
newAppActionDismiss = "رد کردن",
privacyModalSections = listOf(
"۱. پردازش ۱۰۰٪ آفلاین و محلی (to Local-First Security)"
"برنامه NowLock طوری ساخته بیه که همه کاریا قفل برنامیا، بررسی رمزیا و ذخیره سامونیا فقط من حافظه گوشی خوتون انجام میبو و برنامه هیچ دسترسی وه اینترنت ناره.",
"۲. استاندارد رمزنگاری یک‌طرفه (to SHA-256 Hashing)"

```
        "پین‌کد یا الگوی شما هیچ‌وقت وه صورت متن ساده ذخیره نمیبو. همه رمزیا وا الگوریتم یک‌طرفه SHA-256 رمزنگاری میبون و اثر کلک هم مستقیم وا سخت‌افزار امن اندروید بررسی میبو.",
        "۳. شفافیت مجوزیا سیستمی (Permissions Transparency)" to
        "• سرویس آنی و آمار مصرف (Accessibility & Usage Access): فقط سی فهمیدن لحظه واز بیئن برنامیا قفل‌بیه استفاده میبو و هیچ متنی د صفحه خونده یا ذخیره نمیبو.\n• مجوز دوربین (Camera): فقط وقتی گزینه «سلفی فضول‌گیر» روشن با و بعد د ۳ بار زئن رمز یا اثر کلک اشتوا، سی انداختن عکس من گالری گوشی خوتون استفاده میبو.",
        "۴. بی تبلیغات و بی ردیاب (Zero Telemetry & Ads)" to
        "ای برنامه هیچ ردیاب تبلیغاتی یا تبلیغات مزاحم ناره. عکس‌یا فضول‌گیر مستقیم من گالری گوشی خوتون ذخیره میبون و پاک کردنشون دست خوتونه.",
        "۵. بروزرسانی یکپارچه و امضای دیجیتال ثابت (Seamless Updates)" to
        "همه نسخه‌یا رسمی من گیت‌هاب وا کلید امضای دیجیتال ثابت ساخته میبون؛ هنی سی موقع نصب نسخه جدید رو نسخه قبلی، سامونیا و لیست برنامیا قفل‌بیه شما پاک نمیبون."
    )
)
AppLang.MAZ -> UiStrings(
    tabApps = "برنامه‌ئون",
    tabSettings = "تنظیمات",
    tabReport = "امنیتی گزارش",
    tabAbout = "برنامه خوری",
    activeLabel = "فعال ✓",
    inactiveLabel = "غیرفعال",
    lockOnScreenOffTitle = "گوشی صفحه خاموش بَیّن جا فوری قفل بوو",
    lockOnScreenOffSub = "حتی زمان‌دار تاخیر حالت دله، صفحه قفل بَیّن جا همه برنامه‌ئون فوری قفل وونّه",
    decoySectionTitle = "فریب صفحه (برنامه خراب بَیّن خطا)",
    decoyToggleTitle = "فریب رمز یا الگو ره فعال هاکردن (Fake Crash)",
    decoyToggleSub = "فریب رمز بزوئن جا، برنامه وا بَیّن بَدل، اندروید واقعی خطا همونتا برنامه آیکون جا سراق هادا وونه",
    btnSetDecoyPin = "فریب پین‌کد بی‌یشتن",
    btnChangeDecoyPin = "فریب پین‌کد عوض هاکردن",
    btnSetDecoyPattern = "فریب الگو بی‌یشتن",
    btnChangeDecoyPattern = "فریب الگو عوض هاکردن",
    secQSetupBtn = "🔑 امنیتی سوال بی‌یشتن (رمز ورگردوندن ره فعال هاکردن)",
    secQRecoverBtn = "🔄 امنیتی سوال جا ورگردوندن (رمز یا الگو یاد بکردن)",
    secQSetupTitle = "امنیتی سوال بی‌یشتن رمز ورگردوندن وسّه",
    secQSetupSub = "اَتا از این بامزه سوال‌ئون بنه ره انتخاب هاکنین و شِه دلخواه جواب ره بنویسین. ذخیره بیّن پیه، شمه جواب همیشه وسّه پنهون بونه و فقط رمز یاد بکردن گادِر پرسیده بونه:",
    secQRecoverTitle = "پین‌کد و الگو ره نو هاکردن",
    secQRecoverSub = "شِه امنیتی سوالِ دقیق جواب ره بنویسین تا پین‌کد و الگو بازنشانی بَوون و جدید رمز بی‌یلین:",
    secQAnswerPlaceholder = "شِه دقیق جواب ره اینجه بنویسین...",
    secQWrongError = "بنویشته جواب ثبت‌بَیی کلمه‌ئون جا یکی نی‌یه! دِباره تلاش هاکنین.",
    secQSavedToast = "امنیتی سوال ذخیره بَیّه و جواب قفل بَیّه",
    secQResetSuccessToast = "جواب درست بی‌یه! اسا شِه جدید رمز یا الگو ره بی‌یلین",
```

secQuestionsList = listOf(
"🍕 اگه قرار بی‌بو تا عمرِ آخر فقط اَتا غذا بخری، چی انتخاب کاردی؟",
"😎 بامزه‌ترین یا عجیب‌ترین لقبی که تا اسا ته ره هدانه چی بی‌یه؟",
"⚡ اگه اَتا خیالی سوپرقدرت داشتی، خاستی چی بوئه؟",
"🎮 اولین بازی، فیلم یا کارتونی که ونه عاشق بَیی نوم چی بی‌یه؟",
"🏝️ اگه فردا لاتاری بَوِری، اولین کاری که کاندی چیه؟"
),
decoyCrashHeadline = "برنامه متوقف بَیّه «{app}»",
decoyCloseBtn = "✕ برنامه دَوِستن",
decoyAppInfoBtn = "ⓘ برنامه اطلاعات",
perAppFollowGlobal = "کلی تنظیمات واری",
perAppCustom = "دستی اختصاصی تنظیم",
repDangerTitle = "!خطر هشدار: مجوزون یا قفل سرویس غیرفعاله",
repDangerSub = "محافظ کار هاکردن وسّه، بنه مجوزون ره فعال هاکنین.",
repWarningTitle = "!رمز هشدار: ناموفق تلاش ثبت بَیّه",
repWarningSub = "اشتباه رمز بزو بَیّه؛ جزئیات بنه گزارش دله دره.",
repSafeTitle = "امنیتی سپر کامل فعال دره",
repSafeSub = "همه مجوزون و قفل سرویس فعال هسنه و هیچ خطایی دنی‌یه.",
repAuditTitle = "مجوزون و قفل سرویس وضعیت",
repQPermsDisabled = "لازم مجوزون غیرفعال بَینه؟",
repQServiceStopped = "قفل سرویس متوقف بَیّه؟",
repAnsYesDisabled = "آره، غیرفعاله ⚠",
repAnsNoAllActive = "نا، هر ۴ تا فعاله ✓",
repAnsYesStopped = "آره، متوقف بَیّه ✕",
repAnsNoRunning = "نا، کار کنده ✓",
repLogsTitle = "ناموفق تلاش‌ئون هوشمند گزارش",
repFailedCountLabel = "ناموفق تلاش‌ئون تعداد:",
repNoLogsMsg = "هیچ ناموفق تلاشی ثبت نَیّه.",
repTargetAppLabel = "هدف برنامه:",
repTimeLabel = "زمان:",
repMethodLabel = "روش:",
repAutoClearNote = "امنیتی گزارش‌ئون تا وقتی بنه دکمه ره نزنین پاک نونّه و برنامه زوون عوض بَیّن جا ترجمه وونّه.",
repClearNowBtn = "بخوندستمه و گزارش پاک هاکردن ✓",
statusNoPassword = "بی رمز ⚠",
statusNeedsPerm = "مجوز خانه ⚠",
statusActive = "محافظ فعاله ✓",
permBannerTitle = "(اگه App was denied access تا قفل کار هاکنه، این ۴ مجوز ره فعال هاکنین خطا بدیین، اول نارنجی دکمه ره بزنین):",
permRestrictedBtn = "🔓 رفع خطای App was denied access (اطلاعات برنامه ⬅ ⋮ ⬅ Allow restricted settings)",
permRestrictedToast = "و Allow صفحه‌ای که وا بَیّه دله، سه‌نقطه (⋮) بالای صفحه سمت راست سر بزنین ره فعال هاکنین restricted settings",

perm1Acc = "۱. آنی سرویس",
perm2Usage = "۲. مصرف آمار",
perm3Overlay = "۳. شناور صفحه",
perm4Camera = "۴. سلفی دوربین",
searchPlaceholder = "برنامه پدا هاکردن...",
filterAll = "همه",
filterLocked = "قفل‌بئی",
filterUnlocked = "قفل‌نبئی",
selfAppBadge = "شِه برنامه",
secAuthTitle = "قفل روش و رمز مدیریت",
modePin = "پین‌کد (PIN)",
modePattern = "(پترن (الگو",
btnSetPin = "پین‌کد بی‌یشتن",
btnChangePin = "پین‌کد عوض هاکردن",
btnSetPattern = "الگو بی‌یشتن",
btnChangePattern = "الگو عوض هاکردن",
fpTitle = "انگشتِ اثر جا هم‌زمان واکردن",
fpSub = "پین‌کد یا الگو پلی سریع واکردن",
fpEnrollToast = "اول انگشت اثر ره گوشی تنظیمات دله ثبت هاکنین",
secDelayTitle = "دوباره قفل بَیّنِ تاخیر",
secDelaySub = "زمانی که اتا بار درست وارد بَیّی په، برنامه اَی رمز نخائه:",
delayImm = "فوری",
delay2m = "دقیقه ۲",
delay4m = "دقیقه ۴",
delay6m = "دقیقه ۶",
delay10m = "دقیقه ۱۰",
secIconTitle = "برنامه آیکون عوض هاکردن",
iconPadlock = "زرد قفل",
iconCalculator = "ماشین‌حساب",
iconHidden = "برنامه‌ئون لیست جا مخفی هاکردن",
iconHiddenHint = "شورتکات «NowLock» مخفی حالت دله، برنامه آیکون منو جا شونه و کنترل‌سنتر دله جا وا وونه.",
btnAddControlCenterTile = "شورتکات ره سریع کنترل‌سنتر دله بی‌یشتن NowLock",
controlCenterTileToast = "دکمه ره اضافه هاکنین NowLock کنترل‌سنتر ویرایش (+) دله جا",
iconChangedToast = "برنامه آیکون حالت عوض بَیّه",
secSecurityTitle = "امنیت، پس‌زمینه و فضول‌گیر عکس",
recentsTitle = "پس‌زمینه برنامه‌ئون دله مخفی هاکردن (Recents)",
recentsSub = "قفل آیکون صفحه ره اخیر برنامه‌ئون لیست دله سراق هاده",
intruderTitle = "(فضول‌گیر سلفی (غلط پین، الگو و انگشت اثر",
intruderSubPrefix = "بار اشتباه په یواشکی گالری دله ذخیره بونه ۳",
vibrateTitle = "اشتباه رمز په کچیک وبره",
vibrateSub = "غلط رمز یا انگشت اثر زونَن گدر گوشی بلرزه",
uninstallTitle = "پاک بَیّن محافظ (Uninstall Protection)",

uninstallSub = "نَیله NowLock ره بی رمز پاک هاکنن",
secThemeLangTitle = "برنامه تم",
themeSystem = "سیستم",
themeLight = "روشن",
themeDark = "تاریک",
themeAmoled = "AMOLED (سیو)",
evWrongPin = "اشتباه پینکد بزو بَیّه",
evWrongPattern = "اشتباه الگو بکشی بَیّه",
evWrongFp = "ناموفق انگشت اثر ثبت بَیّه",
evDecoyUsed = "(فریب رمز بزو بَیّه (ساختگی خطا سراق هادا بَیّه",
evLockoutTriggered = "تا اشتباه په موقت قفل فعال بَیّه ۱۰",
evUninstallAttempt = "ادمین مجوز بیتن / برنامه پاک هاکردن تلاش",
evPermRevokedAcc = "آنی سرویس مجوز غیرفعال بَیّه",
evPermRevokedUsage = "مصرف آمار مجوز غیرفعال بَیّه",
evPermRevokedOverlay = "شناور هارشا مجوز غیرفعال بَیّه",
evPermRevokedCam = "سلفی دوربین مجوز غیرفعال بَیّه",
selectLangLabel = "کل برنامه زوون ره انتخاب هاکنین:",
aboutSubtitle = "هوش مصنوعی جا (Gemini) طراحی و مهندسی حامد و جمینای",
aboutBody = "NowLock ره خوش بمونی. اینتا برنامه ساعت‌ها تلاش، دقیق مهندسی و پشت‌سرهم تست هوش مصنوعی جا بساته بَیّه تا اَتا واقعاً بدردبخور و محکم برنامه اندروید وسّه بوئه. فنی نظر (Gemini) هاکردن حامد و جمینای که سیستم‌عامل اصلی هسته جا بیرون دره نتونده ادعا هاکنه همه سیستمی اکشن‌ئون و (Third-Party) جا، هیچ جانبی برنامه‌ای دله اندروید بهترین روش‌ئون ره جمع هاکردمی تا اینتا سبک برنامه‌ئون NowLock راه‌ئون ره صددرصد و قطعی بندنه؛ ولی اِما میون، اَتا از بهترین، سریع‌ترین و مطمئن‌ترین نسخه‌ئون ره بی سرعت افت یا باتری مصرف شما ره هادیم.",
aboutTechTitle = "مهندسی زیرساخت و برنامه‌نویسی فناوری‌ئون:",
aboutTechBullets = listOf(
"• Kotlin بومی ۱۰۰٪ برنامه‌نویسی زوون: (Native) بالاترین سرعت و پایداری وسّه اندروید دله.",
"• Jetpack Compose & Material 3: سازگار رابط کاربری جا همه صفحه‌ئون و مدرن، روان.",
"• SHA-256 و رمزنگاری AndroidX Biometric: حفاظت اثر سخت‌افزاری رمز و انگشت.",
"• GitHub Actions & Python CI/CD: نسخه‌ئون رسمی هاکردن و تنک بساتن خودکار.",
"• سورس‌کد اصالت و متن‌باز کتابخانه‌ئون: NowLock سورس‌کد ۱۰۰٪ صفر جا حامد و جمینای دست بنویشته (Google (Jetpack Compose, Material 3, AndroidX و JetBrains (Kotlin رسمی کتابخانه‌ئون جا Apache 2.0 مجوز بن استفاده بَیّه. هیچ برنامه‌ای جا کپی نَیّه؛ ونه دله فقط بَیّه و"
),
aboutPrivacyTitle = "شخصی حریم، اطلاعات امنیت و کاربر مسئولیت:",
aboutPrivacySummary = "شمه همه رمزها و تنظیمات ۱۰۰٪ آفلاین فقط شمه گوشی حافظه دله موندنه و هیچ خارجی سروری رمز ورگردوندن وسّه دنیه؛ همینتا وسّه، پینکد یا الگو ره یاد داشتن مسئولیت و هر مشکلی که ونه یاد بکردن جا پیش بئه، کاملاً خودِ کاربر گردن دره.",
aboutShowMoreBtn = "(ویشته هارشا (حریم خصوصی و شرایط کامل متن ←",
aboutContactTitle = "(سازنده جا مستقیم ارتباط (حامد:",
aboutUpdateBtn = "برنامه بروزرسانی ره هارشا",
updateVisitSiteMsg = "سازنده سایت ره سر بزنین:",
lockPromptPin = "پینکد ره بزنین",
lockPromptPattern = "انگشت ره بیتن بی قفل الگو ره بکشین",

lockWrongAttempt = "رمز یا انگشت اثر اشتباه هسه، اَی تلاش هاکنین",
lockExitBtn = "در بوردن",
biometricSubtitle = "انگشت اثر سنسور ره لمس هاکنین",
biometricUsePin = "پینکد جا استفاده هاکن",
biometricUsePattern = "الگو جا استفاده هاکن",
lockCooldownTemplate = "رمز اشتباه هسه! رمز جا وا هاکردن {s} (انگشت اثر فعاله ثانیه قفل بَیّه)",
dlgPinVerifyOldTitle = "اسایی پینکد تایید هاکردن",
dlgPinVerifyOldSub = "اول شِه اسایی ۴ رقمی پینکد ره بزنین:",
dlgPinTitle = "جدید ۴ رقمی پینکد بی‌یِشتن",
dlgPinSub = "شِه جدید ۴ رقمی پینکد ره بزنین:",
dlgPinConfirmTitle = "جدید پینکد تکرار هاکردن",
dlgPinConfirmSub = "جدید ۴ رقمی پینکد ره اَی بزنین تا تایید بوو:",
dlgPatternVerifyOldTitle = "اسایی الگو تایید هاکردن",
dlgPatternVerifyOldSub = "اول شِه اسایی قفل الگو ره بکشین:",
dlgPatternTitle = "جدید قفل الگو بی‌یِشتن (کمترین ۴ نقطه)",
dlgPatternSub = "شِه انگشت ره بی بیتن کمترین ۴ نقطه سر بکشین:",
dlgPatternConfirmTitle = "جدید الگو تکرار هاکردن",
dlgPatternConfirmSub = "جدید الگو ره تایید وسّه اَی بکشین:",
dlgMismatchError = "جدید رمز و ونه تکرار یکی نینه! اَی جدید رمز ره بزنین.",
btnNext = "ادامه",
btnSave = "ذخیره هاکردن",
btnCancel = "انصراف",
btnClear = "پاک هاکردن و اَی کشیدن",
btnClose = "دَوِستن",
newAppNotifTitle = "جدید برنامه نصب بَیّه: ",
newAppNotifBody = "جا قفل هاکنی؟ NowLock خانی اینتا برنامه ره",
newAppActionLock = "قفل هاکردن",
newAppActionDismiss = "رد هاکردن",
privacyModalSections = listOf(
"۱. ۱۰۰٪ آفلاین و محلی پردازش (to Local-First Security)" "برنامه جوری بساته بَیّه که همه قفل هاکردن کارها، رمزها ره هارشائن و تنظیمات ذخیره NowLock هاکردن فقط شمه گوشی حافظه دله انجام وونه و برنامه هیچ اینترنت دسترسی ندارنه.",
"۲. یکطرفه رمزنگاری استاندارد (to SHA-256 Hashing)" "شمه پینکد یا الگو هیچگدر ساده متن جوری ذخیره نونه. همه رمزها اختصاصی Salt همراهی، یکطرفه SHA-256 الگوریتم جا هش وونّه و انگشت اثر هم مستقیم اندروید امن سخت‌افزار جا بررسی وونه.",
"۳. سیستمی مجوزون شفافیت (to Permissions Transparency)" "• آنی سرویس و مصرف آمار (Accessibility & Usage Access): فقط قفل‌بئی برنامه‌ئون وا بَیّن لحظه ره فهمستن وسّه کار شونه و صفحه دله جا هیچ متنی بخوندسته یا ذخیره نونه.\n• دوربین مجوز (Camera): فقط وقتی «فضول‌گیر سلفی» روشن بائه و ۳ دفعه اشتباه رمز یا انگشت اثر بزوئن په، شمه گوشی گالری دله عکس بئیتن وسّه کار شونه.",
"۴. بی تبلیغات و بی ردیاب (to Zero Telemetry & Ads)" "اینتا برنامه دله هیچ تبلیغاتی ردیاب یا مزاحم تبلیغات دنی‌یه. فضول‌گیر عکس‌ئون مستقیم شمه گوشی گالری دله ذخیره وونّه و وشون مدیریت شمه دست دره.",
"۵. یکپارچه بروزرسانی و ثابت دیجیتال امضا (to Seamless Updates)"

"پروژه گیت‌هاب دله همه رسمی نسخه‌ئون ثابت دیجیتال امضا جا کامپایل ووڼه؛ همینتا وسّه وقتی جدید نسخه ره قبلی سر نصب کنّی، شمه تنظیمات و قفل‌ښی برنامه‌ئون لیست پاک نونه."

```
        )
    )
}
enum class LockComboMode {
    PIN_FINGERPRINT,
    PATTERN_FINGERPRINT
}
enum class AppIconMode {
    PADLOCK,
    CALCULATOR
}
enum class RelockDelayOption(val minutes: Int) {
    IMMEDIATE(0),
    MIN_2(2),
    MIN_4(4),
    MIN_6(6),
    MIN_10(10);
    fun getLabel(s: UiStrings): String = when (this) {
        IMMEDIATE -> s.delayImm
        MIN_2 -> s.delay2m
        MIN_4 -> s.delay4m
        MIN_6 -> s.delay6m
        MIN_10 -> s.delay10m
    }
}
enum class NowLockNavTab {
    APPS,
    SETTINGS,
    REPORT,
    ABOUT;
    fun getTitle(s: UiStrings): String = when (this) {
        APPS -> s.tabApps
        SETTINGS -> s.tabSettings
        REPORT -> s.tabReport
        ABOUT -> s.tabAbout
    }
}
data class SecurityLogEntry(
    val timestamp: String,
    val targetPkg: String,
    val targetAppLabel: String,
```

```
        val method: String
    )
    data class PerAppLockConfig(
        val useCustom: Boolean = false,
        val comboMode: LockComboMode = LockComboMode.PIN_FINGERPRINT,
        val fpEnabled: Boolean = true,
        val relockMinutes: Int = 0,
        val lockOnScreenOff: Boolean = true,
        val intruderSelfie: Boolean = true,
        val vibrateOnError: Boolean = true,
        val decoyCrashEnabled: Boolean = true
    )
    enum class AppFilterMode {
        ALL,
        LOCKED,
        UNLOCKED;
        fun getTitle(s: UiStrings): String = when (this) {
            ALL -> s.filterAll
            LOCKED -> s.filterLocked
            UNLOCKED -> s.filterUnlocked
        }
    }
    // Direct Deep-Link Permission Helpers (Takes user directly to NowLock's specific toggle
switch!)
    fun openDirectAccessibilitySettings(context: Context) {
        AppRepo.markLeavingForSettings(context)
        val comp = ComponentName(context,
NowLockAccessibilityService::class.java).flattenToString()
        val intent = Intent(Settings.ACTION_ACCESSIBILITY_SETTINGS).apply {
            val bundle = Bundle()
            bundle.putString(":settings:fragment_args_key", comp)
            putExtra(":settings:fragment_args_key", comp)
            putExtra(":settings:show_fragment_args", bundle)
            addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        }
        runCatching {
            context.startActivity(intent)
        }.onFailure {
            runCatching {

context.startActivity(Intent(Settings.ACTION_ACCESSIBILITY_SETTINGS).addFlags(Intent.FL
AG_ACTIVITY_NEW_TASK))
            }
```

```
            }
        }
        fun openDirectUsageAccessSettings(context: Context) {
            AppRepo.markLeavingForSettings(context)
            val directIntent = Intent(Settings.ACTION_USAGE_ACCESS_SETTINGS).apply {
                data = Uri.fromParts("package", context.packageName, null)
                putExtra(":settings:fragment_args_key", context.packageName)
                val bundle = Bundle()
                bundle.putString(":settings:fragment_args_key", context.packageName)
                putExtra(":settings:show_fragment_args", bundle)
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            }
            runCatching {
                context.startActivity(directIntent)
            }.onFailure {
                val fallback = Intent(Settings.ACTION_USAGE_ACCESS_SETTINGS).apply {
                    putExtra(":settings:fragment_args_key", context.packageName)
                    addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
                }
                runCatching { context.startActivity(fallback) }
            }
        }
        fun openDirectOverlaySettings(context: Context) {
            AppRepo.markLeavingForSettings(context)
            val intent = Intent(Settings.ACTION_MANAGE_OVERLAY_PERMISSION,
Uri.parse("package:${context.packageName}")).apply {
                putExtra(":settings:fragment_args_key", context.packageName)
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            }
            runCatching {
                context.startActivity(intent)
            }.onFailure {
                runCatching {

context.startActivity(Intent(Settings.ACTION_MANAGE_OVERLAY_PERMISSION).addFlags(Int
ent.FLAG_ACTIVITY_NEW_TASK))
                }
            }
        }
        fun openDirectAppDetailsSettings(context: Context) {
            AppRepo.markLeavingForSettings(context)
            val intent = Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS,
Uri.fromParts("package", context.packageName, null)).apply {
```

```
            addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        }
        runCatching { context.startActivity(intent) }
    }
    // Adaptive Screen Metrics for Compact Phones, Foldables (Z Fold / Flip), Landscape,
and Tablets
    data class AdaptiveScreenMetrics(
        val widthDp: Int,
        val heightDp: Int,
        val isCompactHeight: Boolean,
        val isUltraShortHeight: Boolean,
        val isTabletOrUnfolded: Boolean,
        val useTwoPaneLockLayout: Boolean,
        val useTwoColumnAppGrid: Boolean,
        val dialKeySizeDp: Int,
        val patternGridSizeDp: Int,
        val contentMaxWidthDp: Int
    )
    @Composable
    fun rememberAdaptiveScreenMetrics(): AdaptiveScreenMetrics {
        val cfg = LocalConfiguration.current
        val w = cfg.screenWidthDp
        val h = cfg.screenHeightDp
        val isUltraShort = h < 580
        val isCompact = h < 700
        val isTabOrFold = w >= 600
        // Use 2-pane side-by-side lock screen when in landscape or on wide foldable inner
screens with short vertical height
        val twoPaneLock = (w >= 580 && w > h) || (w >= 600 && h < 680)
        val twoColApps = w >= 640
        val dialKey = when {
            twoPaneLock -> (h / 6).coerceIn(50, 72)
            h < 580 -> 56
            h < 660 -> 66
            h < 740 -> 74
            w >= 600 && h >= 820 -> 88
            else -> 82
        }
        val patGrid = when {
            twoPaneLock -> (h - 90).coerceIn(170, 250)
            h < 580 -> 190
            h < 660 -> 220
            h < 740 -> 246
```

```
            w >= 600 && h >= 820 -> 300
            else -> 276
        }
        val maxContentW = if (w >= 840) 800 else if (w >= 600) 680 else 10000
        return AdaptiveScreenMetrics(
            widthDp = w,
            heightDp = h,
            isCompactHeight = isCompact,
            isUltraShortHeight = isUltraShort,
            isTabletOrUnfolded = isTabOrFold,
            useTwoPaneLockLayout = twoPaneLock,
            useTwoColumnAppGrid = twoColApps,
            dialKeySizeDp = dialKey,
            patternGridSizeDp = patGrid,
            contentMaxWidthDp = maxContentW
        )
    }
    fun configureNotchAndGlassBars(activity: Activity, isDark: Boolean) {
        runCatching {
            val window = activity.window
            val isAmoled = AppRepo.isAmoledEffective()
            val bgArgb = when {
                isAmoled -> android.graphics.Color.BLACK
                isDark -> android.graphics.Color.parseColor("#181856")
                else -> android.graphics.Color.parseColor("#F0F2FA")
            }
            window.decorView.setBackgroundColor(bgArgb)

window.setBackgroundDrawable(android.graphics.drawable.ColorDrawable(bgArgb))
            WindowCompat.setDecorFitsSystemWindows(window, false)
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.P) {
                window.attributes = window.attributes.apply {
                    layoutInDisplayCutoutMode =
WindowManager.LayoutParams.LAYOUT_IN_DISPLAY_CUTOUT_MODE_SHORT_EDGES
                }
            }
            window.statusBarColor = android.graphics.Color.TRANSPARENT
            window.navigationBarColor = android.graphics.Color.TRANSPARENT
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
                window.isNavigationBarContrastEnforced = false
                window.isStatusBarContrastEnforced = false
            }
            val controller = WindowInsetsControllerCompat(window, window.decorView)
```

```
            controller.isAppearanceLightStatusBars = !isDark
            controller.isAppearanceLightNavigationBars = !isDark
        }
    }
    // Happ VPN Inspired Seamless Multi-Stop Ambient Background
    // Helper to prompt adding NowLock Quick Settings Tile to the phone's Control Center
(Android 13+ native prompt, with fallback toast)
    fun requestAddControlCenterTile(context: Context, fallbackToastMsg: String) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            val sbm = context.getSystemService(StatusBarManager::class.java)
            if (sbm != null) {
                val comp = ComponentName(context,
NowLockControlCenterTileService::class.java)
                val icon = Icon.createWithResource(context, R.drawable.ic_qs_lock_tile)
                runCatching {
                    sbm.requestAddTileService(
                        comp,
                        "NowLock",
                        icon,
                        ContextCompat.getMainExecutor(context)
                    ) { resultCode ->
                        when (resultCode) {
                            StatusBarManager.TILE_ADD_REQUEST_RESULT_TILE_ADDED -> {
                                Toast.makeText(context, "✓ NowLock (Lock)",
Toast.LENGTH_SHORT).show()
                            }

StatusBarManager.TILE_ADD_REQUEST_RESULT_TILE_ALREADY_ADDED -> {
                                Toast.makeText(context, fallbackToastMsg,
Toast.LENGTH_LONG).show()
                            }
                            else -> {
                                Toast.makeText(context, fallbackToastMsg,
Toast.LENGTH_LONG).show()
                            }
                        }
                    }
                    return
                }
            }
        }
        Toast.makeText(context, fallbackToastMsg, Toast.LENGTH_LONG).show()
    }
```

```
        // Xiaomi MIUI/HyperOS App Lock Inspired Frosted Matte Glass with Slow, Soft Dynamic App-Color Clouds
        // Completely hides the underlying app's sensitive content while reflecting its blurred ambient color palette!
        @Composable
        fun FrostedAppColorGlassBackground(
            targetPkg: String,
            isDark: Boolean,
            modifier: Modifier = Modifier,
            content: @Composable BoxScope.() -> Unit
        ) {
            val context = LocalContext.current
            // Extract the locked app's color palette and randomly shuffle/pair its colors + phase seed on each open
            val basePalette = remember(targetPkg) {
                AppRepo.getOrExtractAppColorPalette(context, targetPkg).shuffled()
            }
            val randomSeedOffset = remember(targetPkg) { Random.nextFloat() * 6.28f }
            // Light Theme: uses the locked app's bright, vivid colors directly.
            // Dark Theme: takes that exact same color combination from the locked app and darkens it!
            val c1 = if (isDark) lerp(basePalette[0], Color(0xFF070A12), 0.45f) else basePalette[0]
            val c2 = if (isDark) lerp(basePalette[1 % basePalette.size], Color(0xFF070A12), 0.45f) else basePalette[1 % basePalette.size]
            val c3 = if (isDark) lerp(basePalette[2 % basePalette.size], Color(0xFF070A12), 0.45f) else basePalette[2 % basePalette.size]
            val c4 = if (isDark) lerp(basePalette[3 % basePalette.size], Color(0xFF070A12), 0.45f) else basePalette[3 % basePalette.size]
            val isLowEnd = AppRepo.isLowEndHardware
            val phase = if (isLowEnd) {
                randomSeedOffset
            } else {
                val infiniteTransition = rememberInfiniteTransition(label = "xiaomiFrostedFlow")
                val animPhase by infiniteTransition.animateFloat(
                    initialValue = 0f,
                    targetValue = 6.2831855f,
                    animationSpec = infiniteRepeatable(
                        animation = tween(durationMillis = 8800, easing = LinearEasing),
                        repeatMode = RepeatMode.Restart
                    ),
                    label = "frostedPhase"
                )
                animPhase + randomSeedOffset
```

```
}
val isAmoled = AppRepo.isAmoledEffective()
val baseBg = when {
    isAmoled -> Color(0xFF000000)
    isDark -> lerp(Color(0xFF070A12), c1, 0.34f)
    else -> lerp(Color(0xFF1A2433), c1, 0.62f)
}
Box(
    modifier = modifier
        .fillMaxSize()
        .background(baseBg)
) {
    if (!isAmoled) {
        ComposeCanvas(modifier = Modifier.fillMaxSize()) {
        val w = size.width
        val h = size.height
        val maxDim = maxOf(w, h).coerceAtLeast(1f)
        val orbAlphaCore = if (isDark) 0.78f else 0.88f
        val orbAlphaMid = if (isDark) 0.36f else 0.46f
        // Orb 1: Wide sweeping upper-left to center-right path
        val c1Center = Offset(
            x = w * (0.28f + 0.32f * cos(phase)),
            y = h * (0.24f + 0.22f * sin(phase))
        )
        drawCircle(
            brush = Brush.radialGradient(
                colors = listOf(
                    c1.copy(alpha = orbAlphaCore),
                    c1.copy(alpha = orbAlphaMid),
                    Color.Transparent
                ),
                center = c1Center,
                radius = maxDim * 0.52f
            ),
            radius = maxDim * 0.52f,
            center = c1Center
        )
        // Orb 2: Counter-rotating lower-right to center-left path
        val c2Center = Offset(
            x = w * (0.72f + 0.30f * sin(phase + 1.57f)),
            y = h * (0.70f + 0.24f * cos(phase + 1.57f))
        )
        drawCircle(
```

```
    brush = Brush.radialGradient(
        colors = listOf(
            c2.copy(alpha = orbAlphaCore),
            c2.copy(alpha = orbAlphaMid),
            Color.Transparent
        ),
        center = c2Center,
        radius = maxDim * 0.54f
    ),
    radius = maxDim * 0.54f,
    center = c2Center
)
// Orb 3: Diagonal figure-8 ambient wave
val c3Center = Offset(
    x = w * (0.50f + 0.34f * sin(phase + 3.14f)),
    y = h * (0.46f + 0.26f * sin(phase * 2f))
)
drawCircle(
    brush = Brush.radialGradient(
        colors = listOf(
            c3.copy(alpha = orbAlphaCore * 0.92f),
            c3.copy(alpha = orbAlphaMid * 0.85f),
            Color.Transparent
        ),
        center = c3Center,
        radius = maxDim * 0.48f
    ),
    radius = maxDim * 0.48f,
    center = c3Center
)
// Orb 4: Soft accent highlight orb
val c4Center = Offset(
    x = w * (0.36f + 0.28f * cos(phase + 4.4f)),
    y = h * (0.78f + 0.20f * sin(phase + 4.4f))
)
drawCircle(
    brush = Brush.radialGradient(
        colors = listOf(
            c4.copy(alpha = orbAlphaCore * 0.85f),
            c4.copy(alpha = orbAlphaMid * 0.75f),
            Color.Transparent
        ),
        center = c4Center,
```

```
                radius = maxDim * 0.46f
            ),
            radius = maxDim * 0.46f,
            center = c4Center
        )
        // Uniform Matte Frosted Glass Veil (No top or bottom white fade border!)
        drawRect(
            color = if (isDark) Color(0x24050811) else Color(0x18101824)
        )
        }
    }
    content()
  }
}
@Composable
fun HappAmbientBackground(
  isDark: Boolean,
  modifier: Modifier = Modifier,
  content: @Composable BoxScope.() -> Unit
) {
  val isAmoled = AppRepo.isAmoledEffective()
  val baseGradient = if (isAmoled) {
    Brush.verticalGradient(
      colors = listOf(Color(0xFF000000), Color(0xFF000000))
    )
  } else if (isDark) {
    Brush.verticalGradient(
      colors = listOf(
        Color(0xFF1C1B5F),
        Color(0xFF23206D),
        Color(0xFF191957),
        Color(0xFF1E1B52),
        Color(0xFF161336)
      )
    )
  } else {
    Brush.verticalGradient(
      colors = listOf(
        Color(0xFFEAECF5),
        Color(0xFFF0F2FA),
        Color(0xFFF6F7FC),
        Color(0xFFFAFAFE),
        Color(0xFFFFFFFF)
```

```
            )
        )
    }
    Box(
        modifier = modifier
            .fillMaxSize()
            .background(baseGradient),
        content = content
    )
}
// Per-Character Soft Pastel RGB Animated Typography for "NowLock"
private val SOFT_RGB_PALETTE = listOf(
    Color(0xFF67E8F9),
    Color(0xFF818CF8),
    Color(0xFFC084FC),
    Color(0xFFF472B6),
    Color(0xFFFBBF24),
    Color(0xFF5EEAD4)
)
private fun sampleSmoothRgbPalette(position: Float): Color {
    val size = SOFT_RGB_PALETTE.size
    val pos = ((position % 1f) + 1f) % 1f * size
    val idx = floor(pos).toInt() % size
    val nextIdx = (idx + 1) % size
    val frac = pos - floor(pos)
    return lerp(SOFT_RGB_PALETTE[idx], SOFT_RGB_PALETTE[nextIdx], frac)
}
@Composable
fun AnimatedRgbBrandText(
    text: String,
    fontSize: TextUnit,
    modifier: Modifier = Modifier
) {
    val isLowEnd = AppRepo.isLowEndHardware
    val phase = if (isLowEnd) {
        0.15f
    } else {
        val infiniteTransition = rememberInfiniteTransition(label = "nowlockRgbWave")
        val animPhase by infiniteTransition.animateFloat(
            initialValue = 0f,
            targetValue = 1f,
            animationSpec = infiniteRepeatable(
                animation = tween(durationMillis = 5600, easing = LinearEasing),
```

```
                repeatMode = RepeatMode.Restart
            ),
            label = "rgbWavePhase"
        )
        animPhase
    }
    val annotated = buildAnnotatedString {
        text.forEachIndexed { idx, ch ->
            val charColor = sampleSmoothRgbPalette(phase + idx * 0.11f)
            withStyle(
                SpanStyle(
                    color = charColor,
                    fontSize = fontSize,
                    fontWeight = FontWeight.ExtraBold,
                    fontFamily = FontFamily.SansSerif,
                    letterSpacing = 1.1.sp
                )
            ) {
                append(ch)
            }
        }
    }
    Text(
        text = annotated,
        modifier = modifier
    )
}
@Composable
fun NotchTranslucentStatusBarOverlay(isDark: Boolean) {
    val overlayBaseColor = if (isDark) Color(0xFF1C1B5F) else Color(0xFFEAECF5)
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .windowInsetsTopHeight(WindowInsets.statusBars)
            .background(
                Brush.verticalGradient(
                    colors = listOf(
                        overlayBaseColor.copy(alpha = 0.68f),
                        overlayBaseColor.copy(alpha = 0.25f),
                        Color.Transparent
                    )
                )
            )
```

```
            )
        }
        @Composable
        fun GlassyNavigationBarOverlay(isDark: Boolean, modifier: Modifier = Modifier) {
            val overlayBaseColor = if (isDark) Color(0xFF161336) else Color(0xFFFFFFFF)
            Box(
                modifier = modifier
                    .fillMaxWidth()
                    .windowInsetsBottomHeight(WindowInsets.navigationBars)
                    .background(
                        Brush.verticalGradient(
                            colors = listOf(
                                Color.Transparent,
                                overlayBaseColor.copy(alpha = 0.42f),
                                overlayBaseColor.copy(alpha = 0.75f)
                            )
                        )
                    )
            )
        }
        // Instant Overlay Lock Bridge (Attaches a visible SYSTEM_ALERT_WINDOW shield in
0ms so Android 14/15 ALWAYS allows LockScreenActivity to launch over third-party apps!)
        object OverlayLockBridge {
            private val mainHandler = Handler(Looper.getMainLooper())
            private var shieldView: FrameLayout? = null
            private var isShieldAttached = false
            fun showShieldAndLaunchLockActivity(ctx: Context, targetPkg: String) {
                if (AppRepo.isWithinGracePeriod(targetPkg) || LockScreenActivity.isShowing) return
                mainHandler.post {
                    if (AppRepo.isWithinGracePeriod(targetPkg) || LockScreenActivity.isShowing)
return@post
                    val appCtx = ctx.applicationContext
                    // Attach a tiny 1x1 invisible overlay token for 150ms solely to grant Android 14/15
BAL exemption without blocking touches!
                    if (Settings.canDrawOverlays(appCtx) && !isShieldAttached) {
                        runCatching {
                            val wm = appCtx.getSystemService(Context.WINDOW_SERVICE) as?
WindowManager
                            if (wm != null) {
                                val tokenView = FrameLayout(appCtx)
                                val lp = WindowManager.LayoutParams(
                                    1,
                                    1,
```

```
                        WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY,
                        WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE or
                            WindowManager.LayoutParams.FLAG_NOT_TOUCHABLE or
                            WindowManager.LayoutParams.FLAG_LAYOUT_IN_SCREEN,
                        PixelFormat.TRANSLUCENT
                    ).apply {
                        gravity = Gravity.TOP or Gravity.START
                    }
                    wm.addView(tokenView, lp)
                    shieldView = tokenView
                    isShieldAttached = true
                }
            }
        }
        startLockActivityNow(ctx, targetPkg)
        mainHandler.postDelayed({
            dismissShield(appCtx)
        }, 220L)
    }
}
private fun startLockActivityNow(ctx: Context, targetPkg: String) {
    val intent = Intent(ctx, LockScreenActivity::class.java).apply {
        addFlags(
            Intent.FLAG_ACTIVITY_NEW_TASK or
            Intent.FLAG_ACTIVITY_CLEAR_TOP or
            Intent.FLAG_ACTIVITY_SINGLE_TOP or
            Intent.FLAG_ACTIVITY_NO_ANIMATION or
            Intent.FLAG_ACTIVITY_EXCLUDE_FROM_RECENTS
        )
        putExtra("target_pkg", targetPkg)
    }
    runCatching {
        val pi = PendingIntent.getActivity(
            ctx,
            targetPkg.hashCode(),
            intent,
            PendingIntent.FLAG_UPDATE_CURRENT or
PendingIntent.FLAG_IMMUTABLE
        )
        pi.send()
    }
    runCatching { ctx.startActivity(intent) }
}
```

```
            fun dismissShield(ctx: Context) {
                mainHandler.post {
                    val v = shieldView ?: return@post
                    if (isShieldAttached) {
                        runCatching {
                            val wm =
ctx.applicationContext.getSystemService(Context.WINDOW_SERVICE) as? WindowManager
                            wm?.removeViewImmediate(v)
                        }
                    }
                    isShieldAttached = false
                    shieldView = null
                }
            }
        }
        // Real Silent Front-Camera Intruder Photo Saver -> Saves directly to Phone Gallery
(Pictures/NowLock_Intruders)
        object IntruderPhotoSaver {
            @Volatile private var isCapturing = false
            @Volatile private var lastCaptureTime = 0L
            fun captureFrontCameraToGallery(context: Context, targetPkg: String) {
                val now = System.currentTimeMillis()
                if (isCapturing || (now - lastCaptureTime) < 2500L) return
                if (ContextCompat.checkSelfPermission(context, Manifest.permission.CAMERA) !=
PackageManager.PERMISSION_GRANTED) {
                    return
                }
                val cameraManager = context.getSystemService(Context.CAMERA_SERVICE) as?
CameraManager ?: return
                val frontCameraId = runCatching {
                    cameraManager.cameraIdList.firstOrNull { id ->
                        val chars = cameraManager.getCameraCharacteristics(id)
                        chars.get(CameraCharacteristics.LENS_FACING) ==
CameraCharacteristics.LENS_FACING_FRONT
                    } ?: cameraManager.cameraIdList.firstOrNull()
                }.getOrNull() ?: return
                val sensorOrientation = runCatching {

cameraManager.getCameraCharacteristics(frontCameraId).get(CameraCharacteristics.SENSO
R_ORIENTATION) ?: 270
                }.getOrDefault(270)
                isCapturing = true
                lastCaptureTime = now
```

```
            val bgThread = HandlerThread("NowLockIntruderCam").apply { start() }
            val bgHandler = Handler(bgThread.looper)
            runCatching {
                val imageReader = ImageReader.newInstance(640, 480, ImageFormat.JPEG, 2)
                val dummyTexture = SurfaceTexture(10).apply { setDefaultBufferSize(640, 480) }
                val dummySurface = Surface(dummyTexture)
                imageReader.setOnImageAvailableListener({ reader ->
                    val image = runCatching { reader.acquireLatestImage() }.getOrNull()
                    if (image != null) {
                        runCatching {
                            val buffer = image.planes[0].buffer
                            val bytes = ByteArray(buffer.remaining())
                            buffer.get(bytes)
                            saveJpegBytesToGallery(context, bytes, targetPkg, sensorOrientation)
                        }
                        runCatching { image.close() }
                    }
                    runCatching { dummySurface.release() }
                    runCatching { dummyTexture.release() }
                    runCatching { reader.close() }
                    runCatching { bgThread.quitSafely() }
                    isCapturing = false
                }, bgHandler)
                cameraManager.openCamera(frontCameraId, object :
CameraDevice.StateCallback() {
                    override fun onOpened(camera: CameraDevice) {
                        runCatching {
                            val targets = listOf(dummySurface, imageReader.surface)
                            @Suppress("DEPRECATION")
                            camera.createCaptureSession(targets, object :
CameraCaptureSession.StateCallback() {
                                override fun onConfigured(session: CameraCaptureSession) {
                                    runCatching {
                                        val previewReq =
camera.createCaptureRequest(CameraDevice.TEMPLATE_PREVIEW).apply {
                                            addTarget(dummySurface)
                                            set(CaptureRequest.CONTROL_AF_MODE,
CaptureRequest.CONTROL_AF_MODE_CONTINUOUS_PICTURE)
                                            set(CaptureRequest.CONTROL_AE_MODE,
CaptureRequest.CONTROL_AE_MODE_ON)
                                        }.build()
                                        session.setRepeatingRequest(previewReq, null, bgHandler)
                                        bgHandler.postDelayed({
```

```
                                    runCatching {
                                        val stillReq =
camera.createCaptureRequest(CameraDevice.TEMPLATE_STILL_CAPTURE).apply {
                                            addTarget(imageReader.surface)
                                            set(CaptureRequest.CONTROL_AE_MODE,
CaptureRequest.CONTROL_AE_MODE_ON)
                                            set(CaptureRequest.JPEG_ORIENTATION,
sensorOrientation)
                                        }.build()
                                        session.capture(stillReq, object :
CameraCaptureSession.CaptureCallback() {
                                            override fun onCaptureCompleted(
                                                s: CameraCaptureSession,
                                                r: CaptureRequest,
                                                res: android.hardware.camera2.TotalCaptureResult
                                            ) {
                                                bgHandler.postDelayed({ runCatching { camera.close() }
}, 150L)
                                            }
                                        }, bgHandler)
                                    }.onFailure { runCatching { camera.close() } }
                                }, 350L)
                            }.onFailure { runCatching { camera.close() } }
                        }
                        override fun onConfigureFailed(session: CameraCaptureSession) {
                            runCatching { camera.close() }
                            runCatching { bgThread.quitSafely() }
                        }
                    }, bgHandler)
                }.onFailure {
                    runCatching { camera.close() }
                    runCatching { bgThread.quitSafely() }
                }
            }
            override fun onDisconnected(camera: CameraDevice) {
                runCatching { camera.close() }
                runCatching { bgThread.quitSafely() }
            }
            override fun onError(camera: CameraDevice, error: Int) {
                runCatching { camera.close() }
                runCatching { bgThread.quitSafely() }
            }
        }, bgHandler)
```

```
            }.onFailure {
                isCapturing = false
                runCatching { bgThread.quitSafely() }
            }
        }
        private fun saveJpegBytesToGallery(context: Context, jpegBytes: ByteArray, targetPkg:
String, sensorOrientation: Int) {
            val originalBmp = BitmapFactory.decodeByteArray(jpegBytes, 0, jpegBytes.size) ?:
return
            val finalBmp = if (originalBmp.width > originalBmp.height) {
                val matrix = Matrix().apply { postRotate(sensorOrientation.toFloat()) }
                Bitmap.createBitmap(originalBmp, 0, 0, originalBmp.width, originalBmp.height,
matrix, true)
            } else {
                originalBmp
            }
            val timestamp = SimpleDateFormat("yyyyMMdd_HHmmss",
Locale.US).format(Date())
            val filename = "NowLock_Intruder_${timestamp}.jpg"
            val resolver = context.contentResolver
            val contentValues = ContentValues().apply {
                put(MediaStore.Images.Media.DISPLAY_NAME, filename)
                put(MediaStore.Images.Media.MIME_TYPE, "image/jpeg")
                put(MediaStore.Images.Media.TITLE, "NowLock Intruder ($targetPkg)")
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
                    put(MediaStore.Images.Media.RELATIVE_PATH,
Environment.DIRECTORY_PICTURES + "/NowLock_Intruders")
                    put(MediaStore.Images.Media.IS_PENDING, 1)
                }
            }
            val imageUri =
resolver.insert(MediaStore.Images.Media.EXTERNAL_CONTENT_URI, contentValues) ?:
return
            var outStream: OutputStream? = null
            runCatching {
                outStream = resolver.openOutputStream(imageUri)
                outStream?.let { finalBmp.compress(Bitmap.CompressFormat.JPEG, 92, it) }
            }
            runCatching { outStream?.close() }
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
                val updateValues = ContentValues().apply {
                    put(MediaStore.Images.Media.IS_PENDING, 0)
                }
```

```
                runCatching { resolver.update(imageUri, updateValues, null, null) }
            }
        }
    }
    object AppRepo {
        private const val PREFS_NAME = "nowlock_v65_prefs"
        private const val KEY_PIN_HASH = "nowlock_pin_hash"
        private const val KEY_PATTERN_HASH = "nowlock_pattern_hash"
        private const val KEY_COMBO_MODE = "nowlock_combo_mode"
        private const val KEY_FINGERPRINT_ENABLED = "nowlock_fp_enabled"
        private const val KEY_ICON_MODE = "nowlock_icon_mode"
        private const val KEY_RELOCK_MINUTES = "nowlock_relock_minutes"
        private const val KEY_RECENTS_PRIVACY = "nowlock_recents_privacy"
        private const val KEY_LOCKED_PKGS = "nowlock_locked_pkgs"
        private const val KEY_LOCKED_PKGS_CSV = "nowlock_locked_pkgs_csv"
        private const val KEY_THEME_MODE = "nowlock_theme_mode"
        private const val KEY_APP_LANG = "nowlock_app_lang_v76"
        private const val KEY_VIBRATE = "nowlock_vibrate_on_error"
        private const val KEY_INTRUDER = "nowlock_intruder_selfie"
        private const val KEY_INTRUDER_COUNT = "nowlock_intruder_count"
        private const val KEY_UNINSTALL = "nowlock_uninstall_protect"
            private const val KEY_CONSEC_FAILS = "nowlock_consec_fails"
        private const val KEY_LOCKOUT_UNTIL_MS = "nowlock_lockout_until_ms"
        private const val KEY_LAUNCHER_HIDDEN = "nowlock_launcher_hidden_v90"
        private const val KEY_LOCK_ON_SCREEN_OFF =
"nowlock_lock_on_screen_off_v90"
        private const val KEY_DECOY_ENABLED = "nowlock_decoy_enabled_v90"
        private const val KEY_DECOY_PIN_HASH = "nowlock_decoy_pin_hash_v90"
        private const val KEY_DECOY_PATTERN_HASH = "nowlock_decoy_pat_hash_v90"
        private const val KEY_SEC_LOGS = "nowlock_security_logs_v90"
        private lateinit var prefs: SharedPreferences
        private val lockedMemorySet = ConcurrentHashMap.newKeySet<String>()
        private val dynamicHomePackages = ConcurrentHashMap.newKeySet<String>()
        private val dynamicImePackages = ConcurrentHashMap.newKeySet<String>()
        val unlockedSession = ConcurrentHashMap.newKeySet<String>()
        val unlockTimestamps = ConcurrentHashMap<String, Long>()
        @Volatile var cachedLauncherApps: List<RealAppItem>? = null
        @Volatile var isLowEndHardware: Boolean = false
        @Volatile var lastActivePackage: String = ""
        @Volatile var lastLockShownTime: Long = 0L
        @Volatile var settingsNavigationGraceUntil: Long = 0L
        fun markLeavingForSettings(ctx: Context) {
            val now = System.currentTimeMillis()
```

```
            settingsNavigationGraceUntil = now + 90_000L
            unlockedSession.add(ctx.packageName)
            unlockTimestamps[ctx.packageName] = now
        }
        fun isSelfSettingsGraceActive(): Boolean = System.currentTimeMillis() <
settingsNavigationGraceUntil
        fun init(ctx: Context) {
            if (!::prefs.isInitialized) {
                val storageCtx = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N &&
ctx.isDeviceProtectedStorage) {
                    ctx
                } else {
                    ctx.applicationContext ?: ctx
                }
                prefs = storageCtx.getSharedPreferences(PREFS_NAME,
Context.MODE_PRIVATE)
                reloadLockedPackagesFromPrefs()
                refreshDynamicSystemPackages(storageCtx)
            }
        }
        fun getConsecutiveFailedAttempts(): Int = prefs.getInt(KEY_CONSEC_FAILS, 0)
        fun getLockoutRemainingSeconds(): Int {
            val untilMs = prefs.getLong(KEY_LOCKOUT_UNTIL_MS, 0L)
            val remMs = untilMs - System.currentTimeMillis()
            return if (remMs > 0L) ((remMs + 999L) / 1000L).toInt() else 0
        }
        // Progressive Lockout Rule:
        // Attempts 1..9: no timer.
        // Attempt 10: 15s lockout.
        // Every consecutive wrong attempt after 10 (11, 12, 13...): adds +5s (20s, 25s, 30s...)!
        fun registerWrongCredentialAttempt(ctx: Context, targetPkg: String, methodLabel:
String = "PIN"): Pair<Int, Int> {
            val fails = getConsecutiveFailedAttempts() + 1
            prefs.edit().putInt(KEY_CONSEC_FAILS, fails).commit()
            appendSecurityLog(ctx, targetPkg, methodLabel)
            val effCfg = getEffectiveConfig(targetPkg)
            if (fails >= 3 && effCfg.intruderSelfie) {
                recordIntruderAttempt(ctx, targetPkg)
            }
            if (fails >= 10) {
                appendSecurityLog(ctx, targetPkg, "LOCKOUT_TRIGGERED")
                val penaltySec = 15 + (fails - 10) * 5
                val untilMs = System.currentTimeMillis() + (penaltySec * 1000L)
```

```
                prefs.edit().putLong(KEY_LOCKOUT_UNTIL_MS, untilMs).commit()
                return fails to penaltySec
            }
            return fails to 0
        }
        fun appendSecurityLog(ctx: Context, targetPkg: String, method: String) {
            val pkg = if (targetPkg.isBlank()) ctx.packageName else targetPkg
            val label = runCatching {
                if (pkg == ctx.packageName) "NowLock"
                else {
                    val ai = ctx.packageManager.getApplicationInfo(pkg, 0)
                    ctx.packageManager.getApplicationLabel(ai).toString()
                }
            }.getOrDefault(pkg)
            val ts = SimpleDateFormat("yyyy/MM/dd HH:mm:ss", Locale.US).format(Date())
            val entryLine = "$ts|||$pkg|||$label|||$method"
            val currentRaw = prefs.getString(KEY_SEC_LOGS, "") ?: ""
            val lines = currentRaw.lines().filter { it.contains("|||") }.toMutableList()
            lines.add(0, entryLine)
            val trimmed = lines.take(40).joinToString("\n")
            prefs.edit().putString(KEY_SEC_LOGS, trimmed).commit()
        }
        fun getSecurityLogs(): List<SecurityLogEntry> {
            val raw = prefs.getString(KEY_SEC_LOGS, "") ?: ""
            if (raw.isBlank()) return emptyList()
            return raw.lines().mapNotNull { line ->
                val parts = line.split("|||")
                if (parts.size >= 4) {
                    SecurityLogEntry(
                        timestamp = parts[0],
                        targetPkg = parts[1],
                        targetAppLabel = parts[2],
                        method = parts[3]
                    )
                } else null
            }
        }
        fun clearSecurityLogs() {
            prefs.edit().putString(KEY_SEC_LOGS, "").commit()
        }
        fun isLockOnScreenOffEnabled(): Boolean =
prefs.getBoolean(KEY_LOCK_ON_SCREEN_OFF, true)
        fun setLockOnScreenOffEnabled(enabled: Boolean) {
```

```
            prefs.edit().putBoolean(KEY_LOCK_ON_SCREEN_OFF, enabled).commit()
        }
        fun isDecoyEnabled(): Boolean = prefs.getBoolean(KEY_DECOY_ENABLED, false)
        fun setDecoyEnabled(enabled: Boolean) {
            prefs.edit().putBoolean(KEY_DECOY_ENABLED, enabled).commit()
        }
        private const val KEY_SEC_Q_INDEX = "nowlock_sec_q_idx_v902"
        private const val KEY_SEC_Q_HASH = "nowlock_sec_q_hash_v902"
        fun isSecurityQuestionSet(): Boolean = prefs.getString(KEY_SEC_Q_HASH, null) !=
null
        fun getSecurityQuestionIndex(): Int = prefs.getInt(KEY_SEC_Q_INDEX, 0).coerceIn(0,
4)
        fun saveSecurityQuestion(index: Int, answer: String) {
            val normalized = answer.trim()
            prefs.edit()
                .putInt(KEY_SEC_Q_INDEX, index.coerceIn(0, 4))
                .putString(KEY_SEC_Q_HASH, sha256("sec_q_exact_" + normalized))
                .commit()
        }
        fun verifySecurityQuestionAnswer(answer: String): Boolean {
            val saved = prefs.getString(KEY_SEC_Q_HASH, null) ?: return false
            val candidate = sha256("sec_q_exact_" + answer.trim())
            return MessageDigest.isEqual(candidate.toByteArray(Charsets.UTF_8),
saved.toByteArray(Charsets.UTF_8))
        }
        fun resetPrimaryCredentialsBySecurityQuestion() {
            prefs.edit()
                .remove(KEY_PIN_HASH)
                .remove(KEY_PATTERN_HASH)
                .putInt(KEY_CONSEC_FAILS, 0)
                .putLong(KEY_LOCKOUT_UNTIL_MS, 0L)
                .commit()
        }
        fun isDecoyPinSet(): Boolean = prefs.getString(KEY_DECOY_PIN_HASH, null) != null
        fun isDecoyPatternSet(): Boolean = prefs.getString(KEY_DECOY_PATTERN_HASH,
null) != null
        fun setDecoyPin(pin: String) {
            prefs.edit().putString(KEY_DECOY_PIN_HASH, sha256("decoy_pin_" +
pin)).putBoolean(KEY_DECOY_ENABLED, true).commit()
        }
        fun verifyDecoyPinRaw(pin: String): Boolean {
            val saved = prefs.getString(KEY_DECOY_PIN_HASH, null) ?: return false
            val candidate = sha256("decoy_pin_" + pin)
```

```
            return MessageDigest.isEqual(candidate.toByteArray(Charsets.UTF_8),
saved.toByteArray(Charsets.UTF_8))
        }
        fun verifyDecoyPin(pin: String): Boolean {
            if (!isDecoyEnabled()) return false
            return verifyDecoyPinRaw(pin)
        }
        fun setDecoyPattern(pat: String) {
            prefs.edit().putString(KEY_DECOY_PATTERN_HASH, sha256("decoy_pat_" +
pat)).putBoolean(KEY_DECOY_ENABLED, true).commit()
        }
        fun verifyDecoyPatternRaw(pat: String): Boolean {
            val saved = prefs.getString(KEY_DECOY_PATTERN_HASH, null) ?: return false
            val candidate = sha256("decoy_pat_" + pat)
            return MessageDigest.isEqual(candidate.toByteArray(Charsets.UTF_8),
saved.toByteArray(Charsets.UTF_8))
        }
        fun verifyDecoyPattern(pat: String): Boolean {
            if (!isDecoyEnabled()) return false
            return verifyDecoyPatternRaw(pat)
        }
        fun getPerAppConfig(pkg: String): PerAppLockConfig {
            val raw = prefs.getString("per_app_cfg_$pkg", null) ?: return PerAppLockConfig(
                useCustom = false,
                comboMode = getComboMode(),
                fpEnabled = isFingerprintEnabled(),
                relockMinutes = getRelockDelayMinutes(),
                lockOnScreenOff = isLockOnScreenOffEnabled(),
                intruderSelfie = isIntruderSelfieEnabled(),
                vibrateOnError = isVibrationEnabled(),
                decoyCrashEnabled = isDecoyEnabled()
            )
            return runCatching {
                val p = raw.split("|")
                PerAppLockConfig(
                    useCustom = p[0].toBoolean(),
                    comboMode = LockComboMode.valueOf(p[1]),
                    fpEnabled = p[2].toBoolean(),
                    relockMinutes = p[3].toInt(),
                    lockOnScreenOff = p[4].toBoolean(),
                    intruderSelfie = p[5].toBoolean(),
                    vibrateOnError = p[6].toBoolean(),
                    decoyCrashEnabled = p.getOrNull(7)?.toBoolean() ?: isDecoyEnabled()
```

```
            )
        }.getOrDefault(PerAppLockConfig())
    }
    fun savePerAppConfig(pkg: String, cfg: PerAppLockConfig) {
        val serialized = listOf(
            cfg.useCustom,
            cfg.comboMode.name,
            cfg.fpEnabled,
            cfg.relockMinutes,
            cfg.lockOnScreenOff,
            cfg.intruderSelfie,
            cfg.vibrateOnError,
            cfg.decoyCrashEnabled
        ).joinToString("|")
        prefs.edit().putString("per_app_cfg_$pkg", serialized).commit()
    }
    fun getEffectiveConfig(pkg: String): PerAppLockConfig {
        val c = getPerAppConfig(pkg)
        return if (c.useCustom) c else PerAppLockConfig(
            useCustom = false,
            comboMode = getComboMode(),
            fpEnabled = isFingerprintEnabled(),
            relockMinutes = getRelockDelayMinutes(),
            lockOnScreenOff = isLockOnScreenOffEnabled(),
            intruderSelfie = isIntruderSelfieEnabled(),
            vibrateOnError = isVibrationEnabled(),
            decoyCrashEnabled = isDecoyEnabled()
        )
    }
    fun resetFailedAttempts() {
        if (prefs.getInt(KEY_CONSEC_FAILS, 0) != 0 ||
prefs.getLong(KEY_LOCKOUT_UNTIL_MS, 0L) != 0L) {
            prefs.edit().putInt(KEY_CONSEC_FAILS,
0).putLong(KEY_LOCKOUT_UNTIL_MS, 0L).commit()
        }
    }
    fun refreshDynamicSystemPackages(ctx: Context) {
        runCatching {
            val am = ctx.getSystemService(Context.ACTIVITY_SERVICE) as?
ActivityManager
            val cores = Runtime.getRuntime().availableProcessors()
            isLowEndHardware = (am?.isLowRamDevice == true) || (cores <= 4)
        }
```

```
            runCatching {
                val pm = ctx.packageManager
                val homeIntent =
Intent(Intent.ACTION_MAIN).addCategory(Intent.CATEGORY_HOME)
                pm.queryIntentActivities(homeIntent,
PackageManager.MATCH_DEFAULT_ONLY).forEach { ri ->
                    ri.activityInfo?.packageName?.let { if (it.isNotEmpty())
dynamicHomePackages.add(it) }
                }
                pm.resolveActivity(homeIntent,
PackageManager.MATCH_DEFAULT_ONLY)?.activityInfo?.packageName?.let {
                    if (it.isNotEmpty()) dynamicHomePackages.add(it)
                }
            }
            runCatching {
                val imm = ctx.getSystemService(Context.INPUT_METHOD_SERVICE) as?
InputMethodManager
                imm?.enabledInputMethodList?.forEach { imi ->
                    if (imi.packageName.isNotEmpty())
dynamicImePackages.add(imi.packageName)
                }
            }
        }
        private fun reloadLockedPackagesFromPrefs() {
            val csv = prefs.getString(KEY_LOCKED_PKGS_CSV, null)
            if (csv != null) {
                lockedMemorySet.clear()
                csv.split(",").map { it.trim() }.filter { it.isNotEmpty() }.forEach {
lockedMemorySet.add(it) }
            } else {
                val legacy = prefs.getStringSet(KEY_LOCKED_PKGS, emptySet()) ?: emptySet()
                lockedMemorySet.clear()
                lockedMemorySet.addAll(legacy)
                if (legacy.isNotEmpty()) {
                    prefs.edit().putString(KEY_LOCKED_PKGS_CSV,
legacy.joinToString(",")).commit()
                }
            }
        }
        fun getLockedPackages(): Set<String> {
            reloadLockedPackagesFromPrefs()
            return lockedMemorySet.toSet()
        }
```

```
        fun isPinSet(): Boolean = prefs.getString(KEY_PIN_HASH, null) != null
        fun isPatternSet(): Boolean = prefs.getString(KEY_PATTERN_HASH, null) != null
        fun isAnyPasswordSet(): Boolean = isPinSet() || isPatternSet()
        fun setPin(pin: String) {
            prefs.edit().putString(KEY_PIN_HASH, sha256("pin_" + pin)).commit()
        }
        fun verifyPin(pin: String): Boolean {
            val saved = prefs.getString(KEY_PIN_HASH, null) ?: return false
            val candidate = sha256("pin_" + pin)
            return MessageDigest.isEqual(candidate.toByteArray(Charsets.UTF_8),
saved.toByteArray(Charsets.UTF_8))
        }
        fun setPattern(patternStr: String) {
            prefs.edit().putString(KEY_PATTERN_HASH, sha256("pat_" + patternStr)).commit()
        }
        fun verifyPattern(patternStr: String): Boolean {
            val saved = prefs.getString(KEY_PATTERN_HASH, null) ?: return false
            val candidate = sha256("pat_" + patternStr)
            return MessageDigest.isEqual(candidate.toByteArray(Charsets.UTF_8),
saved.toByteArray(Charsets.UTF_8))
        }
        fun getComboMode(): LockComboMode =
            runCatching {
                LockComboMode.valueOf(prefs.getString(KEY_COMBO_MODE,
LockComboMode.PIN_FINGERPRINT.name) ?: LockComboMode.PIN_FINGERPRINT.name)
            }.getOrDefault(LockComboMode.PIN_FINGERPRINT)
        fun setComboMode(m: LockComboMode) {
            prefs.edit().putString(KEY_COMBO_MODE, m.name).commit()
        }
        fun isFingerprintEnabled(): Boolean =
prefs.getBoolean(KEY_FINGERPRINT_ENABLED, true)
        fun setFingerprintEnabled(enabled: Boolean) {
            prefs.edit().putBoolean(KEY_FINGERPRINT_ENABLED, enabled).commit()
        }
        fun getRelockDelayMinutes(): Int = prefs.getInt(KEY_RELOCK_MINUTES, 0)
        fun setRelockDelayMinutes(minutes: Int) {
            prefs.edit().putInt(KEY_RELOCK_MINUTES, minutes).commit()
        }
        fun isRecentsPrivacyEnabled(): Boolean =
prefs.getBoolean(KEY_RECENTS_PRIVACY, true)
        fun setRecentsPrivacyEnabled(enabled: Boolean) {
            prefs.edit().putBoolean(KEY_RECENTS_PRIVACY, enabled).commit()
        }
```

```
        fun getIconMode(): AppIconMode =
            runCatching {
                AppIconMode.valueOf(prefs.getString(KEY_ICON_MODE,
AppIconMode.PADLOCK.name) ?: AppIconMode.PADLOCK.name)
            }.getOrDefault(AppIconMode.PADLOCK)
        private val appPaletteCache = ConcurrentHashMap<String, List<Color>>()
        fun getOrExtractAppColorPalette(ctx: Context, pkg: String): List<Color> {
            val key = if (pkg.isBlank()) ctx.packageName else pkg
            appPaletteCache[key]?.let { return it }
            val extracted = runCatching {
                val d = ctx.packageManager.getApplicationIcon(key)
                val bmp = Bitmap.createBitmap(28, 28, Bitmap.Config.ARGB_8888)
                val canvas = Canvas(bmp)
                d.setBounds(0, 0, 28, 28)
                d.draw(canvas)
                val hsv = FloatArray(3)
                val distinctHues = mutableListOf<Pair<Float, Color>>()
                for (y in 2 until 26 step 2) {
                    for (x in 2 until 26 step 2) {
                        val px = bmp.getPixel(x, y)
                        val a = android.graphics.Color.alpha(px)
                        if (a < 150) continue
                        android.graphics.Color.colorToHSV(px, hsv)
                        val sat = hsv[1]
                        val v = hsv[2]
                        if (v < 0.16f || (sat < 0.14f && v > 0.88f)) continue
                        hsv[1] = (sat * 1.28f).coerceIn(0.58f, 0.95f)
                        hsv[2] = v.coerceIn(0.62f, 0.90f)
                        val c = Color(android.graphics.Color.HSVToColor(hsv))
                        distinctHues.add(hsv[0] to c)
                    }
                }
                if (distinctHues.isEmpty()) {
                    listOf(Color(0xFF10B981), Color(0xFF059669), Color(0xFF34D399),
Color(0xFF0D9488))
                } else {
                    val sorted = distinctHues.sortedBy { it.first }
                    val baseH = sorted[sorted.size / 2].first
                    val c1 = sorted.first().second
                    val c2 = sorted[sorted.size / 2].second
                    val c3 = sorted.last().second
                    // Add rich harmonic neighbor hues derived from the app's dominant color so
movement is always vivid and visible!
```

```
                    val harmonic1 = Color(android.graphics.Color.HSVToColor(floatArrayOf((baseH
+ 42f) % 360f, 0.78f, 0.84f)))
                    val harmonic2 = Color(android.graphics.Color.HSVToColor(floatArrayOf((baseH
+ 315f) % 360f, 0.82f, 0.72f)))
                    listOf(c1, c2, harmonic1, harmonic2, c3).distinct()
                }
            }.getOrDefault(listOf(Color(0xFF10B981), Color(0xFF059669), Color(0xFF34D399),
Color(0xFF0D9488)))
            appPaletteCache[key] = extracted
            return extracted
        }
        fun isLauncherHidden(): Boolean = prefs.getBoolean(KEY_LAUNCHER_HIDDEN,
false)
        fun setLauncherHidden(ctx: Context, hidden: Boolean) {
            prefs.edit().putBoolean(KEY_LAUNCHER_HIDDEN, hidden).commit()
            applyLauncherAliasesState(ctx)
        }
        fun setIconMode(ctx: Context, mode: AppIconMode) {
            prefs.edit().putString(KEY_ICON_MODE, mode.name).commit()
            applyLauncherAliasesState(ctx)
        }
        fun applyLauncherAliasesState(ctx: Context) {
            val pm = ctx.packageManager
            val defaultAlias = ComponentName(ctx, "io.nowlock.app.LauncherDefaultAlias")
            val calcAlias = ComponentName(ctx, "io.nowlock.app.LauncherCalculatorAlias")
            if (isLauncherHidden()) {
                // Both launcher aliases disabled + MainActivity is exported=false -> 0 exported
launcher activities, so icon 100% vanishes from launcher!
                pm.setComponentEnabledSetting(defaultAlias,
PackageManager.COMPONENT_ENABLED_STATE_DISABLED,
PackageManager.DONT_KILL_APP)
                pm.setComponentEnabledSetting(calcAlias,
PackageManager.COMPONENT_ENABLED_STATE_DISABLED,
PackageManager.DONT_KILL_APP)
            } else {
                val mode = getIconMode()
                if (mode == AppIconMode.PADLOCK) {
                    pm.setComponentEnabledSetting(defaultAlias,
PackageManager.COMPONENT_ENABLED_STATE_ENABLED,
PackageManager.DONT_KILL_APP)
                    pm.setComponentEnabledSetting(calcAlias,
PackageManager.COMPONENT_ENABLED_STATE_DISABLED,
PackageManager.DONT_KILL_APP)
```

```
                } else {
                    pm.setComponentEnabledSetting(calcAlias,
PackageManager.COMPONENT_ENABLED_STATE_ENABLED,
PackageManager.DONT_KILL_APP)
                    pm.setComponentEnabledSetting(defaultAlias,
PackageManager.COMPONENT_ENABLED_STATE_DISABLED,
PackageManager.DONT_KILL_APP)
                }
            }
        }
        fun toggleAppLock(pkg: String, lock: Boolean) {
            reloadLockedPackagesFromPrefs()
            if (lock) {
                lockedMemorySet.add(pkg)
                unlockedSession.remove(pkg)
                unlockTimestamps.remove(pkg)
            } else {
                lockedMemorySet.remove(pkg)
                unlockedSession.remove(pkg)
                unlockTimestamps.remove(pkg)
            }
            val csv = lockedMemorySet.joinToString(",")
            prefs.edit()
                .putString(KEY_LOCKED_PKGS_CSV, csv)
                .putStringSet(KEY_LOCKED_PKGS, HashSet(lockedMemorySet))
                .commit()
        }
        fun markUnlocked(pkg: String) {
            val now = System.currentTimeMillis()
            unlockedSession.add(pkg)
            unlockTimestamps[pkg] = now
            prefs.edit().putLong("unlock_ts_$pkg", now).commit()
            if (pkg.isNotEmpty()) {
                lastActivePackage = pkg
            }
        }
        fun isWithinGracePeriod(pkg: String): Boolean {
            if (pkg.isEmpty()) return true
            val now = System.currentTimeMillis()
            val lastTs = unlockTimestamps[pkg] ?: prefs.getLong("unlock_ts_$pkg", 0L)
            // Always grant a 4.5-second post-unlock transition immunity so closing the lock
screen never re-locks the app!
            if (lastTs > 0L && (now - lastTs) < 4500L) {
```

```
            unlockedSession.add(pkg)
            return true
        }
        val delayMin = getEffectiveConfig(pkg).relockMinutes
        if (delayMin <= 0) {
            return unlockedSession.contains(pkg)
        }
        val windowMs = delayMin * 60_000L
        if (lastTs <= 0L) return false
        val valid = (now - lastTs) < windowMs
        if (!valid) {
            unlockedSession.remove(pkg)
            unlockTimestamps.remove(pkg)
        }
        return valid
    }
    fun clearAllUnlockedSessionsOnScreenOff() {
        if (getRelockDelayMinutes() <= 0 || isLockOnScreenOffEnabled()) {
            val allLocked = getLockedPackages()
            allLocked.forEach { pkg ->
                prefs.edit().putLong("unlock_ts_$pkg", 0L).commit()
            }
            unlockedSession.clear()
            unlockTimestamps.clear()
            lastActivePackage = ""
        } else {
            // Check per-app lockOnScreenOff overrides
            val toLock = getLockedPackages().filter { getEffectiveConfig(it).lockOnScreenOff }
            toLock.forEach { pkg ->
                unlockedSession.remove(pkg)
                unlockTimestamps.remove(pkg)
                prefs.edit().putLong("unlock_ts_$pkg", 0L).commit()
            }
        }
    }
    fun isIgnoredSystemOrOverlayPackage(pkg: String, myPkg: String): Boolean {
        if (pkg.isBlank() || pkg == myPkg || pkg == "android") return true
        if (dynamicImePackages.contains(pkg)) return true
        val lower = pkg.lowercase()
        // Covers Samsung One UI, Xiaomi MIUI/HyperOS, OPPO/OnePlus ColorOS, Vivo
OriginOS, Huawei EMUI, Pixel/AOSP
        // including keyboards, biometric prompts, permission controllers, incoming call
screens, and floating sidebars
```

```
    return lower.contains("systemui") ||
        lower.contains("inputmethod") ||
        lower.contains("keyboard") ||
        lower.contains("honeyboard") ||
        lower.contains("latinime") ||
        lower.contains("swiftkey") ||
        lower.contains("biometric") ||
        lower.contains("credentialmanager") ||
        lower.contains("permissioncontroller") ||
        lower.contains("packageinstaller") ||
        lower.contains("incallui") ||
        lower.contains("server.telecom") ||
        lower.contains("cocktailbarservice") ||
        lower.contains("smartsidebar") ||
        lower.contains("smartmultiwindow") ||
        lower.contains("miui.securitycenter") ||
        lower.contains("miui.notification") ||
        lower.contains("FloatingFeature".lowercase()) ||
        lower.contains("samsung.android.app.aodservice") ||
        lower.contains("samsung.android.app.smartcapture") ||
        lower.contains("sec.android.app.smartclip")
}
fun isHomeLauncherPackage(pkg: String): Boolean {
    if (lockedMemorySet.contains(pkg)) return false
    if (dynamicHomePackages.contains(pkg)) return true
    val lower = pkg.lowercase()
    return lower == "com.sec.android.app.launcher" ||
        lower == "com.miui.home" ||
        lower == "com.mi.android.globallauncher" ||
        lower == "com.google.android.apps.nexuslauncher" ||
        lower == "com.oppo.launcher" ||
        lower == "com.android.launcher" ||
        lower == "com.android.launcher3" ||
        lower == "net.oneplus.launcher" ||
        lower == "com.huawei.android.launcher" ||
        lower == "com.hihonor.android.launcher" ||
        lower == "com.bbk.launcher2" ||
        lower == "com.vivo.launcher" ||
        lower == "com.nothing.launcher" ||
        lower == "com.motorola.launcher3" ||
        lower == "com.teslacoilsw.launcher" ||
        lower == "com.microsoft.launcher" ||
        lower.endsWith(".launcher") ||
```

```
                lower.endsWith(".home")
        }
        fun evaluateForegroundPackage(pkg: String, myPkg: String): Boolean {
            // Never evaluate NowLock itself or system overlays in background services!
            // NowLock's own lock screen is handled directly inside MainActivity without task
switching.
            if (isIgnoredSystemOrOverlayPackage(pkg, myPkg)) {
                return false
            }
            if (LockScreenActivity.isShowing) {
                return false
            }
            if (!isAnyPasswordSet()) return false
            val now = System.currentTimeMillis()
            val delayMin = getRelockDelayMinutes()
            val prevPkg = lastActivePackage
            if (prevPkg.isNotEmpty() && prevPkg != pkg) {
                val prevUnlockTs = unlockTimestamps[prevPkg] ?:
prefs.getLong("unlock_ts_$prevPkg", 0L)
                val isPastTransitionImmunity = (now - prevUnlockTs) >= 4500L
                if (delayMin <= 0 && isPastTransitionImmunity) {
                    if (isHomeLauncherPackage(pkg)) {
                        // Clear unlocked third-party apps whose 4.5s transition immunity has passed
                        val toRemove = unlockedSession.filter { itemPkg ->
                            val ts = unlockTimestamps[itemPkg] ?: 0L
                            (now - ts) >= 4500L
                        }
                        unlockedSession.removeAll(toRemove.toSet())
                    } else {
                        unlockedSession.remove(prevPkg)
                    }
                }
            }
            if (!isHomeLauncherPackage(pkg)) {
                lastActivePackage = pkg
            }
            val lockedSet = getLockedPackages()
            if (!lockedSet.contains(pkg)) return false
            return !isWithinGracePeriod(pkg)
        }
        fun launchLockScreenIfNeeded(ctx: Context, targetPkg: String) {
            val now = System.currentTimeMillis()
            if (now - lastLockShownTime < 300L && LockScreenActivity.activeTargetPkg ==
```

```
targetPkg && LockScreenActivity.isShowing) {
                return
            }
            lastLockShownTime = now
            if (targetPkg == ctx.packageName) {
                val intent = Intent(ctx, LockScreenActivity::class.java).apply {
                    addFlags(
                        Intent.FLAG_ACTIVITY_NEW_TASK or
                        Intent.FLAG_ACTIVITY_CLEAR_TOP or
                        Intent.FLAG_ACTIVITY_SINGLE_TOP or
                        Intent.FLAG_ACTIVITY_NO_ANIMATION
                    )
                    putExtra("target_pkg", targetPkg)
                }
                runCatching { ctx.startActivity(intent) }
            } else {
                OverlayLockBridge.showShieldAndLaunchLockActivity(ctx, targetPkg)
            }
        }
        fun isVibrationEnabled(): Boolean = prefs.getBoolean(KEY_VIBRATE, true)
        fun setVibrationEnabled(v: Boolean) = prefs.edit().putBoolean(KEY_VIBRATE,
v).commit()
        fun isIntruderSelfieEnabled(): Boolean = prefs.getBoolean(KEY_INTRUDER, true)
        fun setIntruderSelfieEnabled(v: Boolean) = prefs.edit().putBoolean(KEY_INTRUDER,
v).commit()
        fun recordIntruderAttempt(ctx: Context, targetPkg: String) {
            val c = prefs.getInt(KEY_INTRUDER_COUNT, 0) + 1
            prefs.edit().putInt(KEY_INTRUDER_COUNT, c).commit()
            IntruderPhotoSaver.captureFrontCameraToGallery(ctx.applicationContext,
targetPkg)
        }
        fun getIntruderCount(): Int = prefs.getInt(KEY_INTRUDER_COUNT, 0)
        fun isUninstallProtectionEnabled(): Boolean = prefs.getBoolean(KEY_UNINSTALL,
false)
        fun setUninstallProtectionEnabled(v: Boolean) =
prefs.edit().putBoolean(KEY_UNINSTALL, v).commit()
        fun getThemeMode(): ThemeMode =
            runCatching { ThemeMode.valueOf(prefs.getString(KEY_THEME_MODE,
ThemeMode.SYSTEM.name) ?: ThemeMode.SYSTEM.name)
}.getOrDefault(ThemeMode.SYSTEM)
        fun setThemeMode(m: ThemeMode) = prefs.edit().putString(KEY_THEME_MODE,
m.name).commit()
        fun getAppLang(): AppLang =
```

```
            runCatching { AppLang.valueOf(prefs.getString(KEY_APP_LANG,
AppLang.EN.name) ?: AppLang.EN.name) }.getOrDefault(AppLang.EN)
        fun setAppLang(lang: AppLang) = prefs.edit().putString(KEY_APP_LANG,
lang.name).commit()
        fun isAmoledEffective(): Boolean = getThemeMode() == ThemeMode.AMOLED
        fun isDarkEffective(ctx: Context): Boolean {
            return when (getThemeMode()) {
                ThemeMode.DARK, ThemeMode.AMOLED -> true
                ThemeMode.LIGHT -> false
                ThemeMode.SYSTEM -> {
                    val nightFlags = ctx.resources.configuration.uiMode and
Configuration.UI_MODE_NIGHT_MASK
                    nightFlags == Configuration.UI_MODE_NIGHT_YES
                }
            }
        }
        fun triggerShortVibration(ctx: Context, durationMs: Long = 55L) {
            if (!isVibrationEnabled()) return
            val vib = ctx.getSystemService(Context.VIBRATOR_SERVICE) as? Vibrator ?:
return
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                vib.vibrate(VibrationEffect.createOneShot(durationMs,
VibrationEffect.DEFAULT_AMPLITUDE))
            } else {
                @Suppress("DEPRECATION")
                vib.vibrate(durationMs)
            }
        }
        private fun sha256(s: String): String =
            MessageDigest.getInstance("SHA-256").digest(("nowlock_v65_salt_" +
s).toByteArray()).joinToString("") { "%02x".format(it) }
    }
    fun isAccessibilityServiceEnabled(context: Context): Boolean {
        val expected = ComponentName(context,
NowLockAccessibilityService::class.java).flattenToString()
        val enabledServices = Settings.Secure.getString(context.contentResolver,
Settings.Secure.ENABLED_ACCESSIBILITY_SERVICES) ?: return false
        val splitter = TextUtils.SimpleStringSplitter(':')
        splitter.setString(enabledServices)
        while (splitter.hasNext()) {
            val component = splitter.next()
            if (component.equals(expected, ignoreCase = true)) return true
        }
```

```
            return false
        }
        fun isUsageStatsPermissionGranted(context: Context): Boolean {
            val appOps = context.getSystemService(Context.APP_OPS_SERVICE) as?
AppOpsManager ?: return false
            val mode = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
                appOps.unsafeCheckOpNoThrow(AppOpsManager.OPSTR_GET_USAGE_STATS,
Process.myUid(), context.packageName)
            } else {
                @Suppress("DEPRECATION")
                appOps.checkOpNoThrow(AppOpsManager.OPSTR_GET_USAGE_STATS,
Process.myUid(), context.packageName)
            }
            return mode == AppOpsManager.MODE_ALLOWED
        }
        fun pollTopPackageViaUsageStats(context: Context): Pair<String, String> {
            if (!isUsageStatsPermissionGranted(context)) return "" to ""
            val usm = context.getSystemService(Context.USAGE_STATS_SERVICE) as?
UsageStatsManager ?: return "" to ""
            val now = System.currentTimeMillis()
            val events = usm.queryEvents(now - 2500L, now)
            var latestPkg = ""
            var latestClass = ""
            var latestTime = 0L
            if (events != null) {
                val event = UsageEvents.Event()
                while (events.hasNextEvent()) {
                    events.getNextEvent(event)
                    if (event.eventType == UsageEvents.Event.ACTIVITY_RESUMED ||
                        event.eventType == UsageEvents.Event.MOVE_TO_FOREGROUND
                    ) {
                        val p = event.packageName ?: ""
                        if (p.isNotEmpty() && p != context.packageName && event.timeStamp >=
latestTime) {
                            latestTime = event.timeStamp
                            latestPkg = p
                            latestClass = event.className ?: ""
                        }
                    }
                }
            }
            return latestPkg to latestClass
        }
```

```
        // Helper to post "New App Installed: Lock with NowLock?" notification
        object NewAppNotificationHelper {
            private const val CHANNEL_ID = "nowlock_new_app_prompt"
            fun showLockPromptNotification(context: Context, newPkg: String) {
                AppRepo.init(context)
                if (newPkg == context.packageName ||
AppRepo.getLockedPackages().contains(newPkg)) return
                val pm = context.packageManager
                val appLabel = runCatching {
                    val ai = pm.getApplicationInfo(newPkg, 0)
                    pm.getApplicationLabel(ai).toString()
                }.getOrNull() ?: return
                val s = getStrings(AppRepo.getAppLang())
                val nm = context.getSystemService(Context.NOTIFICATION_SERVICE) as?
NotificationManager ?: return
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                    val chan = NotificationChannel(
                        CHANNEL_ID,
                        "NowLock New App Prompt",
                        NotificationManager.IMPORTANCE_HIGH
                    )
                    nm.createNotificationChannel(chan)
                }
                val notifId = newPkg.hashCode().and(0x7FFFFFFF)
                val lockIntent = Intent(context, NewAppLockActionReceiver::class.java).apply {
                    action = "io.nowlock.app.ACTION_LOCK_NEW_APP"
                    putExtra("target_pkg", newPkg)
                    putExtra("notif_id", notifId)
                }
                val dismissIntent = Intent(context, NewAppLockActionReceiver::class.java).apply {
                    action = "io.nowlock.app.ACTION_DISMISS_NEW_APP"
                    putExtra("notif_id", notifId)
                }
                val lockPi = PendingIntent.getBroadcast(
                    context,
                    notifId,
                    lockIntent,
                    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
                )
                val dismissPi = PendingIntent.getBroadcast(
                    context,
                    notifId + 1,
                    dismissIntent,
```

```
                PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
            )
            val builder = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                Notification.Builder(context, CHANNEL_ID)
            } else {
                @Suppress("DEPRECATION")
                Notification.Builder(context)
            }
            val notification = builder
                .setSmallIcon(R.drawable.ic_nowlock_logo)
                .setContentTitle(s.newAppNotifTitle + appLabel)
                .setContentText(s.newAppNotifBody)
                .setAutoCancel(true)
                .addAction(Notification.Action.Builder(R.drawable.ic_nowlock_logo,
s.newAppActionLock, lockPi).build())
                .addAction(Notification.Action.Builder(R.drawable.ic_nowlock_logo,
s.newAppActionDismiss, dismissPi).build())
                .build()
            runCatching { nm.notify(notifId, notification) }
        }
    }
    class NewAppInstallReceiver : BroadcastReceiver() {
        override fun onReceive(context: Context?, intent: Intent?) {
            val ctx = context ?: return
            if (intent?.action != Intent.ACTION_PACKAGE_ADDED) return
            if (intent.getBooleanExtra(Intent.EXTRA_REPLACING, false)) return
            val pkg = intent.data?.schemeSpecificPart ?: return
            AppRepo.cachedLauncherApps = null
            NewAppNotificationHelper.showLockPromptNotification(ctx, pkg)
        }
    }
    class NewAppLockActionReceiver : BroadcastReceiver() {
        override fun onReceive(context: Context?, intent: Intent?) {
            val ctx = context ?: return
            AppRepo.init(ctx)
            val notifId = intent?.getIntExtra("notif_id", 0) ?: 0
            val nm = ctx.getSystemService(Context.NOTIFICATION_SERVICE) as?
NotificationManager
            if (notifId != 0) nm?.cancel(notifId)
            if (intent?.action == "io.nowlock.app.ACTION_LOCK_NEW_APP") {
                val pkg = intent.getStringExtra("target_pkg") ?: return
                AppRepo.toggleAppLock(pkg, true)
                val s = getStrings(AppRepo.getAppLang())
```

```
                Toast.makeText(ctx, "🔒 ${s.filterLocked}: $pkg", Toast.LENGTH_SHORT).show()
            }
        }
    }
    // Engine 1: Instant 0ms Accessibility + UsageStats Hybrid Service (Auto-started by
Android OS on boot!)
    class NowLockAccessibilityService : AccessibilityService() {
        private val mainHandler = Handler(Looper.getMainLooper())
        private var isPolling = false
        private val pollRunnable = object : Runnable {
            override fun run() {
                if (!isPolling) return
                runCatching {
                    val (latestPkg, latestCls) =
pollTopPackageViaUsageStats(this@NowLockAccessibilityService)
                    if (latestPkg.isNotEmpty() && !(latestPkg == packageName &&
latestCls.contains("LockScreenActivity"))) {
                        if (AppRepo.evaluateForegroundPackage(latestPkg, packageName)) {
                            AppRepo.launchLockScreenIfNeeded(this@NowLockAccessibilityService,
latestPkg)
                        }
                    }
                }
                mainHandler.postDelayed(this, 200L)
            }
        }
        private val dynamicReceiver = object : BroadcastReceiver() {
            override fun onReceive(context: Context?, intent: Intent?) {
                val ctx = context ?: return
                when (intent?.action) {
                    Intent.ACTION_SCREEN_OFF ->
AppRepo.clearAllUnlockedSessionsOnScreenOff()
                    Intent.ACTION_PACKAGE_ADDED -> {
                        if (!intent.getBooleanExtra(Intent.EXTRA_REPLACING, false)) {
                            val pkg = intent.data?.schemeSpecificPart ?: return
                            AppRepo.cachedLauncherApps = null
            NewAppNotificationHelper.showLockPromptNotification(ctx, pkg)
                        }
                    }
                }
            }
        }
        override fun onServiceConnected() {
```

```
            super.onServiceConnected()
            AppRepo.init(this)
            runCatching {
                registerReceiver(dynamicReceiver, IntentFilter(Intent.ACTION_SCREEN_OFF))
                val pkgFilter = IntentFilter(Intent.ACTION_PACKAGE_ADDED).apply {
addDataScheme("package") }
                registerReceiver(dynamicReceiver, pkgFilter)
            }
            isPolling = true
            mainHandler.post(pollRunnable)
            NowLockMonitorService.startServiceSafely(this)
        }
        override fun onAccessibilityEvent(event: AccessibilityEvent?) {
            if (event == null) return
            if (event.eventType != AccessibilityEvent.TYPE_WINDOW_STATE_CHANGED)
return
            AppRepo.init(this)
            val eventPkg = event.packageName?.toString() ?: return
            val cls = event.className?.toString() ?: ""
            if (eventPkg == packageName) return
            if (cls.startsWith("android.widget.") || cls.startsWith("android.view.")) return
            if (AppRepo.evaluateForegroundPackage(eventPkg, packageName)) {
                AppRepo.launchLockScreenIfNeeded(this, eventPkg)
            }
        }
        override fun onInterrupt() {}
        override fun onDestroy() {
            isPolling = false
            mainHandler.removeCallbacks(pollRunnable)
            runCatching { unregisterReceiver(dynamicReceiver) }
            super.onDestroy()
        }
    }
    // Engine 2: Standalone UsageStats Foreground Service (Ensures locking works on
reboot & even if Accessibility is off!)
    class NowLockMonitorService : Service() {
        private val handler = Handler(Looper.getMainLooper())
        private var isRunning = false
        private val pollRunnable = object : Runnable {
            override fun run() {
                if (!isRunning) return
                runCatching {
                    val (latestPkg, latestCls) =
```

```
pollTopPackageViaUsageStats(this@NowLockMonitorService)
                    if (latestPkg.isNotEmpty() && !(latestPkg == packageName &&
latestCls.contains("LockScreenActivity"))) {
                        if (AppRepo.evaluateForegroundPackage(latestPkg, packageName)) {
                            AppRepo.launchLockScreenIfNeeded(this@NowLockMonitorService,
latestPkg)
                        }
                    }
                }
                handler.postDelayed(this, 200L)
            }
        }
        private val dynamicReceiver = object : BroadcastReceiver() {
            override fun onReceive(context: Context?, intent: Intent?) {
                val ctx = context ?: return
                when (intent?.action) {
                    Intent.ACTION_SCREEN_OFF ->
AppRepo.clearAllUnlockedSessionsOnScreenOff()
                    Intent.ACTION_PACKAGE_ADDED -> {
                        if (!intent.getBooleanExtra(Intent.EXTRA_REPLACING, false)) {
                            val pkg = intent.data?.schemeSpecificPart ?: return
                            AppRepo.cachedLauncherApps = null
                NewAppNotificationHelper.showLockPromptNotification(ctx, pkg)
                        }
                    }
                }
            }
        }
        override fun onCreate() {
            super.onCreate()
            AppRepo.init(this)
            // Cancel any legacy ongoing notification from older versions so the notification bar
stays 100% clean!
            runCatching {
                val nm = getSystemService(Context.NOTIFICATION_SERVICE) as?
NotificationManager
                nm?.cancel(1001)
            }
            runCatching {
                registerReceiver(dynamicReceiver, IntentFilter(Intent.ACTION_SCREEN_OFF))
                val pkgFilter = IntentFilter(Intent.ACTION_PACKAGE_ADDED).apply {
addDataScheme("package") }
                registerReceiver(dynamicReceiver, pkgFilter)
```

```
            }
            isRunning = true
            handler.post(pollRunnable)
        }
        override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
            AppRepo.init(this)
            if (!isRunning) {
                isRunning = true
                handler.post(pollRunnable)
            }
            return START_STICKY
        }
        override fun onDestroy() {
            isRunning = false
            handler.removeCallbacks(pollRunnable)
            runCatching { unregisterReceiver(dynamicReceiver) }
            super.onDestroy()
        }
        override fun onBind(intent: Intent?): IBinder? = null
        companion object {
            fun startServiceSafely(ctx: Context) {
                runCatching {
                    val intent = Intent(ctx, NowLockMonitorService::class.java)
                    ctx.startService(intent)
                }
            }
        }
    }
    // Control Center / Quick Settings Tile Service (Works on Samsung, Xiaomi,
OPPO/OnePlus, Vivo, Huawei, Pixel!)
    class NowLockControlCenterTileService : TileService() {
        private fun refreshTileState() {
            runCatching {
                AppRepo.init(this)
                val tile = qsTile ?: return
                tile.label = "NowLock"
                tile.icon = Icon.createWithResource(this, R.drawable.ic_qs_lock_tile)
                tile.state = Tile.STATE_ACTIVE
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
                    tile.subtitle = "Lock"
                }
                tile.updateTile()
            }
```

```
        }
        override fun onTileAdded() {
            super.onTileAdded()
            refreshTileState()
        }
        override fun onStartListening() {
            super.onStartListening()
            refreshTileState()
        }
        override fun onClick() {
            super.onClick()
            runCatching {
                val launchIntent = Intent(this, MainActivity::class.java).apply {
                    addFlags(
                        Intent.FLAG_ACTIVITY_NEW_TASK or
                        Intent.FLAG_ACTIVITY_CLEAR_TOP or
                        Intent.FLAG_ACTIVITY_SINGLE_TOP
                    )
                }
                if (Build.VERSION.SDK_INT >= 34) {
                    val pi = PendingIntent.getActivity(
                        this,
                        777,
                        launchIntent,
                        PendingIntent.FLAG_UPDATE_CURRENT or
PendingIntent.FLAG_IMMUTABLE
                    )
                    startActivityAndCollapse(pi)
                } else {
                    @Suppress("DEPRECATION")
                    startActivityAndCollapse(launchIntent)
                }
            }
        }
    }
    class BootStartupReceiver : BroadcastReceiver() {
        override fun onReceive(context: Context?, intent: Intent?) {
            val ctx = context ?: return
            AppRepo.init(ctx)
            NowLockMonitorService.startServiceSafely(ctx)
        }
    }
    class UninstallProtectionReceiver : DeviceAdminReceiver() {
```

```
            override fun onDisableRequested(context: Context, intent: Intent): CharSequence {
                runCatching {
                    AppRepo.init(context)
                    AppRepo.appendSecurityLog(context, context.packageName,
"UNINSTALL_ATTEMPT")
                }
                return "NowLock: غیرفعال کردن این گزینه باعث حذف محافظ برنامه می‌شود."
            }
            override fun onDisabled(context: Context, intent: Intent) {
                super.onDisabled(context, intent)
                runCatching {
                    AppRepo.init(context)
                    AppRepo.appendSecurityLog(context, context.packageName,
"UNINSTALL_ATTEMPT")
                }
            }
        }
        // Full-Screen Lock Activity (Clean Top Header: ONLY Small Padlock + RGB "NowLock",
Intruder Selfie on Failed Fingerprint Too)
        class LockScreenActivity : FragmentActivity() {
            companion object {
                @Volatile var isShowing = false
                @Volatile var activeTargetPkg = ""
            }
            override fun onResume() {
                super.onResume()
                isShowing = true
                OverlayLockBridge.dismissShield(this)
            }
            override fun onStop() {
                super.onStop()
                isShowing = false
                OverlayLockBridge.dismissShield(this)
            }
            override fun onCreate(savedInstanceState: Bundle?) {
                super.onCreate(savedInstanceState)
                overridePendingTransition(0, 0)
                AppRepo.init(this)
                if (AppRepo.isRecentsPrivacyEnabled()) {
                    runCatching { window.addFlags(WindowManager.LayoutParams.FLAG_SECURE)
}
                }
                configureNotchAndGlassBars(this, AppRepo.isDarkEffective(this))
```

```
            val targetPkg = intent.getStringExtra("target_pkg") ?: ""
            activeTargetPkg = targetPkg
            isShowing = true
            OverlayLockBridge.dismissShield(this)
            onBackPressedDispatcher.addCallback(this, object : OnBackPressedCallback(true) {
                override fun handleOnBackPressed() {
                    navigateHomeAndClose()
                }
            })
            setContent {
                val lang = AppRepo.getAppLang()
                val s = getStrings(lang)
                val themeMode = AppRepo.getThemeMode()
                val sysDark = isSystemInDarkTheme()
                val isAmoled = themeMode == ThemeMode.AMOLED
                val isDark = when (themeMode) {
                    ThemeMode.SYSTEM -> sysDark
                    ThemeMode.DARK, ThemeMode.AMOLED -> true
                    ThemeMode.LIGHT -> false
                }
                SideEffect {
                    configureNotchAndGlassBars(this@LockScreenActivity, isDark)
                }
                val config = LocalConfiguration.current
                val isCompactScreen = config.screenHeightDp < 700
                val primaryTextColor = Color(0xFFF8FAFC)
                val secondaryTextColor = Color(0xFFD8DEE9)
                val accentBlue = Color(0xFF6EE7B7)
                val effCfg = remember(targetPkg) { AppRepo.getEffectiveConfig(targetPkg) }
                val comboMode = effCfg.comboMode
                val usePattern = (comboMode == LockComboMode.PATTERN_FINGERPRINT
&& AppRepo.isPatternSet()) ||
                    (!AppRepo.isPinSet() && AppRepo.isPatternSet())
                val fpEnabled = effCfg.fpEnabled && canUseFingerprint(this)
                var isShowingFakeCrash by remember { mutableStateOf(false) }
                var enteredPin by remember { mutableStateOf("") }
                var cooldownRemainingSec by remember {
mutableStateOf(AppRepo.getLockoutRemainingSeconds()) }
                val defaultPrompt = if (usePattern) s.lockPromptPattern else s.lockPromptPin
                var statusMessage by remember(lang, usePattern) {
                    mutableStateOf(
                        if (cooldownRemainingSec > 0) s.formatCooldown(cooldownRemainingSec)
else defaultPrompt
```

```
    )
}
var isErrorState by remember { mutableStateOf(cooldownRemainingSec > 0) }
var isUnlockingFade by remember { mutableStateOf(false) }
var telegramBiometricPulse by remember { mutableStateOf(false) }
LaunchedEffect(cooldownRemainingSec) {
    if (cooldownRemainingSec > 0) {
        isErrorState = true
        statusMessage = s.formatCooldown(cooldownRemainingSec)
        delay(1000L)
        val rem = AppRepo.getLockoutRemainingSeconds()
        cooldownRemainingSec = rem
        if (rem <= 0) {
            isErrorState = false
            statusMessage = defaultPrompt
        }
    }
}
val screenAlpha by animateFloatAsState(
    targetValue = if (isUnlockingFade) 0f else 1f,
    animationSpec = tween(durationMillis = 180, easing = FastOutSlowInEasing),
    finishedListener = {
        if (isUnlockingFade) {
            isShowing = false
            OverlayLockBridge.dismissShield(this@LockScreenActivity)
            finish()
            overridePendingTransition(0, android.R.anim.fade_out)
        }
    },
    label = "unlockFade"
)
val scope = rememberCoroutineScope()
fun onSuccessfulUnlock() {
    AppRepo.resetFailedAttempts()
    cooldownRemainingSec = 0
    if (targetPkg.isNotEmpty()) {
        AppRepo.markUnlocked(targetPkg)
    }
    AppRepo.markUnlocked(packageName)
    isShowing = false
    OverlayLockBridge.dismissShield(this@LockScreenActivity)
    finish()
    overridePendingTransition(0, 0)
```

```
                }
                fun onWrongAttempt(methodStr: String = if (usePattern) "Pattern" else "PIN") {
                    isErrorState = true
                    if (effCfg.vibrateOnError) {
                        AppRepo.triggerShortVibration(this@LockScreenActivity, 65L)
                    }
                    val (_, penaltySec) =
AppRepo.registerWrongCredentialAttempt(this@LockScreenActivity, targetPkg, methodStr)
                    if (penaltySec > 0) {
                        cooldownRemainingSec = penaltySec
                        statusMessage = s.formatCooldown(penaltySec)
                    } else {
                        val activeRem = AppRepo.getLockoutRemainingSeconds()
                        if (activeRem > 0) {
                            cooldownRemainingSec = activeRem
                            statusMessage = s.formatCooldown(activeRem)
                        } else {
                            statusMessage = s.lockWrongAttempt
                        }
                    }
                }
                LaunchedEffect(Unit) {
                    if (fpEnabled) {
                        delay(180)
                        telegramBiometricPulse = true
                        triggerFingerprintPrompt(
                            s = s,
                            onSuccess = { onSuccessfulUnlock() },
                            onFailedAttempt = { onWrongAttempt("Fingerprint") },
                            onCancelOrError = { telegramBiometricPulse = false }
                        )
                    }
                }
                FrostedAppColorGlassBackground(
                    targetPkg = targetPkg,
                    isDark = isDark,
                    modifier = Modifier
                        .fillMaxSize()
                        .alpha(screenAlpha)
                ) {
                    AdaptiveLockScreenBody(
                        s = s,
                        usePattern = usePattern,
```

```
                    fpEnabled = fpEnabled,
                    isFingerprintPulsing = telegramBiometricPulse,
                    statusMessage = statusMessage,
                    isErrorState = isErrorState,
                    enteredPinLength = enteredPin.length,
                    primaryTextColor = primaryTextColor,
                    secondaryTextColor = secondaryTextColor,
                    accentBlue = accentBlue,
                    onPatternAttempt = { patternStr, resetPatternCallback ->
                        val rem = AppRepo.getLockoutRemainingSeconds()
                        val isCorrect = AppRepo.verifyPattern(patternStr)
                        val isDecoy = effCfg.decoyCrashEnabled &&
AppRepo.verifyDecoyPattern(patternStr)
                        if (rem > 0) {
                            cooldownRemainingSec = rem
                            if (!isCorrect && !isDecoy) {
                                onWrongAttempt("Pattern")
                            } else {
                                isErrorState = true
                                AppRepo.triggerShortVibration(this@LockScreenActivity, 50L)
                                statusMessage = s.formatCooldown(cooldownRemainingSec)
                            }
                            scope.launch {
                                delay(320)
                                resetPatternCallback()
                            }
                        } else if (isCorrect) {
                            onSuccessfulUnlock()
                        } else if (isDecoy) {
                            AppRepo.appendSecurityLog(this@LockScreenActivity, targetPkg,
"DECOY_USED")
                            resetPatternCallback()
                            isShowingFakeCrash = true
                        } else {
                            onWrongAttempt("Pattern")
                            scope.launch {
                                delay(350)
                                resetPatternCallback()
                                if (cooldownRemainingSec == 0) {
                                    isErrorState = false
                                    statusMessage = defaultPrompt
                                }
                            }
```

```
                    }
                },
                onDigitClick = { digit ->
                    if (enteredPin.length < 4) {
                        if (cooldownRemainingSec <= 0) isErrorState = false
                        enteredPin += digit.toString()
                        if (enteredPin.length == 4) {
                            val rem = AppRepo.getLockoutRemainingSeconds()
                            val isCorrect = AppRepo.verifyPin(enteredPin)
                            val isDecoy = effCfg.decoyCrashEnabled &&
AppRepo.verifyDecoyPin(enteredPin)
                            if (rem > 0) {
                                cooldownRemainingSec = rem
                                if (!isCorrect && !isDecoy) {
                                    onWrongAttempt("PIN")
                                } else {
                                    isErrorState = true
                                    AppRepo.triggerShortVibration(this@LockScreenActivity, 50L)
                                    statusMessage = s.formatCooldown(cooldownRemainingSec)
                                }
                                scope.launch {
                                    delay(260)
                                    enteredPin = ""
                                }
                            } else if (isCorrect) {
                                onSuccessfulUnlock()
                            } else if (isDecoy) {
                                AppRepo.appendSecurityLog(this@LockScreenActivity, targetPkg,
"DECOY_USED")
                                enteredPin = ""
                                isShowingFakeCrash = true
                            } else {
                                onWrongAttempt("PIN")
                                scope.launch {
                                    delay(260)
                                    enteredPin = ""
                                }
                            }
                        }
                    }
                },
                onBackspaceClick = {
                    if (enteredPin.isNotEmpty()) {
```

```
                    isErrorState = false
                    enteredPin = enteredPin.dropLast(1)
                }
            },
            onFingerprintClick = {
                telegramBiometricPulse = true
                triggerFingerprintPrompt(
                    s = s,
                    onSuccess = { onSuccessfulUnlock() },
                    onFailedAttempt = { onWrongAttempt("Fingerprint") },
                    onCancelOrError = { telegramBiometricPulse = false }
                )
            },
            onExitClick = { navigateHomeAndClose() }
        )
        if (isShowingFakeCrash) {
            FakeAndroidSystemCrashOverlay(
                targetPkg = targetPkg,
                s = s,
                onCloseApp = { navigateHomeAndClose() }
            )
        }
    }
  }
}
override fun onNewIntent(intent: Intent) {
  super.onNewIntent(intent)
  val pkg = intent.getStringExtra("target_pkg") ?: ""
  if (pkg.isNotEmpty()) activeTargetPkg = pkg
  OverlayLockBridge.dismissShield(this)
}
override fun onDestroy() {
  isShowing = false
  OverlayLockBridge.dismissShield(this)
  super.onDestroy()
}
private fun navigateHomeAndClose() {
  OverlayLockBridge.dismissShield(this)
  val home = Intent(Intent.ACTION_MAIN).apply {
    addCategory(Intent.CATEGORY_HOME)
    flags = Intent.FLAG_ACTIVITY_NEW_TASK
  }
  runCatching { startActivity(home) }
```

```
            isShowing = false
            finish()
        }
        private fun triggerFingerprintPrompt(
            s: UiStrings,
            onSuccess: () -> Unit,
            onFailedAttempt: () -> Unit,
            onCancelOrError: () -> Unit
        ) {
            val executor = ContextCompat.getMainExecutor(this)
            val prompt = BiometricPrompt(this, executor, object :
BiometricPrompt.AuthenticationCallback() {
                override fun onAuthenticationSucceeded(result:
BiometricPrompt.AuthenticationResult) {
                    super.onAuthenticationSucceeded(result)
                    onSuccess()
                }
                override fun onAuthenticationError(errorCode: Int, errString: CharSequence) {
                    super.onAuthenticationError(errorCode, errString)
                    if (errorCode == BiometricPrompt.ERROR_LOCKOUT || errorCode ==
BiometricPrompt.ERROR_LOCKOUT_PERMANENT) {
                        onFailedAttempt()
                    }
                    onCancelOrError()
                }
                override fun onAuthenticationFailed() {
                    super.onAuthenticationFailed()
                    // Every wrong fingerprint attempt triggers vibration and counts toward Intruder
Selfie!
                    onFailedAttempt()
                }
            })
            val promptInfo = BiometricPrompt.PromptInfo.Builder()
                .setTitle("NowLock")
                .setSubtitle(s.biometricSubtitle)
                .setNegativeButtonText(
                    if (AppRepo.getComboMode() ==
LockComboMode.PATTERN_FINGERPRINT) s.biometricUsePattern else s.biometricUsePin
                )

.setAllowedAuthenticators(BiometricManager.Authenticators.BIOMETRIC_STRONG or
BiometricManager.Authenticators.BIOMETRIC_WEAK)
                .build()
```

```
            runCatching { prompt.authenticate(promptInfo) }.onFailure { onCancelOrError() }
        }
    }
    fun canUseFingerprint(context: Context): Boolean {
        val bm = BiometricManager.from(context)
        return bm.canAuthenticate(BiometricManager.Authenticators.BIOMETRIC_STRONG or
BiometricManager.Authenticators.BIOMETRIC_WEAK) ==
BiometricManager.BIOMETRIC_SUCCESS
    }
    @Composable
    fun FakeAndroidSystemCrashOverlay(
        targetPkg: String,
        s: UiStrings,
        onCloseApp: () -> Unit
    ) {
        val context = LocalContext.current
        val (appName, appIconBmp) = remember(targetPkg) {
            val pkg = if (targetPkg.isBlank()) context.packageName else targetPkg
            runCatching {
                val pm = context.packageManager
                val ai = pm.getApplicationInfo(pkg, 0)
                val label = pm.getApplicationLabel(ai).toString()
                val img = drawableToHdImageBitmap(pm.getApplicationIcon(ai), 96)
                label to img
            }.getOrElse {
                "Application" to
drawableToHdImageBitmap(context.packageManager.defaultActivityIcon, 96)
            }
        }
        val pkgDisplay = if (targetPkg.isBlank()) context.packageName else targetPkg
        Box(
            modifier = Modifier
                .fillMaxSize()
                .background(Color(0xFF0B0D12))
                .pointerInput(Unit) { detectTapGestures {} },
            contentAlignment = Alignment.Center
        ) {
            Surface(
                shape = RoundedCornerShape(26.dp),
                color = Color(0xFF22252E),
                border = BorderStroke(1.dp, Color(0x22FFFFFF)),
                shadowElevation = 18.dp,
                modifier = Modifier
```

```
                .widthIn(max = 380.dp)
                .fillMaxWidth()
                .padding(horizontal = 24.dp)
            ) {
                Column(
                    modifier = Modifier.padding(22.dp),
                    verticalArrangement = Arrangement.spacedBy(14.dp)
                ) {
                    Row(
                        verticalAlignment = Alignment.CenterVertically,
                        horizontalArrangement = Arrangement.spacedBy(12.dp)
                    ) {
                        Image(
                            bitmap = appIconBmp,
                            contentDescription = appName,
                            modifier = Modifier
                                .size(42.dp)
                                .clip(RoundedCornerShape(10.dp))
                        )
                        Text(
                            text = s.decoyCrashHeadline.replace("{app}", appName),
                            color = Color(0xFFF3F4F6),
                            fontSize = 16.sp,
                            fontWeight = FontWeight.Bold,
                            lineHeight = 22.sp
                        )
                    }
                    Surface(
                        shape = RoundedCornerShape(12.dp),
                        color = Color(0xFF15171E),
                        border = BorderStroke(1.dp, Color(0x18FFFFFF)),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        CompositionLocalProvider(LocalLayoutDirection provides
LayoutDirection.Ltr) {
                            Text(
                                text = "java.lang.RuntimeException: Unable to start activity
ComponentInfo{$pkgDisplay/androidx.core.app.CoreComponentFactory}:
android.database.sqlite.SQLiteDatabaseCorruptException: database disk image is malformed
(code 11 SQLITE_CORRUPT)",
                                color = Color(0xFF9CA3AF),
                                fontSize = 10.5.sp,
                                fontFamily = FontFamily.Monospace,
```

```
                        lineHeight = 15.sp,
                        modifier = Modifier.padding(10.dp)
                    )
                }
            }
            HorizontalDivider(color = Color(0x22FFFFFF))
            TextButton(
                onClick = {
                    Toast.makeText(context, "Android System: Process $pkgDisplay
terminated (SIGABRT)", Toast.LENGTH_SHORT).show()
                    onCloseApp()
                },
                modifier = Modifier.fillMaxWidth()
            ) {
                Text(
                    text = s.decoyAppInfoBtn,
                    color = Color(0xFFD1D5DB),
                    fontSize = 14.sp,
                    fontWeight = FontWeight.Medium
                )
            }
            Button(
                onClick = onCloseApp,
                modifier = Modifier.fillMaxWidth().height(44.dp),
                shape = RoundedCornerShape(12.dp),
                colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF374151))
            ) {
                Text(
                    text = s.decoyCloseBtn,
                    color = Color.White,
                    fontSize = 14.sp,
                    fontWeight = FontWeight.Bold
                )
            }
        }
    }
}

@Composable
fun AdaptiveLockScreenBody(
    s: UiStrings,
    usePattern: Boolean,
```

```
        fpEnabled: Boolean,
        isFingerprintPulsing: Boolean,
        statusMessage: String,
        isErrorState: Boolean,
        enteredPinLength: Int,
        primaryTextColor: Color,
        secondaryTextColor: Color,
        accentBlue: Color,
        onPatternAttempt: (String, () -> Unit) -> Unit,
        onDigitClick: (Int) -> Unit,
        onBackspaceClick: () -> Unit,
        onFingerprintClick: () -> Unit,
        onExitClick: () -> Unit
    ) {
        val metrics = rememberAdaptiveScreenMetrics()
        @Composable
        fun BrandBadgeRow() {
            Row(
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.spacedBy(10.dp)
            ) {
                Image(
                    painter = painterResource(id = R.drawable.ic_nowlock_logo),
                    contentDescription = "NowLock",
                    modifier = Modifier
                        .size(if (metrics.isUltraShortHeight) 28.dp else 32.dp)
                        .clip(RoundedCornerShape(9.dp))
                )
                AnimatedRgbBrandText(
                    text = "NowLock",
                    fontSize = if (metrics.isUltraShortHeight) 21.sp else 24.sp
                )
            }
        }
        @Composable
        fun HeaderAndPinDotsSection() {
            Column(
                horizontalAlignment = Alignment.CenterHorizontally,
                verticalArrangement = Arrangement.Center
            ) {
                if (!usePattern) {
                    // PIN Mode: Prompt + 4 PIN Dots at top, and Padlock Icon + "NowLock"
positioned BELOW the PIN dots section!
```

```
                Spacer(Modifier.height(if (metrics.isUltraShortHeight) 6.dp else 18.dp))
                Text(
                    text = statusMessage,
                    color = if (isErrorState) Color(0xFFEF4444) else secondaryTextColor,
                    fontSize = 13.5.sp,
                    textAlign = TextAlign.Center,
                    modifier = Modifier.padding(horizontal = 12.dp)
                )
                Spacer(Modifier.height(if (metrics.isUltraShortHeight) 14.dp else 22.dp))
                CompositionLocalProvider(LocalLayoutDirection provides LayoutDirection.Ltr) {
                    Row(
                        horizontalArrangement = Arrangement.spacedBy(18.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        repeat(4) { idx ->
                            val filled = idx < enteredPinLength
                            val dotScale by animateFloatAsState(
                                targetValue = if (filled) 1.18f else 1.0f,
                                animationSpec = spring(dampingRatio =
Spring.DampingRatioMediumBouncy),
                                label = "dotScale"
                            )
                            Box(
                                modifier = Modifier
                                    .size(14.dp)
                                    .scale(dotScale)
                                    .clip(CircleShape)
                                    .background(
                                        when {
                                            isErrorState -> Color(0xFFEF4444)
                                            filled -> primaryTextColor
                                            else -> Color.Transparent
                                        }
                                    )
                                    .border(
                                        width = 1.5.dp,
                                        color = if (isErrorState) Color(0xFFEF4444) else
primaryTextColor.copy(alpha = 0.55f),
                                        shape = CircleShape
                                    )
                            )
                        }
                    }
```

```
                }
                Spacer(Modifier.height(if (metrics.isUltraShortHeight) 16.dp else 30.dp))
                BrandBadgeRow()
            } else {
                // Pattern Mode: Move Padlock Icon + "NowLock" lower down toward the
vertical middle of the screen!
                Spacer(Modifier.height(if (metrics.isUltraShortHeight) 14.dp else if
(metrics.isCompactHeight) 44.dp else 72.dp))
                BrandBadgeRow()
                Spacer(Modifier.height(if (metrics.isUltraShortHeight) 10.dp else 16.dp))
                Text(
                    text = statusMessage,
                    color = if (isErrorState) Color(0xFFEF4444) else secondaryTextColor,
                    fontSize = 13.5.sp,
                    textAlign = TextAlign.Center,
                    modifier = Modifier.padding(horizontal = 12.dp)
                )
            }
        }
    }
    @Composable
    fun InputPadSection() {
        CompositionLocalProvider(LocalLayoutDirection provides LayoutDirection.Ltr) {
            if (usePattern) {
                Column(
                    horizontalAlignment = Alignment.CenterHorizontally,
                    modifier = Modifier.padding(bottom = if (metrics.isUltraShortHeight) 4.dp else
12.dp)
                ) {
                    InteractivePatternLockView(
                        gridSizeDp = metrics.patternGridSizeDp,
                        dotColor = primaryTextColor,
                        lineColor = accentBlue,
                        errorColor = Color(0xFFEF4444),
                        onPatternCompleted = onPatternAttempt
                    )
                    Spacer(Modifier.height(if (metrics.isUltraShortHeight) 6.dp else 14.dp))
                    Row(
                        modifier = Modifier
                            .widthIn(max = 340.dp)
                            .fillMaxWidth()
                            .padding(horizontal = 24.dp),
                        horizontalArrangement = Arrangement.SpaceBetween,
```

```
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        TextButton(onClick = onExitClick) {
                            Text(
                                text = s.lockExitBtn,
                                color = secondaryTextColor,
                                fontSize = 15.sp
                            )
                        }
                        if (fpEnabled) {
                            TelegramFingerprintButton(
                                sizeDp = if (metrics.isUltraShortHeight) 56 else 74,
                                isActivePulse = isFingerprintPulsing,
                                tintColor = primaryTextColor,
                                accentColor = accentBlue,
                                onClick = onFingerprintClick
                            )
                        }
                    }
                }
            } else {
                Box(modifier = Modifier.widthIn(max = 380.dp)) {
                    SamsungOneUiDialpad(
                        isCompact = metrics.isCompactHeight,
                        customKeySizeDp = metrics.dialKeySizeDp,
                        primaryTextColor = primaryTextColor,
                        secondaryTextColor = secondaryTextColor,
                        accentColor = accentBlue,
                        showFingerprintKey = fpEnabled,
                        isFingerprintPulsing = isFingerprintPulsing,
                        onDigitClick = onDigitClick,
                        onBackspaceClick = onBackspaceClick,
                        onFingerprintClick = onFingerprintClick
                    )
                }
            }
        }
    }
    Box(
        modifier = Modifier
            .fillMaxSize()
.windowInsetsPadding(WindowInsets.systemBars.union(WindowInsets.displayCutout)),
```

```
            contentAlignment = Alignment.Center
        ) {
            if (metrics.useTwoPaneLockLayout) {
                // 2-Pane Side-by-Side Layout for Foldable Inner Screens & Tablet/Phone
Landscape Mode
                Row(
                    modifier = Modifier
                        .widthIn(max = 860.dp)
                        .fillMaxSize()
                        .padding(horizontal = 28.dp, vertical = 12.dp),
                    horizontalArrangement = Arrangement.SpaceEvenly,
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Box(
                        modifier = Modifier.weight(1f),
                        contentAlignment = Alignment.Center
                    ) {
                        HeaderAndPinDotsSection()
                    }
                    Box(
                        modifier = Modifier.weight(1.1f),
                        contentAlignment = Alignment.Center
                    ) {
                        InputPadSection()
                    }
                }
            } else {
                // Responsive Portrait Layout for Phones, Foldable Cover Screens, and Portrait
Tablets
                Column(
                    modifier = Modifier
                        .widthIn(max = 460.dp)
                        .fillMaxSize()
                        .padding(horizontal = 24.dp, vertical = if (metrics.isCompactHeight) 10.dp
else 22.dp),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    if (!usePattern) {
                        // PIN Mode: Prompt + 4 PIN Dots at the top, and Padlock + "NowLock"
positioned in the lower-middle space closer to the Number Pad!
                        Spacer(Modifier.height(if (metrics.isCompactHeight) 18.dp else 36.dp))
                        Text(
                            text = statusMessage,
```

```
                    color = if (isErrorState) Color(0xFFEF4444) else secondaryTextColor,
                    fontSize = 13.5.sp,
                    textAlign = TextAlign.Center,
                    modifier = Modifier.padding(horizontal = 12.dp)
                )
                Spacer(Modifier.height(if (metrics.isUltraShortHeight) 14.dp else 22.dp))
                CompositionLocalProvider(LocalLayoutDirection provides
LayoutDirection.Ltr) {
                    Row(
                        horizontalArrangement = Arrangement.spacedBy(18.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        repeat(4) { idx ->
                            val filled = idx < enteredPinLength
                            val dotScale by animateFloatAsState(
                                targetValue = if (filled) 1.18f else 1.0f,
                                animationSpec = spring(dampingRatio =
Spring.DampingRatioMediumBouncy),
                                label = "dotScale"
                            )
                            Box(
                                modifier = Modifier
                                    .size(14.dp)
                                    .scale(dotScale)
                                    .clip(CircleShape)
                                    .background(
                                        when {
                                            isErrorState -> Color(0xFFEF4444)
                                            filled -> primaryTextColor
                                            else -> Color.Transparent
                                        }
                                    )
                                    .border(
                                        width = 1.5.dp,
                                        color = if (isErrorState) Color(0xFFEF4444) else
primaryTextColor.copy(alpha = 0.55f),
                                        shape = CircleShape
                                    )
                            )
                        }
                    }
                }
                Spacer(Modifier.weight(1.25f))
```

```
                BrandBadgeRow()
                Spacer(Modifier.weight(0.75f))
                InputPadSection()
            } else {
                // Pattern Mode: Centered in the middle space above the Pattern grid
                Spacer(Modifier.weight(1.1f))
                BrandBadgeRow()
                Spacer(Modifier.height(14.dp))
                Text(
                    text = statusMessage,
                    color = if (isErrorState) Color(0xFFEF4444) else secondaryTextColor,
                    fontSize = 13.5.sp,
                    textAlign = TextAlign.Center,
                    modifier = Modifier.padding(horizontal = 12.dp)
                )
                Spacer(Modifier.weight(0.9f))
                InputPadSection()
            }
        }
    }
}

@Composable
fun SamsungOneUiDialpad(
    isCompact: Boolean = false,
    customKeySizeDp: Int? = null,
    primaryTextColor: Color,
    secondaryTextColor: Color,
    accentColor: Color,
    showFingerprintKey: Boolean,
    isFingerprintPulsing: Boolean,
    onDigitClick: (Int) -> Unit,
    onBackspaceClick: () -> Unit,
    onFingerprintClick: () -> Unit
) {
    val dialpadRows = listOf(
        listOf(Triple(1, "➿", ""), Triple(2, "ABC", ""), Triple(3, "DEF", "")),
        listOf(Triple(4, "GHI", ""), Triple(5, "JKL", ""), Triple(6, "MNO", "")),
        listOf(Triple(7, "PQRS", ""), Triple(8, "TUV", ""), Triple(9, "WXYZ", ""))
    )
    val keySizeDp = customKeySizeDp ?: (if (isCompact) 72 else 82)
    Column(
```

```
        modifier = Modifier
            .fillMaxWidth()
            .padding(bottom = if (isCompact) 8.dp else 16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.spacedBy(if (isCompact) 6.dp else 10.dp)
    ) {
        dialpadRows.forEach { row ->
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceEvenly,
                verticalAlignment = Alignment.CenterVertically
            ) {
                row.forEach { (num, subLabel, _) ->
                    SamsungDialKey(
                        sizeDp = keySizeDp,
                        mainText = num.toString(),
                        subText = subLabel,
                        primaryColor = primaryTextColor,
                        secondaryColor = secondaryTextColor,
                        onClick = { onDigitClick(num) }
                    )
                }
            }
        }
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceEvenly,
            verticalAlignment = Alignment.CenterVertically
        ) {
            SamsungIconActionKey(
                sizeDp = keySizeDp - 4,
                symbol = "⌫",
                tintColor = primaryTextColor,
                onClick = onBackspaceClick
            )
            SamsungDialKey(
                sizeDp = keySizeDp,
                mainText = "0",
                subText = "+",
                primaryColor = primaryTextColor,
                secondaryColor = secondaryTextColor,
                onClick = { onDigitClick(0) }
            )
```

```
            if (showFingerprintKey) {
                TelegramFingerprintButton(
                    sizeDp = keySizeDp - 4,
                    isActivePulse = isFingerprintPulsing,
                    tintColor = primaryTextColor,
                    accentColor = accentColor,
                    onClick = onFingerprintClick
                )
            } else {
                Spacer(modifier = Modifier.size((keySizeDp - 4).dp))
            }
        }
    }
}
@Composable
fun SamsungDialKey(
    sizeDp: Int = 82,
    mainText: String,
    subText: String,
    primaryColor: Color,
    secondaryColor: Color,
    onClick: () -> Unit
) {
    val interactionSource = remember { MutableInteractionSource() }
    val isPressed by interactionSource.collectIsPressedAsState()
    val scale by animateFloatAsState(
        targetValue = if (isPressed) 0.86f else 1f,
        animationSpec = spring(stiffness = Spring.StiffnessHigh),
        label = "keyScale"
    )
    val highlightAlpha by animateFloatAsState(
        targetValue = if (isPressed) 0.14f else 0f,
        animationSpec = tween(durationMillis = 90),
        label = "keyHighlight"
    )
    Box(
        contentAlignment = Alignment.Center,
        modifier = Modifier
            .size(sizeDp.dp)
            .scale(scale)
            .clip(CircleShape)
            .background(primaryColor.copy(alpha = highlightAlpha))
            .clickable(
```

```
            interactionSource = interactionSource,
            indication = null,
            onClick = onClick
        )
    ) {
        Column(horizontalAlignment = Alignment.CenterHorizontally) {
            Text(
                text = mainText,
                color = primaryColor,
                fontSize = if (sizeDp < 64) 24.sp else if (sizeDp < 74) 29.sp else 33.sp,
                fontWeight = FontWeight.Normal,
                lineHeight = if (sizeDp < 64) 26.sp else 35.sp
            )
            Text(
                text = subText,
                color = secondaryColor,
                fontSize = 11.sp,
                fontWeight = FontWeight.Medium,
                letterSpacing = 1.2.sp
            )
        }
    }
}
@Composable
fun SamsungIconActionKey(
    sizeDp: Int = 78,
    symbol: String,
    tintColor: Color,
    onClick: () -> Unit
) {
    val interactionSource = remember { MutableInteractionSource() }
    val isPressed by interactionSource.collectIsPressedAsState()
    val scale by animateFloatAsState(
        targetValue = if (isPressed) 0.84f else 1f,
        animationSpec = spring(stiffness = Spring.StiffnessHigh),
        label = "actionScale"
    )
    val highlightAlpha by animateFloatAsState(
        targetValue = if (isPressed) 0.14f else 0f,
        animationSpec = tween(durationMillis = 90),
        label = "actionHighlight"
    )
    Box(
```

```
            contentAlignment = Alignment.Center,
            modifier = Modifier
                .size(sizeDp.dp)
                .scale(scale)
                .clip(CircleShape)
                .background(tintColor.copy(alpha = highlightAlpha))
                .clickable(
                    interactionSource = interactionSource,
                    indication = null,
                    onClick = onClick
                )
        ) {
            Text(
                text = symbol,
                color = tintColor,
                fontSize = 26.sp,
                fontWeight = FontWeight.Normal
            )
        }
    }
    @Composable
    fun TelegramFingerprintButton(
        sizeDp: Int = 78,
        isActivePulse: Boolean,
        tintColor: Color,
        accentColor: Color,
        onClick: () -> Unit
    ) {
        val infiniteTransition = rememberInfiniteTransition(label = "fpPulse")
        val pulseScale by infiniteTransition.animateFloat(
            initialValue = 1.0f,
            targetValue = if (isActivePulse) 1.14f else 1.0f,
            animationSpec = infiniteRepeatable(
                animation = tween(750, easing = FastOutSlowInEasing),
                repeatMode = RepeatMode.Reverse
            ),
            label = "fpPulseScale"
        )
        val interactionSource = remember { MutableInteractionSource() }
        val isPressed by interactionSource.collectIsPressedAsState()
        val pressScale by animateFloatAsState(
            targetValue = if (isPressed) 0.85f else 1f,
            animationSpec = spring(stiffness = Spring.StiffnessHigh),
```

```
        label = "fpPress"
    )
    Box(
        contentAlignment = Alignment.Center,
        modifier = Modifier
            .size(sizeDp.dp)
            .scale(pulseScale * pressScale)
            .clip(CircleShape)
            .background(
                if (isActivePulse) accentColor.copy(alpha = 0.18f)
                else if (isPressed) tintColor.copy(alpha = 0.14f)
                else Color.Transparent
            )
            .clickable(
                interactionSource = interactionSource,
                indication = null,
                onClick = onClick
            )
    ) {
        val iconTint = if (isActivePulse) accentColor else tintColor
        val iconDp = (sizeDp * 0.48f).coerceIn(28f, 42f).dp
        Image(
            painter = painterResource(id = R.drawable.ic_nowlock_fingerprint),
            contentDescription = "Fingerprint Sensor",
            colorFilter = ColorFilter.tint(iconTint),
            modifier = Modifier.size(iconDp)
        )
    }
}
@Composable
fun InteractivePatternLockView(
    gridSizeDp: Int = 276,
    dotColor: Color,
    lineColor: Color,
    errorColor: Color,
    onPatternCompleted: (String, () -> Unit) -> Unit
) {
    val selectedDots = remember { mutableStateListOf<Int>() }
    var currentTouchPos by remember { mutableStateOf<Offset?>(null) }
    var isError by remember { mutableStateOf(false) }
    var boxSize by remember { mutableStateOf(IntSize.Zero) }
    fun nodeCenter(index: Int): Offset {
        val row = (index - 1) / 3
```

```
    val col = (index - 1) % 3
    val cellW = boxSize.width / 3f
    val cellH = boxSize.height / 3f
    return Offset(cellW * (col + 0.5f), cellH * (row + 0.5f))
}
fun checkHitNode(pos: Offset): Int? {
    if (boxSize.width == 0) return null
    val hitRadius = (boxSize.width / 6f) * 0.68f
    for (i in 1..9) {
        val c = nodeCenter(i)
        if (hypot(pos.x - c.x, pos.y - c.y) <= hitRadius) {
            return i
        }
    }
    return null
}
fun finishPatternAttempt() {
    if (selectedDots.size >= 4) {
        val patternStr = selectedDots.joinToString("-")
        onPatternCompleted(patternStr) {
            selectedDots.clear()
            currentTouchPos = null
            isError = false
        }
        isError = true
    } else if (selectedDots.isNotEmpty()) {
        selectedDots.clear()
        currentTouchPos = null
    }
}
Box(
    modifier = Modifier
        .size(gridSizeDp.dp)
        .onSizeChanged { boxSize = it }
        .pointerInput(Unit) {
            detectDragGestures(
                onDragStart = { offset ->
                    if (isError) {
                        selectedDots.clear()
                        isError = false
                    }
                    checkHitNode(offset)?.let { node ->
                        if (!selectedDots.contains(node)) selectedDots.add(node)
```

```
                }
                currentTouchPos = offset
            },
            onDrag = { change, _ ->
                change.consume()
                currentTouchPos = change.position
                checkHitNode(change.position)?.let { node ->
                    if (!selectedDots.contains(node)) {
                        selectedDots.add(node)
                    }
                }
            },
            onDragEnd = {
                currentTouchPos = null
                finishPatternAttempt()
            },
            onDragCancel = {
                currentTouchPos = null
                selectedDots.clear()
            }
        )
    }
    .pointerInput(Unit) {
        detectTapGestures { offset ->
            if (isError) {
                selectedDots.clear()
                isError = false
            }
            checkHitNode(offset)?.let { node ->
                if (!selectedDots.contains(node)) {
                    selectedDots.add(node)
                    if (selectedDots.size >= 4) {
                        finishPatternAttempt()
                    }
                }
            }
        }
    }
) {
    ComposeCanvas(modifier = Modifier.fillMaxSize()) {
        val activeLineColor = if (isError) errorColor else lineColor
        for (i in 0 until selectedDots.size - 1) {
            val start = nodeCenter(selectedDots[i])
```

```
            val end = nodeCenter(selectedDots[i + 1])
            drawLine(
                color = activeLineColor,
                start = start,
                end = end,
                strokeWidth = 4.5.dp.toPx(),
                cap = StrokeCap.Round
            )
        }
        if (selectedDots.isNotEmpty() && currentTouchPos != null && !isError) {
            drawLine(
                color = activeLineColor.copy(alpha = 0.65f),
                start = nodeCenter(selectedDots.last()),
                end = currentTouchPos!!,
                strokeWidth = 3.5.dp.toPx(),
                cap = StrokeCap.Round
            )
        }
        for (i in 1..9) {
            val center = nodeCenter(i)
            val isSelected = selectedDots.contains(i)
            if (isSelected) {
                drawCircle(
                    color = activeLineColor.copy(alpha = 0.24f),
                    radius = 22.dp.toPx(),
                    center = center
                )
                drawCircle(
                    color = activeLineColor,
                    radius = 9.dp.toPx(),
                    center = center
                )
            } else {
                drawCircle(
                    color = dotColor.copy(alpha = 0.55f),
                    radius = 6.5.dp.toPx(),
                    center = center
                )
            }
        }
    }
  }
}
```

```
data class RealAppItem(
    val label: String,
    val packageName: String,
    val iconImage: ImageBitmap,
    val isLocked: Boolean,
    val isSelfApp: Boolean = false
)
fun drawableToHdImageBitmap(drawable: Drawable, targetPx: Int): ImageBitmap {
    val bitmap = Bitmap.createBitmap(targetPx, targetPx, Bitmap.Config.ARGB_8888)
    val canvas = Canvas(bitmap)
    drawable.setFilterBitmap(true)
    drawable.setBounds(0, 0, targetPx, targetPx)
    drawable.draw(canvas)
    return bitmap.asImageBitmap()
}
// Animated Top Security Status Emblem for Smart Security Report Tab:
// - Critical Red Danger State (#EF4444) when permissions or lock service are disabled (or initially not granted)
// - Warning Yellow Exclamation State (#F59E0B) when failed password attempts or password warnings exist
// - Smooth natural color & geometry morphing between states!
@Composable
fun AnimatedSecurityReportEmblem(
    isDangerRed: Boolean,
    isWarningYellow: Boolean,
    titleText: String,
    subtitleText: String,
    cardColor: Color,
    borderColor: Color,
    txtColor: Color,
    subColor: Color
) {
    val targetAccent = when {
        isDangerRed -> Color(0xFFEF4444)
        isWarningYellow -> Color(0xFFF59E0B)
        else -> Color(0xFF10B981)
    }
    val animatedColor by animateColorAsState(
        targetValue = targetAccent,
        animationSpec = tween(durationMillis = 700, easing = FastOutSlowInEasing),
        label = "emblemColorMorph"
    )
    // 1f = Red Danger Triangle, 0f = Yellow Warning Circle / Shield
```

```
val shapeMorph by animateFloatAsState(
    targetValue = if (isDangerRed) 1f else 0f,
    animationSpec = tween(durationMillis = 700, easing = FastOutSlowInEasing),
    label = "emblemShapeMorph"
)
val infiniteTransition = rememberInfiniteTransition(label = "securityEmblemPulse")
val pulseProgress by infiniteTransition.animateFloat(
    initialValue = 0f,
    targetValue = 1f,
    animationSpec = infiniteRepeatable(
        animation = tween(durationMillis = 1900, easing = FastOutSlowInEasing),
        repeatMode = RepeatMode.Restart
    ),
    label = "pulseWave"
)
val emblemScale by infiniteTransition.animateFloat(
    initialValue = 0.94f,
    targetValue = 1.06f,
    animationSpec = infiniteRepeatable(
        animation = tween(durationMillis = 1100, easing = FastOutSlowInEasing),
        repeatMode = RepeatMode.Reverse
    ),
    label = "emblemBreath"
)
Surface(
    shape = RoundedCornerShape(22.dp),
    color = cardColor,
    border = BorderStroke(1.5.dp, animatedColor.copy(alpha = 0.55f)),
    modifier = Modifier.fillMaxWidth()
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 18.dp, vertical = 18.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Box(
            contentAlignment = Alignment.Center,
            modifier = Modifier.size(96.dp)
        ) {
            ComposeCanvas(
                modifier = Modifier
                    .fillMaxSize()
```

```
                .scale(emblemScale)
            ) {
                val cx = size.width / 2f
                val cy = size.height / 2f
                val baseR = size.minDimension * 0.32f
                if (shapeMorph > 0.45f) {
                    // Expanding Triangular Pulse Waves matching the Red Danger Triangle
geometry!
                    val waveScale1 = 1f + pulseProgress * 0.62f
                    val rW1 = baseR * waveScale1
                    val waveTriPath1 = Path().apply {
                        moveTo(cx, cy - rW1 * 0.95f)
                        lineTo(cx + rW1 * 0.92f, cy + rW1 * 0.78f)
                        lineTo(cx - rW1 * 0.92f, cy + rW1 * 0.78f)
                        close()
                    }
                    drawPath(
                        path = waveTriPath1,
                        color = animatedColor.copy(alpha = (1f - pulseProgress) * 0.42f),
                        style = androidx.compose.ui.graphics.drawscope.Stroke(
                            width = 2.8.dp.toPx(),
                            cap = StrokeCap.Round
                        )
                    )
                    val rW2 = baseR * 1.22f
                    val waveTriPath2 = Path().apply {
                        moveTo(cx, cy - rW2 * 0.95f)
                        lineTo(cx + rW2 * 0.92f, cy + rW2 * 0.78f)
                        lineTo(cx - rW2 * 0.92f, cy + rW2 * 0.78f)
                        close()
                    }
                    drawPath(
                        path = waveTriPath2,
                        color = animatedColor.copy(alpha = 0.16f)
                    )
                } else {
                    // Expanding Circular Pulse Waves for Yellow Warning / Emerald Safe
Circle!
                    drawCircle(
                        color = animatedColor.copy(alpha = (1f - pulseProgress) * 0.34f),
                        radius = baseR * (1f + pulseProgress * 0.65f),
                        center = Offset(cx, cy)
                    )
```

```
                drawCircle(
                    color = animatedColor.copy(alpha = 0.18f),
                    radius = baseR * 1.18f,
                    center = Offset(cx, cy)
                )
            }
            if (shapeMorph > 0.45f) {
                // Dynamic Red Danger Hazard Triangle with rounded stroke + inner fill
                val triPath = Path().apply {
                    moveTo(cx, cy - baseR * 0.95f)
                    lineTo(cx + baseR * 0.92f, cy + baseR * 0.78f)
                    lineTo(cx - baseR * 0.92f, cy + baseR * 0.78f)
                    close()
                }
                drawPath(
                    path = triPath,
                    color = animatedColor.copy(alpha = 0.25f)
                )
                drawPath(
                    path = triPath,
                    color = animatedColor,
                    style = androidx.compose.ui.graphics.drawscope.Stroke(
                        width = 3.5.dp.toPx(),
                        cap = StrokeCap.Round
                    )
                )
            } else {
                // Dynamic Yellow Warning Exclamation Circle / Safe Badge
                drawCircle(
                    color = animatedColor.copy(alpha = 0.25f),
                    radius = baseR * 0.92f,
                    center = Offset(cx, cy)
                )
                drawCircle(
                    color = animatedColor,
                    radius = baseR * 0.92f,
                    center = Offset(cx, cy),
                    style = androidx.compose.ui.graphics.drawscope.Stroke(width =
3.5.dp.toPx())
                )
            }
            // Central Exclamation Mark (!) for both Danger (Red) and Warning (Yellow),
or Checkmark when Safe
```

```
            if (isDangerRed || isWarningYellow) {
                drawLine(
                    color = animatedColor,
                    start = Offset(cx, cy - baseR * 0.36f),
                    end = Offset(cx, cy + baseR * 0.16f),
                    strokeWidth = 4.2.dp.toPx(),
                    cap = StrokeCap.Round
                )
                drawCircle(
                    color = animatedColor,
                    radius = 2.6.dp.toPx(),
                    center = Offset(cx, cy + baseR * 0.44f)
                )
            } else {
                drawLine(
                    color = animatedColor,
                    start = Offset(cx - baseR * 0.32f, cy + baseR * 0.04f),
                    end = Offset(cx - baseR * 0.06f, cy + baseR * 0.30f),
                    strokeWidth = 3.8.dp.toPx(),
                    cap = StrokeCap.Round
                )
                drawLine(
                    color = animatedColor,
                    start = Offset(cx - baseR * 0.06f, cy + baseR * 0.30f),
                    end = Offset(cx + baseR * 0.38f, cy - baseR * 0.24f),
                    strokeWidth = 3.8.dp.toPx(),
                    cap = StrokeCap.Round
                )
            }
        }
    }
    Spacer(Modifier.height(10.dp))
    Text(
        text = titleText,
        color = animatedColor,
        fontSize = 14.5.sp,
        fontWeight = FontWeight.Bold,
        textAlign = TextAlign.Center
    )
    Spacer(Modifier.height(4.dp))
    Text(
        text = subtitleText,
        color = subColor,
```

```
                    fontSize = 11.5.sp,
                    textAlign = TextAlign.Center,
                    lineHeight = 17.sp
                )
            }
        }
    }

    @OptIn(ExperimentalFoundationApi::class)
    class MainActivity : FragmentActivity() {
        private var hasOverlayState = mutableStateOf(false)
        private var hasAccState = mutableStateOf(false)
        private var hasUsageState = mutableStateOf(false)
        private var hasCameraState = mutableStateOf(false)
        private var isPausedInRecentsState = mutableStateOf(false)
        private var isSelfLockedState = mutableStateOf(false)
        override fun onPause() {
            super.onPause()
            isPausedInRecentsState.value = true
        }
        override fun onStop() {
            super.onStop()
            if (!isChangingConfigurations && !AppRepo.isSelfSettingsGraceActive()) {
                if (AppRepo.getRelockDelayMinutes() <= 0) {
                    AppRepo.unlockedSession.remove(packageName)
                    AppRepo.unlockTimestamps.remove(packageName)
                }
            }
        }
        override fun onResume() {
            super.onResume()
            configureNotchAndGlassBars(this, AppRepo.isDarkEffective(this))
            isPausedInRecentsState.value = false
            hasOverlayState.value = Settings.canDrawOverlays(this)
            hasAccState.value = isAccessibilityServiceEnabled(this)
            hasUsageState.value = isUsageStatsPermissionGranted(this)
            hasCameraState.value = ContextCompat.checkSelfPermission(this,
Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED
            applyRecentsSecureFlag(AppRepo.isRecentsPrivacyEnabled())
            NowLockMonitorService.startServiceSafely(this)
            if (AppRepo.isAnyPasswordSet() &&
                AppRepo.getLockedPackages().contains(packageName) &&
                !AppRepo.isSelfSettingsGraceActive() &&
```

```
                !AppRepo.isWithinGracePeriod(packageName)
            ) {
                isSelfLockedState.value = true
            } else {
                isSelfLockedState.value = false
            }
        }
        override fun onWindowFocusChanged(hasFocus: Boolean) {
            super.onWindowFocusChanged(hasFocus)
            if (hasFocus) {
                hasOverlayState.value = Settings.canDrawOverlays(this)
                hasAccState.value = isAccessibilityServiceEnabled(this)
                hasUsageState.value = isUsageStatsPermissionGranted(this)
                hasCameraState.value = ContextCompat.checkSelfPermission(this,
Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED
            }
        }
        private fun requestCameraPermissionSafely() {
            AppRepo.markLeavingForSettings(this)
            runCatching {
                val perms = mutableListOf(Manifest.permission.CAMERA)
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
                    perms.add(Manifest.permission.POST_NOTIFICATIONS)
                }
                ActivityCompat.requestPermissions(this, perms.toTypedArray(), 1001)
            }.onFailure {
                openDirectAppDetailsSettings(this)
            }
        }
        fun triggerSelfFingerprintPrompt(
            s: UiStrings,
            onSuccess: () -> Unit,
            onFailedAttempt: () -> Unit,
            onCancelOrError: () -> Unit
        ) {
            val executor = ContextCompat.getMainExecutor(this)
            val prompt = BiometricPrompt(this, executor, object :
BiometricPrompt.AuthenticationCallback() {
                override fun onAuthenticationSucceeded(result:
BiometricPrompt.AuthenticationResult) {
                    super.onAuthenticationSucceeded(result)
                    onSuccess()
                }
```

```
                override fun onAuthenticationError(errorCode: Int, errString: CharSequence) {
                    super.onAuthenticationError(errorCode, errString)
                    if (errorCode == BiometricPrompt.ERROR_LOCKOUT || errorCode ==
BiometricPrompt.ERROR_LOCKOUT_PERMANENT) {
                        onFailedAttempt()
                    }
                    onCancelOrError()
                }
                override fun onAuthenticationFailed() {
                    super.onAuthenticationFailed()
                    onFailedAttempt()
                }
            })
            val promptInfo = BiometricPrompt.PromptInfo.Builder()
                .setTitle("NowLock")
                .setSubtitle(s.biometricSubtitle)
                .setNegativeButtonText(
                    if (AppRepo.getComboMode() ==
LockComboMode.PATTERN_FINGERPRINT) s.biometricUsePattern else s.biometricUsePin
                )

.setAllowedAuthenticators(BiometricManager.Authenticators.BIOMETRIC_STRONG or
BiometricManager.Authenticators.BIOMETRIC_WEAK)
                .build()
            runCatching { prompt.authenticate(promptInfo) }.onFailure { onCancelOrError() }
        }
        private fun applyRecentsSecureFlag(enabled: Boolean) {
            runCatching {
                if (enabled) {
                    window.addFlags(WindowManager.LayoutParams.FLAG_SECURE)
                } else {
                    window.clearFlags(WindowManager.LayoutParams.FLAG_SECURE)
                }
            }
        }
        override fun onCreate(savedInstanceState: Bundle?) {
            super.onCreate(savedInstanceState)
            AppRepo.init(this)
            applyRecentsSecureFlag(AppRepo.isRecentsPrivacyEnabled())
            configureNotchAndGlassBars(this, AppRepo.isDarkEffective(this))
            hasOverlayState.value = Settings.canDrawOverlays(this)
            hasAccState.value = isAccessibilityServiceEnabled(this)
            hasUsageState.value = isUsageStatsPermissionGranted(this)
```

```
            hasCameraState.value = ContextCompat.checkSelfPermission(this,
Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED
            NowLockMonitorService.startServiceSafely(this)
            setContent {
                var themeMode by remember { mutableStateOf(AppRepo.getThemeMode()) }
                var appLang by remember { mutableStateOf(AppRepo.getAppLang()) }
                val s = getStrings(appLang)
                val isFa = appLang.isRtl
                val pagerState = rememberPagerState(initialPage = 0, pageCount = {
NowLockNavTab.entries.size })
                val selectedTab = NowLockNavTab.entries[pagerState.currentPage]
                var comboMode by remember { mutableStateOf(AppRepo.getComboMode()) }
                var fpEnabled by remember { mutableStateOf(AppRepo.isFingerprintEnabled()) }
                var iconMode by remember { mutableStateOf(AppRepo.getIconMode()) }
                var relockMinutes by remember {
mutableStateOf(AppRepo.getRelockDelayMinutes()) }
                var recentsPrivacy by remember {
mutableStateOf(AppRepo.isRecentsPrivacyEnabled()) }
                val sysDark = isSystemInDarkTheme()
                val isAmoled = themeMode == ThemeMode.AMOLED
                val isDark = when (themeMode) {
                    ThemeMode.SYSTEM -> sysDark
                    ThemeMode.DARK, ThemeMode.AMOLED -> true
                    ThemeMode.LIGHT -> false
                }
                SideEffect {
                    configureNotchAndGlassBars(this@MainActivity, isDark)
                }
                val hasOverlayPerm by hasOverlayState
                val hasAccPerm by hasAccState
                val hasUsagePerm by hasUsageState
                val hasCamPerm by hasCameraState
                val isPausedInRecents by isPausedInRecentsState
                LaunchedEffect(hasCamPerm) {
                    while (!hasCamPerm) {
                        delay(450)
                        hasCameraState.value = ContextCompat.checkSelfPermission(
                            this@MainActivity,
                            Manifest.permission.CAMERA
                        ) == PackageManager.PERMISSION_GRANTED
                    }
                }
                val isEngineReady = hasAccPerm && hasUsagePerm && hasOverlayPerm &&
```

```
hasCamPerm
                val topHeaderBg = if (isAmoled) Color(0xFF000000) else if (isDark)
Color(0xCC1C1B5A) else Color(0xE6F0F2FA)
                val card = if (isAmoled) Color(0xFF0B0B10) else if (isDark) Color(0xCC282664)
else Color(0xF0FFFFFF)
                val border = if (isAmoled) Color(0x26FFFFFF) else if (isDark) Color(0x18FFFFFF)
else Color(0x125D56F5)
                val blue = if (isDark) Color(0xFF6C63FF) else Color(0xFF5D56F5)
                val txt = if (isDark) Color(0xFFF5F6FA) else Color(0xFF1F2232)
                val sub = if (isDark) Color(0xFFA7A4D6) else Color(0xFF757992)
                val lockedPkgsNow = remember { AppRepo.getLockedPackages() }
                var appList by remember {
                    mutableStateOf(
                        AppRepo.cachedLauncherApps?.map { item ->
                            item.copy(isLocked = lockedPkgsNow.contains(item.packageName))
                        } ?: emptyList()
                    )
                }
                LaunchedEffect(Unit) {
                    if (appList.isEmpty()) {
                        val loaded = withContext(Dispatchers.Default) {
                            loadRealLauncherApps()
                        }
                        AppRepo.cachedLauncherApps = loaded
                        appList = loaded
                    }
                }
                var filterMode by remember { mutableStateOf(AppFilterMode.ALL) }
                var searchQuery by remember { mutableStateOf("") }
                var isLauncherHidden by remember {
mutableStateOf(AppRepo.isLauncherHidden()) }
                var lockOnScreenOff by remember {
mutableStateOf(AppRepo.isLockOnScreenOffEnabled()) }
                var decoyEnabled by remember { mutableStateOf(AppRepo.isDecoyEnabled()) }
                var showPinDialog by remember { mutableStateOf(false) }
                var showPatternDialog by remember { mutableStateOf(false) }
                var showDecoyPinDialog by remember { mutableStateOf(false) }
                var showDecoyPatternDialog by remember { mutableStateOf(false) }
                var showSecQSetupDialog by remember { mutableStateOf(false) }
                var showSecQRecoverDialog by remember { mutableStateOf(false) }
                var secQuestionSetState by remember {
mutableStateOf(AppRepo.isSecurityQuestionSet()) }
                var decoyPinSetState by remember { mutableStateOf(AppRepo.isDecoyPinSet())
```

```
}
                var decoyPatternSetState by remember {
mutableStateOf(AppRepo.isDecoyPatternSet()) }
                var showPrivacyFullModal by remember { mutableStateOf(false) }
                var pendingLockPkg by remember { mutableStateOf<String?>(null) }
                var expandedPerAppPkg by remember { mutableStateOf<String?>(null) }
                var passwordSetState by remember {
mutableStateOf(AppRepo.isAnyPasswordSet()) }
                var securityLogs by remember { mutableStateOf(AppRepo.getSecurityLogs()) }
                var showUpdateSubtitle by remember { mutableStateOf(false) }
                // Refresh security logs whenever switching to the Security Report tab, and
auto-clear from storage once read upon leaving the tab!
                var prevAcc by remember { mutableStateOf(hasAccPerm) }
                var prevUsage by remember { mutableStateOf(hasUsagePerm) }
                var prevOverlay by remember { mutableStateOf(hasOverlayPerm) }
                var prevCam by remember { mutableStateOf(hasCamPerm) }
                LaunchedEffect(hasAccPerm, hasUsagePerm, hasOverlayPerm, hasCamPerm) {
                    if (prevAcc && !hasAccPerm) AppRepo.appendSecurityLog(this@MainActivity,
packageName, "PERM_REVOKED_ACC")
                    if (prevUsage && !hasUsagePerm)
AppRepo.appendSecurityLog(this@MainActivity, packageName, "PERM_REVOKED_USAGE")
                    if (prevOverlay && !hasOverlayPerm)
AppRepo.appendSecurityLog(this@MainActivity, packageName,
"PERM_REVOKED_OVERLAY")
                    if (prevCam && !hasCamPerm)
AppRepo.appendSecurityLog(this@MainActivity, packageName, "PERM_REVOKED_CAM")
                    prevAcc = hasAccPerm
                    prevUsage = hasUsagePerm
                    prevOverlay = hasOverlayPerm
                    prevCam = hasCamPerm
                    securityLogs = AppRepo.getSecurityLogs()
                }
                LaunchedEffect(pagerState.currentPage) {
                    if (NowLockNavTab.entries[pagerState.currentPage] ==
NowLockNavTab.REPORT) {
                        securityLogs = AppRepo.getSecurityLogs()
                    }
                }
                val scope = rememberCoroutineScope()
                val navBarBottomInset =
WindowInsets.navigationBars.asPaddingValues().calculateBottomPadding()
                CompositionLocalProvider(LocalLayoutDirection provides if (appLang.isRtl)
LayoutDirection.Rtl else LayoutDirection.Ltr) {
```

```
                MaterialTheme {
                    HappAmbientBackground(isDark = isDark) {
                        Scaffold(
                            containerColor = Color.Transparent,
                            contentWindowInsets = WindowInsets(0, 0, 0, 0),
                            topBar = {
                                Column(
                                    modifier = Modifier
                                        .fillMaxWidth()
                                        .background(topHeaderBg)

.windowInsetsPadding(WindowInsets.statusBars.union(WindowInsets.displayCutout))
                                        .padding(top = 8.dp, bottom = 2.dp),
                                    horizontalAlignment = Alignment.CenterHorizontally
                                ) {
                                    Row(
                                        modifier = Modifier
                                            .fillMaxWidth()
                                            .padding(horizontal = 20.dp, vertical = 6.dp),
                                        horizontalArrangement = Arrangement.SpaceBetween,
                                        verticalAlignment = Alignment.CenterVertically
                                    ) {
                                        Row(verticalAlignment = Alignment.CenterVertically) {
                                            Image(
                                                painter = painterResource(
                                                    id = if (iconMode == AppIconMode.CALCULATOR)
R.drawable.ic_nowlock_calculator else R.drawable.ic_nowlock_logo
                                                ),
                                                contentDescription = "NowLock Icon",
                                                modifier = Modifier
                                                    .size(34.dp)
                                                    .clip(RoundedCornerShape(9.dp))
                                            )
                                            Spacer(Modifier.width(10.dp))
                                            AnimatedRgbBrandText(
                                                text = "NowLock",
                                                fontSize = 23.sp
                                            )
                                        }
                                        Surface(
                                            shape = RoundedCornerShape(50),
                                            color = if (passwordSetState && isEngineReady)
Color(0x2210B981) else Color(0x22F43F5E)
```

```
                ) {
                    Text(
                        text = if (!passwordSetState) s.statusNoPassword
                        else if (!isEngineReady) s.statusNeedsPerm
                        else s.statusActive,
                        color = if (passwordSetState && isEngineReady)
Color(0xFF10B981) else Color(0xFFF43F5E),
                        fontSize = 11.sp,
                        fontWeight = FontWeight.SemiBold,
                        modifier = Modifier.padding(horizontal = 10.dp, vertical =
5.dp)
                    )
                }
            }
            TabRow(
                selectedTabIndex = pagerState.currentPage,
                containerColor = Color.Transparent,
                contentColor = blue,
                divider = { HorizontalDivider(color = border) }
            ) {
                NowLockNavTab.entries.forEach { tab ->
                    val isSelected = pagerState.currentPage == tab.ordinal
                    Tab(
                        selected = isSelected,
                        onClick = {
                            scope.launch {
                                pagerState.animateScrollToPage(
                                    page = tab.ordinal,
                                    animationSpec = tween(durationMillis = 320, easing
= FastOutSlowInEasing)
                                )
                            }
                        },
                        text = {
                            Text(
                                tab.getTitle(s),
                                fontSize = 13.sp,
                                fontWeight = if (isSelected) FontWeight.Bold else
FontWeight.Normal,
                                color = if (isSelected) blue else sub
                            )
                        }
                    )
```

```
                    }
                }
            }
        }
    ) { innerPadding ->
        val screenMetrics = rememberAdaptiveScreenMetrics()
        Box(
            contentAlignment = Alignment.TopCenter,
            modifier = Modifier
                .fillMaxSize()
                .padding(top = innerPadding.calculateTopPadding())
        ) {
            HorizontalPager(
                state = pagerState,
                key = { it },
                modifier = Modifier.fillMaxSize()
            ) { pageIdx ->
                Box(
                    contentAlignment = Alignment.TopCenter,
                    modifier = Modifier
                        .fillMaxSize()
                        .graphicsLayer {
                            val pageOffset = ((pagerState.currentPage - pageIdx) +
pagerState.currentPageOffsetFraction).let { abs(it) }.coerceIn(0f, 1f)
                            alpha = 1f - (pageOffset * 0.35f)
                            val scale = 1f - (pageOffset * 0.025f)
                            scaleX = scale
                            scaleY = scale
                        }
                ) {
                    when (NowLockNavTab.entries[pageIdx]) {
                    NowLockNavTab.APPS -> {
                        Column(modifier = Modifier.widthIn(max =
screenMetrics.contentMaxWidthDp.dp).fillMaxSize().padding(horizontal = 16.dp)) {
                            OutlinedTextField(
                                value = searchQuery,
                                onValueChange = { searchQuery = it },
                                placeholder = {
                                    Text(s.searchPlaceholder, fontSize = 13.sp, color = sub)
                                },
                                singleLine = true,
                                shape = RoundedCornerShape(14.dp),
                                modifier = Modifier.fillMaxWidth().padding(top = 10.dp)
```

```
                )
                Row(
                    modifier = Modifier.fillMaxWidth().padding(vertical = 8.dp),
                    horizontalArrangement = Arrangement.spacedBy(8.dp)
                ) {
                    AppFilterMode.entries.forEach { filter ->
                        val sel = filterMode == filter
                        FilterChip(
                            selected = sel,
                            onClick = { filterMode = filter },
                            label = {
                                val count = when (filter) {
                                    AppFilterMode.ALL -> appList.size
                                    AppFilterMode.LOCKED -> appList.count {
it.isLocked }
                                    AppFilterMode.UNLOCKED -> appList.count {
!it.isLocked }
                                }
                                Text("${filter.getTitle(s)} ($count)", fontSize = 12.sp)
                            },
                            colors = FilterChipDefaults.filterChipColors(
                                selectedContainerColor = blue,
                                selectedLabelColor = Color.White
                            )
                        )
                    }
                }
                val filteredApps = appList.filter { app ->
                    val matchesFilter = when (filterMode) {
                        AppFilterMode.ALL -> true
                        AppFilterMode.LOCKED -> app.isLocked
                        AppFilterMode.UNLOCKED -> !app.isLocked
                    }
                    val matchesQuery = searchQuery.isBlank() ||
                        app.label.contains(searchQuery, ignoreCase = true) ||
                        app.packageName.contains(searchQuery, ignoreCase =
true)
                    matchesFilter && matchesQuery
                }
                LazyColumn(
                    verticalArrangement = Arrangement.spacedBy(8.dp),
                    contentPadding = PaddingValues(bottom =
navBarBottomInset + 28.dp)
```

```
                                    ) {
                                        items(items = filteredApps, key = { it.packageName },
contentType = { "app_row" }) { app ->
                                            val isExpanded = expandedPerAppPkg ==
app.packageName
                                            var perAppCfg by remember(app.packageName,
isExpanded) {

mutableStateOf(AppRepo.getPerAppConfig(app.packageName))
                                            }
                                            Surface(
                                                shape = RoundedCornerShape(18.dp),
                                                color = card,
                                                border = BorderStroke(
                                                    width = if (app.isSelfApp) 1.5.dp else 1.dp,
                                                    color = if (app.isSelfApp) blue.copy(alpha = 0.45f)
else border
                                                ),
                                                modifier = Modifier.fillMaxWidth()
                                            ) {
                                                Column(modifier = Modifier.fillMaxWidth()) {
                                                    Row(
                                                        modifier = Modifier
                                                            .fillMaxWidth()
                                                            .padding(horizontal = 14.dp, vertical = 11.dp),
                                                        horizontalArrangement =
Arrangement.SpaceBetween,
                                                        verticalAlignment = Alignment.CenterVertically
                                                    ) {
                                                        Row(
                                                            verticalAlignment = Alignment.CenterVertically,
                                                            modifier = Modifier.weight(1f)
                                                        ) {
                                                            Image(
                                                                bitmap = app.iconImage,
                                                                contentDescription = app.label,
                                                                modifier = Modifier
                                                                    .size(44.dp)
                                                                    .clip(RoundedCornerShape(11.dp))
                                                            )
                                                            Spacer(Modifier.width(12.dp))
                                                            Column {
                                                                Row(verticalAlignment =
```

```
Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(6.dp)) {
                                                    Text(app.label, color = txt, fontSize =
14.sp, fontWeight = FontWeight.SemiBold)
                                                    if (app.isSelfApp) {
                                                        Surface(
                                                            shape =
RoundedCornerShape(6.dp),
                                                            color = blue.copy(alpha = 0.15f)
                                                        ) {
                                                            Text(
                                                                s.selfAppBadge,
                                                                color = blue,
                                                                fontSize = 10.sp,
                                                                fontWeight = FontWeight.Bold,
                                                                modifier =
Modifier.padding(horizontal = 6.dp, vertical = 2.dp)
                                                            )
                                                        }
                                                    }
                                                }
                                                    Text(app.packageName, color = sub,
fontSize = 11.sp)
                                                }
                                            }
                                            Row(
                                                verticalAlignment = Alignment.CenterVertically,
                                                horizontalArrangement =
Arrangement.spacedBy(6.dp)
                                            ) {
                                                // Expandable Per-App Settings Arrow Button
right next to the Lock Switch!
                                                Surface(
                                                    shape = RoundedCornerShape(8.dp),
                                                    color = if (isExpanded ||
perAppCfg.useCustom) blue.copy(alpha = 0.16f) else Color.Transparent,
                                                    border = BorderStroke(1.dp, if
(perAppCfg.useCustom) blue else border),
                                                    modifier = Modifier
                                                        .size(32.dp)
                                                        .clickable {
                                                            expandedPerAppPkg = if (isExpanded)
null else app.packageName
                                                        }
```

```
                                                ) {
                                                    Box(contentAlignment = Alignment.Center) {
                                                        Text(
                                                            text = if (isExpanded) "▲" else "▼",
                                                            color = if (perAppCfg.useCustom) blue
else sub,
                                                            fontSize = 11.sp,
                                                            fontWeight = FontWeight.Bold
                                                        )
                                                    }
                                                }
                                                Switch(
                                                    checked = app.isLocked,
                                                    onCheckedChange = { lock ->
                                                        if (!AppRepo.isAnyPasswordSet() &&
lock) {
                                                            pendingLockPkg = app.packageName
                                                            if (comboMode ==
LockComboMode.PATTERN_FINGERPRINT) {
                                                                showPatternDialog = true
                                                            } else {
                                                                showPinDialog = true
                                                            }
                                                        } else {

AppRepo.toggleAppLock(app.packageName, lock)
                                                            if (app.isSelfApp && lock) {

AppRepo.markUnlocked(packageName)
                                                            }
                                                            appList = appList.map {
                                                                if (it.packageName ==
app.packageName) it.copy(isLocked = lock) else it
                                                            }

NowLockMonitorService.startServiceSafely(this@MainActivity)
                                                        }
                                                    }
                                                )
                                            }
                                        }
                                        // Expandable Per-App Settings Drawer (Default:
Follow Global Settings, or Custom Override for every setting!)
```

```
                                    AnimatedVisibility(visible = isExpanded) {
                                        Column(
                                            modifier = Modifier
                                                .fillMaxWidth()
                                                .padding(start = 14.dp, end = 14.dp, bottom
= 12.dp),
                                            verticalArrangement =
Arrangement.spacedBy(8.dp)
                                        ) {
                                            HorizontalDivider(color = border)
                                            Row(
                                                modifier = Modifier.fillMaxWidth(),
                                                horizontalArrangement =
Arrangement.spacedBy(6.dp)
                                            ) {
                                                listOf(
                                                    false to s.perAppFollowGlobal,
                                                    true to s.perAppCustom
                                                ).forEach { (isCustomMode, modeTitle) ->
                                                    val sel = perAppCfg.useCustom ==
isCustomMode
                                                    Surface(
                                                        shape = RoundedCornerShape(10.dp),
                                                        color = if (sel) blue else
blue.copy(alpha = 0.08f),
                                                        modifier = Modifier
                                                            .weight(1f)
                                                            .height(34.dp)
                                                            .clickable {
                                                                perAppCfg =
perAppCfg.copy(useCustom = isCustomMode)

AppRepo.savePerAppConfig(app.packageName, perAppCfg)
                                                            }
                                                    ) {
                                                        Box(contentAlignment =
Alignment.Center, modifier = Modifier.padding(horizontal = 4.dp)) {
                                                            Text(
                                                                text = modeTitle,
                                                                color = if (sel) Color.White else txt,
                                                                fontSize = 11.sp,
                                                                fontWeight =
FontWeight.SemiBold,
```

```
                                                    textAlign = TextAlign.Center
                                                )
                                            }
                                        }
                                    }
                                }
                                if (perAppCfg.useCustom) {
                                    // 1. Lock Method (PIN vs Pattern) for this
app
                                    Row(horizontalArrangement =
Arrangement.spacedBy(6.dp), modifier = Modifier.fillMaxWidth()) {
                                        LockComboMode.entries.forEach { m ->
                                            val sel = perAppCfg.comboMode == m
                                            val lbl = if (m ==
LockComboMode.PIN_FINGERPRINT) s.modePin else s.modePattern
                                            Surface(
                                                shape =
RoundedCornerShape(9.dp),
                                                color = if (sel) blue else
blue.copy(alpha = 0.08f),
                                                modifier = Modifier
                                                    .weight(1f)
                                                    .height(32.dp)
                                                    .clickable {
                                                        perAppCfg =
perAppCfg.copy(comboMode = m)

AppRepo.savePerAppConfig(app.packageName, perAppCfg)
                                                    }
                                            ) {
                                                Box(contentAlignment =
Alignment.Center) {
                                                    Text(lbl, color = if (sel) Color.White
else txt, fontSize = 10.5.sp)
                                                }
                                            }
                                        }
                                    }
                                    // 2. Relock Delay for this app
                                    Row(horizontalArrangement =
Arrangement.spacedBy(4.dp), modifier = Modifier.fillMaxWidth()) {
                                        RelockDelayOption.entries.forEach { opt
->
```

```
                                                    val sel = perAppCfg.relockMinutes ==
opt.minutes
                                                    Surface(
                                                        shape =
RoundedCornerShape(8.dp),
                                                        color = if (sel) blue else
blue.copy(alpha = 0.08f),
                                                        modifier = Modifier
                                                            .weight(1f)
                                                            .height(30.dp)
                                                            .clickable {
                                                                perAppCfg =
perAppCfg.copy(relockMinutes = opt.minutes)

AppRepo.savePerAppConfig(app.packageName, perAppCfg)
                                                            }
                                                    ) {
                                                        Box(contentAlignment =
Alignment.Center) {
                                                            Text(opt.getLabel(s), color = if
(sel) Color.White else txt, fontSize = 10.sp)
                                                        }
                                                    }
                                                }
                                            }
                                            // 3. Individual Toggles for this app
(Fingerprint, Lock on Screen Off, Intruder Selfie, Vibration, Fake Crash Decoy)
                                            SettingsToggleRow(
                                                title = s.fpTitle,
                                                subtitle = s.fpSub,
                                                checked = perAppCfg.fpEnabled,
                                                txtColor = txt,
                                                subColor = sub,
                                                onCheckedChange = {
                                                    perAppCfg =
perAppCfg.copy(fpEnabled = it)

AppRepo.savePerAppConfig(app.packageName, perAppCfg)
                                                }
                                            )
                                            if (perAppCfg.relockMinutes > 0) {
                                                SettingsToggleRow(
                                                    title = s.lockOnScreenOffTitle,
```

```
                                                    subtitle = s.lockOnScreenOffSub,
                                                    checked =
perAppCfg.lockOnScreenOff,
                                                    txtColor = txt,
                                                    subColor = sub,
                                                    onCheckedChange = {
                                                        perAppCfg =
perAppCfg.copy(lockOnScreenOff = it)

AppRepo.savePerAppConfig(app.packageName, perAppCfg)
                                                    }
                                                )
                                            }
                                            SettingsToggleRow(
                                                title = s.intruderTitle,
                                                subtitle = s.intruderSubPrefix,
                                                checked = perAppCfg.intruderSelfie,
                                                txtColor = txt,
                                                subColor = sub,
                                                onCheckedChange = {
                                                    perAppCfg =
perAppCfg.copy(intruderSelfie = it)

AppRepo.savePerAppConfig(app.packageName, perAppCfg)
                                                }
                                            )
                                            SettingsToggleRow(
                                                title = s.vibrateTitle,
                                                subtitle = s.vibrateSub,
                                                checked = perAppCfg.vibrateOnError,
                                                txtColor = txt,
                                                subColor = sub,
                                                onCheckedChange = {
                                                    perAppCfg =
perAppCfg.copy(vibrateOnError = it)

AppRepo.savePerAppConfig(app.packageName, perAppCfg)
                                                }
                                            )
                                            SettingsToggleRow(
                                                title = s.decoyToggleTitle,
                                                subtitle = s.decoySectionTitle,
                                                checked =
```

```
perAppCfg.decoyCrashEnabled,
                                                txtColor = txt,
                                                subColor = sub,
                                                onCheckedChange = {
                                                    perAppCfg =
perAppCfg.copy(decoyCrashEnabled = it)

AppRepo.savePerAppConfig(app.packageName, perAppCfg)
                                                }
                                            )
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
                NowLockNavTab.SETTINGS -> {
                    LazyColumn(
                        modifier = Modifier.widthIn(max =
screenMetrics.contentMaxWidthDp.dp).fillMaxSize().padding(horizontal = 16.dp),
                        verticalArrangement = Arrangement.spacedBy(12.dp),
                        contentPadding = PaddingValues(top = 14.dp, bottom =
navBarBottomInset + 36.dp)
                    ) {
                        item {
                            MinimalSectionCard(
                                title = s.secAuthTitle,
                                cardColor = card,
                                borderColor = border,
                                txtColor = txt
                            ) {
                                Row(horizontalArrangement =
Arrangement.spacedBy(8.dp), modifier = Modifier.fillMaxWidth()) {
                                    LockComboMode.entries.forEach { itemMode ->
                                        val sel = comboMode == itemMode
                                        val modeLabel = if (itemMode ==
LockComboMode.PIN_FINGERPRINT) s.modePin else s.modePattern
                                        Surface(
                                            shape = RoundedCornerShape(12.dp),
                                            color = if (sel) blue else blue.copy(alpha = 0.08f),
```

```
                                                modifier = Modifier
                                                    .weight(1f)
                                                    .height(40.dp)
                                                    .clickable {
                                                        comboMode = itemMode
                                                        AppRepo.setComboMode(itemMode)
                                                        if (itemMode ==
LockComboMode.PIN_FINGERPRINT && !AppRepo.isPinSet()) {
                                                            showPinDialog = true
                                                        } else if (itemMode ==
LockComboMode.PATTERN_FINGERPRINT && !AppRepo.isPatternSet()) {
                                                            showPatternDialog = true
                                                        }
                                                    }
                                            ) {
                                                Box(contentAlignment = Alignment.Center,
modifier = Modifier.padding(horizontal = 6.dp)) {
                                                    Text(
                                                        modeLabel,
                                                        color = if (sel) Color.White else txt,
                                                        fontSize = 12.sp,
                                                        fontWeight = FontWeight.SemiBold,
                                                        textAlign = TextAlign.Center
                                                    )
                                                }
                                            }
                                        }
                                    }
                                    Spacer(Modifier.height(10.dp))
                                    Row(horizontalArrangement =
Arrangement.spacedBy(8.dp), modifier = Modifier.fillMaxWidth()) {
                                        Button(
                                            onClick = { showPinDialog = true },
                                            modifier = Modifier.weight(1f).height(42.dp),
                                            shape = RoundedCornerShape(12.dp),
                                            colors =
ButtonDefaults.buttonColors(containerColor = blue)
                                        ) {
                                            Text(
                                                if (passwordSetState && AppRepo.isPinSet())
s.btnChangePin else s.btnSetPin,
                                                color = Color.White,
                                                fontSize = 12.sp
```

```
                )
            }
            OutlinedButton(
                onClick = { showPatternDialog = true },
                modifier = Modifier.weight(1f).height(42.dp),
                shape = RoundedCornerShape(12.dp)
            ) {
                Text(
                    if (passwordSetState && AppRepo.isPatternSet())
s.btnChangePattern else s.btnSetPattern,
                    color = txt,
                    fontSize = 12.sp
                )
            }
        }
        Spacer(Modifier.height(8.dp))
        Surface(
            shape = RoundedCornerShape(12.dp),
            color = blue.copy(alpha = 0.10f),
            border = BorderStroke(1.dp, blue.copy(alpha = 0.35f)),
            modifier = Modifier
                .fillMaxWidth()
                .height(40.dp)
                .clickable {
                    if (secQuestionSetState) {
                        showSecQRecoverDialog = true
                    } else {
                        showSecQSetupDialog = true
                    }
                }
        ) {
            Box(contentAlignment = Alignment.Center, modifier =
Modifier.padding(horizontal = 10.dp)) {
                Text(
                    text = if (secQuestionSetState)
s.secQRecoverBtn else s.secQSetupBtn,
                    color = blue,
                    fontSize = 11.5.sp,
                    fontWeight = FontWeight.SemiBold,
                    textAlign = TextAlign.Center,
                    maxLines = 1
                )
            }
```

```
                                        }
                                        HorizontalDivider(color = border, modifier =
Modifier.padding(vertical = 8.dp))
                                        SettingsToggleRow(
                                            title = s.fpTitle,
                                            subtitle = s.fpSub,
                                            checked = fpEnabled,
                                            txtColor = txt,
                                            subColor = sub,
                                            onCheckedChange = { enabled ->
                                                if (enabled &&
!canUseFingerprint(this@MainActivity)) {
                                                    Toast.makeText(this@MainActivity,
s.fpEnrollToast, Toast.LENGTH_LONG).show()
                                                    runCatching {
startActivity(Intent(Settings.ACTION_SECURITY_SETTINGS)) }
                                                } else {
                                                    fpEnabled = enabled
                                                    AppRepo.setFingerprintEnabled(enabled)
                                                }
                                            }
                                        )
                                        HorizontalDivider(color = border, modifier =
Modifier.padding(vertical = 8.dp))
                                        // Decoy / Fake Crash Password Section
                                        SettingsToggleRow(
                                            title = s.decoyToggleTitle,
                                            subtitle = s.decoyToggleSub,
                                            checked = decoyEnabled,
                                            txtColor = txt,
                                            subColor = sub,
                                            onCheckedChange = { enabled ->
                                                decoyEnabled = enabled
                                                AppRepo.setDecoyEnabled(enabled)
                                                if (enabled && !AppRepo.isDecoyPinSet() &&
!AppRepo.isDecoyPatternSet()) {
                                                    showDecoyPinDialog = true
                                                }
                                            }
                                        )
                                        Row(horizontalArrangement =
Arrangement.spacedBy(8.dp), modifier = Modifier.fillMaxWidth().padding(top = 6.dp)) {
                                            OutlinedButton(
```

```
                                    onClick = { showDecoyPinDialog = true },
                                    modifier = Modifier.weight(1f).height(38.dp),
                                    shape = RoundedCornerShape(10.dp)
                                ) {
                                    Text(
                                        text = if (decoyPinSetState)
s.btnChangeDecoyPin else s.btnSetDecoyPin,
                                        color = txt,
                                        fontSize = 11.sp
                                    )
                                }
                                OutlinedButton(
                                    onClick = { showDecoyPatternDialog = true },
                                    modifier = Modifier.weight(1f).height(38.dp),
                                    shape = RoundedCornerShape(10.dp)
                                ) {
                                    Text(
                                        text = if (decoyPatternSetState)
s.btnChangeDecoyPattern else s.btnSetDecoyPattern,
                                        color = txt,
                                        fontSize = 11.sp
                                    )
                                }
                            }
                        }
                    }
                    item {
                        MinimalSectionCard(
                            title = s.secDelayTitle,
                            cardColor = card,
                            borderColor = border,
                            txtColor = txt
                        ) {
                            Text(s.secDelaySub, color = sub, fontSize = 11.sp)
                            Spacer(Modifier.height(10.dp))
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement =
Arrangement.spacedBy(6.dp)
                            ) {
                                RelockDelayOption.entries.forEach { opt ->
                                    val sel = relockMinutes == opt.minutes
                                    Surface(
```

```
                                        shape = RoundedCornerShape(11.dp),
                                        color = if (sel) blue else blue.copy(alpha = 0.08f),
                                        modifier = Modifier
                                            .weight(1f)
                                            .height(38.dp)
                                            .clickable {
                                                relockMinutes = opt.minutes

AppRepo.setRelockDelayMinutes(opt.minutes)
                                            }
                                    ) {
                                        Box(contentAlignment = Alignment.Center) {
                                            Text(
                                                opt.getLabel(s),
                                                color = if (sel) Color.White else txt,
                                                fontSize = 11.sp,
                                                fontWeight = if (sel) FontWeight.Bold else
FontWeight.Medium
                                            )
                                        }
                                    }
                                }
                            }
                            // Whenever a non-immediate delay (2, 4, 6, 10 min) is
chosen, show the "Lock Immediately on Screen Off" option!
                            AnimatedVisibility(visible = relockMinutes > 0) {
                                Column {
                                    HorizontalDivider(color = border, modifier =
Modifier.padding(vertical = 8.dp))
                                    SettingsToggleRow(
                                        title = s.lockOnScreenOffTitle,
                                        subtitle = s.lockOnScreenOffSub,
                                        checked = lockOnScreenOff,
                                        txtColor = txt,
                                        subColor = sub,
                                        onCheckedChange = { enabled ->
                                            lockOnScreenOff = enabled

AppRepo.setLockOnScreenOffEnabled(enabled)
                                        }
                                    )
                                }
                            }
```

```
                }
            }
            item {
                MinimalSectionCard(
                    title = s.secIconTitle,
                    cardColor = card,
                    borderColor = border,
                    txtColor = txt
                ) {
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement =
Arrangement.spacedBy(10.dp),
                        verticalAlignment = Alignment.CenterVertically
                    ) {
                        listOf(
                            Triple(AppIconMode.PADLOCK,
R.drawable.ic_nowlock_logo, s.iconPadlock),
                            Triple(AppIconMode.CALCULATOR,
R.drawable.ic_nowlock_calculator, s.iconCalculator)
                        ).forEach { (modeItem, resId, titleStr) ->
                            val sel = iconMode == modeItem
                            Surface(
                                shape = RoundedCornerShape(14.dp),
                                color = if (sel) blue.copy(alpha = 0.16f) else
Color.Transparent,
                                border = BorderStroke(if (sel) 1.5.dp else 1.dp, if
(sel) blue else border),
                                modifier = Modifier
                                    .weight(1f)
                                    .height(58.dp)
                                    .clickable {
                                        iconMode = modeItem
                                        AppRepo.setIconMode(this@MainActivity,
modeItem)
                                        Toast.makeText(this@MainActivity,
"${s.iconChangedToast}: $titleStr", Toast.LENGTH_SHORT).show()
                                    }
                            ) {
                                Row(
                                    modifier =
Modifier.fillMaxSize().padding(horizontal = 10.dp),
                                    horizontalArrangement = Arrangement.Center,
```

```
                                        verticalAlignment = Alignment.CenterVertically
                                    ) {
                                        Image(
                                            painter = painterResource(id = resId),
                                            contentDescription = titleStr,
                                            modifier =
Modifier.size(32.dp).clip(RoundedCornerShape(8.dp))
                                        )
                                        Spacer(Modifier.width(8.dp))
                                        Text(
                                            titleStr,
                                            color = txt,
                                            fontSize = 12.sp,
                                            fontWeight = FontWeight.SemiBold
                                        )
                                    }
                                }
                            }
                        }
                    }
                    item {
                        MinimalSectionCard(
                            title = s.secSecurityTitle,
                            cardColor = card,
                            borderColor = border,
                            txtColor = txt
                        ) {
                            var vibrateState by remember {
mutableStateOf(AppRepo.isVibrationEnabled()) }
                            var intruderState by remember {
mutableStateOf(AppRepo.isIntruderSelfieEnabled()) }
                            var uninstallState by remember {
mutableStateOf(AppRepo.isUninstallProtectionEnabled()) }
                            SettingsToggleRow(
                                title = s.recentsTitle,
                                subtitle = s.recentsSub,
                                checked = recentsPrivacy,
                                txtColor = txt,
                                subColor = sub,
                                onCheckedChange = { enabled ->
                                    recentsPrivacy = enabled
                                    AppRepo.setRecentsPrivacyEnabled(enabled)
```

```
                applyRecentsSecureFlag(enabled)
            }
        )
        HorizontalDivider(color = border)
        SettingsToggleRow(
            title = s.intruderTitle,
            subtitle = s.intruderSubPrefix,
            checked = intruderState,
            txtColor = txt,
            subColor = sub,
            onCheckedChange = { enabled ->
                intruderState = enabled
                AppRepo.setIntruderSelfieEnabled(enabled)
                if (enabled && !hasCamPerm) {
                    requestCameraPermissionSafely()
                }
            }
        )
        HorizontalDivider(color = border)
        SettingsToggleRow(
            title = s.vibrateTitle,
            subtitle = s.vibrateSub,
            checked = vibrateState,
            txtColor = txt,
            subColor = sub,
            onCheckedChange = { vibrateState = it;
AppRepo.setVibrationEnabled(it) }
        )
        HorizontalDivider(color = border)
        SettingsToggleRow(
            title = s.uninstallTitle,
            subtitle = s.uninstallSub,
            checked = uninstallState,
            txtColor = txt,
            subColor = sub,
            onCheckedChange = { enabled ->
                uninstallState = enabled
                AppRepo.setUninstallProtectionEnabled(enabled)
                val adminComponent =
ComponentName(this@MainActivity, UninstallProtectionReceiver::class.java)
                if (enabled) {

AppRepo.markLeavingForSettings(this@MainActivity)
```

```
                                                val intent =
Intent(DevicePolicyManager.ACTION_ADD_DEVICE_ADMIN).apply {

putExtra(DevicePolicyManager.EXTRA_DEVICE_ADMIN, adminComponent)

putExtra(DevicePolicyManager.EXTRA_ADD_EXPLANATION, s.uninstallSub)
                                                }
                                                runCatching { startActivity(intent) }
                                            } else {
                                                runCatching {
                                                    val dpm =
getSystemService(Context.DEVICE_POLICY_SERVICE) as? DevicePolicyManager
                                                    dpm?.removeActiveAdmin(adminComponent)
                                                }
                                            }
                                        }
                                    )
                                }
                            }
                            item {
                                MinimalSectionCard(
                                    title = s.secThemeLangTitle,
                                    cardColor = card,
                                    borderColor = border,
                                    txtColor = txt
                                ) {
                                    Column(verticalArrangement =
Arrangement.spacedBy(6.dp), modifier = Modifier.fillMaxWidth()) {
                                        listOf(
                                            listOf(ThemeMode.SYSTEM to s.themeSystem,
ThemeMode.LIGHT to s.themeLight),
                                            listOf(ThemeMode.DARK to s.themeDark,
ThemeMode.AMOLED to s.themeAmoled)
                                        ).forEach { rowItems ->
                                            Row(horizontalArrangement =
Arrangement.spacedBy(6.dp), modifier = Modifier.fillMaxWidth()) {
                                                rowItems.forEach { (m, label) ->
                                                    val sel = themeMode == m
                                                    Surface(
                                                        shape = RoundedCornerShape(10.dp),
                                                        color = if (sel) blue else blue.copy(alpha =
0.08f),

                                                        modifier = Modifier
```

```
                                                .weight(1f)
                                                .height(38.dp)
                                                .clickable { themeMode = m;
AppRepo.setThemeMode(m) }
                                        ) {
                                            Box(
                                                contentAlignment = Alignment.Center,
                                                modifier = Modifier.padding(horizontal =
8.dp)
                                            ) {
                                                Text(
                                                    text = label,
                                                    color = if (sel) Color.White else txt,
                                                    fontSize = 11.5.sp,
                                                    fontWeight = if (sel) FontWeight.Bold
else FontWeight.Medium,
                                                    maxLines = 1,
                                                    textAlign = TextAlign.Center
                                                )
                                            }
                                        }
                                    }
                                }
                            }
                            HorizontalDivider(color = border, modifier =
Modifier.padding(vertical = 10.dp))
                            Text(s.selectLangLabel, color = txt, fontWeight =
FontWeight.Bold, fontSize = 13.sp)
                            Spacer(Modifier.height(8.dp))
                            Row(horizontalArrangement =
Arrangement.spacedBy(6.dp), modifier = Modifier.fillMaxWidth()) {
                                listOf(AppLang.FA, AppLang.EN,
AppLang.AZ).forEach { langItem ->
                                    val sel = appLang == langItem
                                    Surface(
                                        shape = RoundedCornerShape(10.dp),
                                        color = if (sel) blue else blue.copy(alpha = 0.08f),
                                        modifier = Modifier
                                            .weight(1f)
                                            .height(36.dp)
                                            .clickable { appLang = langItem;
AppRepo.setAppLang(langItem) }
```

```
                                        ) {
                                            Box(contentAlignment = Alignment.Center,
modifier = Modifier.padding(horizontal = 4.dp)) {
                                                Text(langItem.nativeLabel, color = if (sel)
Color.White else txt, fontSize = 11.sp, textAlign = TextAlign.Center)
                                            }
                                        }
                                    }
                                }
                                Spacer(Modifier.height(6.dp))
                                Row(horizontalArrangement =
Arrangement.spacedBy(6.dp), modifier = Modifier.fillMaxWidth()) {
                                    listOf(AppLang.LOR, AppLang.MAZ).forEach {
langItem ->
                                        val sel = appLang == langItem
                                        Surface(
                                            shape = RoundedCornerShape(10.dp),
                                            color = if (sel) blue else blue.copy(alpha = 0.08f),
                                            modifier = Modifier
                                                .weight(1f)
                                                .height(36.dp)
                                                .clickable { appLang = langItem;
AppRepo.setAppLang(langItem) }
                                        ) {
                                            Box(contentAlignment = Alignment.Center,
modifier = Modifier.padding(horizontal = 4.dp)) {
                                                Text(langItem.nativeLabel, color = if (sel)
Color.White else txt, fontSize = 11.sp, textAlign = TextAlign.Center)
                                            }
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
                NowLockNavTab.REPORT -> {
                    val isDangerRed = !isEngineReady || !passwordSetState
                    val isWarningYellow = !isDangerRed &&
(securityLogs.isNotEmpty() || AppRepo.getConsecutiveFailedAttempts() > 0)
                    val topEmblemTitle = when {
                        isDangerRed -> s.repDangerTitle
                        isWarningYellow -> s.repWarningTitle
```

```
                                    else -> s.repSafeTitle
                                }
                                val topEmblemSub = when {
                                    isDangerRed -> s.repDangerSub
                                    isWarningYellow -> s.repWarningSub
                                    else -> s.repSafeSub
                                }
                                LazyColumn(
                                    modifier = Modifier.widthIn(max =
screenMetrics.contentMaxWidthDp.dp).fillMaxSize().padding(horizontal = 16.dp),
                                    verticalArrangement = Arrangement.spacedBy(12.dp),
                                    contentPadding = PaddingValues(top = 14.dp, bottom =
navBarBottomInset + 36.dp)
                                ) {
                                    // 1. Animated Top Security Emblem (Red Triangle Pulse <->
Yellow Circle Exclamation <-> Emerald Safe)
                                    item {
                                        AnimatedSecurityReportEmblem(
                                            isDangerRed = isDangerRed,
                                            isWarningYellow = isWarningYellow,
                                            titleText = topEmblemTitle,
                                            subtitleText = topEmblemSub,
                                            cardColor = card,
                                            borderColor = border,
                                            txtColor = txt,
                                            subColor = sub
                                        )
                                    }
                                    // 2. Open 4-Permission Active Guard Section
                                    item {
                                        val grantedCount = listOf(hasAccPerm, hasUsagePerm,
hasOverlayPerm, hasCamPerm).count { it }
                                        MinimalSectionCard(
                                            title = "${if (isEngineReady) s.statusActive else
s.statusNeedsPerm} ($grantedCount/4)",
                                            cardColor = card,
                                            borderColor = if (isEngineReady) Color(0x5510B981)
else Color(0x88EF4444),
                                            txtColor = txt
                                        ) {
                                            Text(
                                                text = s.permBannerTitle,
                                                color = sub,
```

```
                fontSize = 11.5.sp,
                lineHeight = 17.sp
            )
            Spacer(Modifier.height(10.dp))
            if (!hasAccPerm || !hasUsagePerm) {
                Button(
                    onClick = {
                        Toast.makeText(this@MainActivity,
s.permRestrictedToast, Toast.LENGTH_LONG).show()
                        openDirectAppDetailsSettings(this@MainActivity)
                    },
                    modifier = Modifier.fillMaxWidth(),
                    colors =
ButtonDefaults.buttonColors(containerColor = Color(0xFFD97706)),
                    shape = RoundedCornerShape(11.dp)
                ) {
                    Text(
                        text = s.permRestrictedBtn,
                        fontSize = 11.sp,
                        fontWeight = FontWeight.Bold,
                        color = Color.White,
                        textAlign = TextAlign.Center
                    )
                }
                Spacer(Modifier.height(8.dp))
            }
            val permRows: List<Triple<String, Boolean, () -> Unit>>
= listOf(
                Triple(s.perm1Acc, hasAccPerm, {
openDirectAccessibilitySettings(this@MainActivity) }),
                Triple(s.perm2Usage, hasUsagePerm, {
openDirectUsageAccessSettings(this@MainActivity) }),
                Triple(s.perm3Overlay, hasOverlayPerm, {
openDirectOverlaySettings(this@MainActivity) }),
                Triple(s.perm4Camera, hasCamPerm, {
requestCameraPermissionSafely() })
            )
            Row(horizontalArrangement =
Arrangement.spacedBy(8.dp), modifier = Modifier.fillMaxWidth()) {
                permRows.take(2).forEach { item ->
                    Button(
                        onClick = item.third,
                        modifier = Modifier.weight(1f).height(42.dp),
```

```
                                                colors = ButtonDefaults.buttonColors(
                                                    containerColor = if (item.second)
Color(0xFF10B981) else Color(0xFFEF4444)
                                                ),
                                                shape = RoundedCornerShape(11.dp)
                                            ) {
                                                Text(
                                                    text = if (item.second) "✓ ${item.first}" else "✕
${item.first}",
                                                    fontSize = 11.sp,
                                                    fontWeight = FontWeight.Bold,
                                                    color = Color.White
                                                )
                                            }
                                        }
                                    }
                                    Spacer(Modifier.height(8.dp))
                                    Row(horizontalArrangement =
Arrangement.spacedBy(8.dp), modifier = Modifier.fillMaxWidth()) {
                                        permRows.drop(2).forEach { item ->
                                            Button(
                                                onClick = item.third,
                                                modifier = Modifier.weight(1f).height(42.dp),
                                                colors = ButtonDefaults.buttonColors(
                                                    containerColor = if (item.second)
Color(0xFF10B981) else Color(0xFFEF4444)
                                                ),
                                                shape = RoundedCornerShape(11.dp)
                                            ) {
                                                Text(
                                                    text = if (item.second) "✓ ${item.first}" else "✕
${item.first}",
                                                    fontSize = 11.sp,
                                                    fontWeight = FontWeight.Bold,
                                                    color = Color.White
                                                )
                                            }
                                        }
                                    }
                                }
                                // 3. Unified Smart Security Report (Wrong PIN/Pattern/FP,
Decoy Triggered, Permission Revoked, Uninstall Attempt; Persists until Clear button is tapped &
```

```
dynamically translates!)
                                    item {
                                        MinimalSectionCard(
                                            title = s.repLogsTitle,
                                            cardColor = card,
                                            borderColor = border,
                                            txtColor = txt
                                        ) {
                                            Row(
                                                modifier = Modifier.fillMaxWidth(),
                                                horizontalArrangement =
Arrangement.SpaceBetween,
                                                verticalAlignment = Alignment.CenterVertically
                                            ) {
                                                Text(s.repFailedCountLabel, color = txt, fontSize =
12.5.sp, fontWeight = FontWeight.SemiBold)
                                                Surface(
                                                    shape = RoundedCornerShape(50),
                                                    color = if (securityLogs.isEmpty())
Color(0x2210B981) else Color(0x22F59E0B)
                                                ) {
                                                    Text(
                                                        text = "${securityLogs.size}",
                                                        color = if (securityLogs.isEmpty())
Color(0xFF10B981) else Color(0xFFF59E0B),
                                                        fontSize = 12.sp,
                                                        fontWeight = FontWeight.Bold,
                                                        modifier = Modifier.padding(horizontal = 12.dp,
vertical = 4.dp)
                                                    )
                                                }
                                            }
                                            Spacer(Modifier.height(10.dp))
                                            if (securityLogs.isEmpty()) {
                                                Text(
                                                    text = s.repNoLogsMsg,
                                                    color = sub,
                                                    fontSize = 12.sp,
                                                    modifier = Modifier.padding(vertical = 6.dp)
                                                )
                                            } else {
                                                Column(verticalArrangement =
Arrangement.spacedBy(8.dp)) {
```

```
                securityLogs.forEach { entry ->
                    val isCriticalEvent =
entry.method.startsWith("PERM_") || entry.method == "UNINSTALL_ATTEMPT" || entry.method
== "LOCKOUT_TRIGGERED"
                    val badgeColor = if (isCriticalEvent)
Color(0xFFEF4444) else Color(0xFFF59E0B)
                    Surface(
                        shape = RoundedCornerShape(12.dp),
                        color = badgeColor.copy(alpha = 0.11f),
                        border = BorderStroke(1.dp,
badgeColor.copy(alpha = 0.35f)),
                        modifier = Modifier.fillMaxWidth()
                    ) {
                        Column(modifier = Modifier.padding(10.dp),
verticalArrangement = Arrangement.spacedBy(4.dp)) {
                            Text(
                                text =
s.translateSecurityEvent(entry.method),
                                color = badgeColor,
                                fontSize = 12.sp,
                                fontWeight = FontWeight.Bold
                            )
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement =
Arrangement.SpaceBetween
                            ) {
                                Text(
                                    text = "${s.repTargetAppLabel}
${entry.targetAppLabel}",
                                    color = txt,
                                    fontSize = 11.5.sp,
                                    fontWeight = FontWeight.SemiBold
                                )
                                Text(
                                    text = entry.timestamp,
                                    color = sub,
                                    fontSize = 10.5.sp
                                )
                            }
                        }
                    }
                }
```

```
                                        Spacer(Modifier.height(4.dp))
                                        Text(
                                            text = s.repAutoClearNote,
                                            color = sub,
                                            fontSize = 10.5.sp,
                                            lineHeight = 15.sp
                                        )
                                        Button(
                                            onClick = {
                                                AppRepo.clearSecurityLogs()
                                                AppRepo.resetFailedAttempts()
                                                securityLogs = emptyList()
                                            },
                                            modifier = Modifier.fillMaxWidth(),
                                            shape = RoundedCornerShape(11.dp),
                                            colors =
ButtonDefaults.buttonColors(containerColor = blue)
                                        ) {
                                            Text(s.repClearNowBtn, color = Color.White,
fontSize = 12.sp, fontWeight = FontWeight.SemiBold)
                                        }
                                    }
                                }
                            }
                        }
                    }
                }
                NowLockNavTab.ABOUT -> {
                    LazyColumn(
                        modifier = Modifier.widthIn(max =
screenMetrics.contentMaxWidthDp.dp).fillMaxSize().padding(horizontal = 16.dp),
                        verticalArrangement = Arrangement.spacedBy(14.dp),
                        contentPadding = PaddingValues(top = 16.dp, bottom =
navBarBottomInset + 76.dp)
                    ) {
                        item {
                            Surface(
                                shape = RoundedCornerShape(24.dp),
                                color = card,
                                border = BorderStroke(1.dp, border),
                                modifier = Modifier.fillMaxWidth()
                            ) {
                                Column(
```

```
            modifier = Modifier.padding(22.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Image(
                painter = painterResource(id =
R.drawable.ic_nowlock_logo),
                contentDescription = "NowLock Logo",
                modifier = Modifier
                    .size(76.dp)
                    .clip(RoundedCornerShape(18.dp))
            )
            Spacer(Modifier.height(12.dp))
            AnimatedRgbBrandText(
                text = "NowLock v9.0.2",
                fontSize = 25.sp
            )
            Text(
                s.aboutSubtitle,
                color = blue,
                fontSize = 13.sp,
                fontWeight = FontWeight.SemiBold,
                textAlign = TextAlign.Center
            )
            Spacer(Modifier.height(14.dp))
            Text(
                text = s.aboutBody,
                color = sub,
                fontSize = 12.5.sp,
                textAlign = TextAlign.Center,
                lineHeight = 21.sp
            )
            Spacer(Modifier.height(18.dp))
            HorizontalDivider(color = border)
            Spacer(Modifier.height(14.dp))
            Text(
                s.aboutTechTitle,
                color = txt,
                fontWeight = FontWeight.Bold,
                fontSize = 13.5.sp
            )
            Spacer(Modifier.height(10.dp))
            Column(
                modifier = Modifier.fillMaxWidth(),
```

```
                verticalArrangement =
Arrangement.spacedBy(6.dp)
            ) {
                s.aboutTechBullets.forEach { techLine ->
                    Text(
                        text = techLine,
                        color = sub,
                        fontSize = 11.5.sp,
                        lineHeight = 18.sp
                    )
                }
            }
            Spacer(Modifier.height(18.dp))
            HorizontalDivider(color = border)
            Spacer(Modifier.height(14.dp))
            Text(
                s.aboutPrivacyTitle,
                color = txt,
                fontWeight = FontWeight.Bold,
                fontSize = 13.5.sp
            )
            Spacer(Modifier.height(6.dp))
            Text(
                text = s.aboutPrivacySummary,
                color = sub,
                fontSize = 12.sp,
                textAlign = TextAlign.Center,
                lineHeight = 19.sp
            )
            Spacer(Modifier.height(8.dp))
            TextButton(
                onClick = { showPrivacyFullModal = true }
            ) {
                Text(
                    s.aboutShowMoreBtn,
                    color = blue,
                    fontSize = 12.5.sp,
                    fontWeight = FontWeight.Bold
                )
            }
            Spacer(Modifier.height(12.dp))
            HorizontalDivider(color = border)
            Spacer(Modifier.height(14.dp))
```

```
                                        Text(
                                            s.aboutContactTitle,
                                            color = txt,
                                            fontWeight = FontWeight.SemiBold,
                                            fontSize = 13.5.sp
                                        )
                                        Spacer(Modifier.height(10.dp))
                                        Button(
                                            onClick = {
                                                val emailIntent =
Intent(Intent.ACTION_SENDTO).apply {
                                                    data =
Uri.parse("mailto:hades.plagueis@gmail.com")
                                                    putExtra(Intent.EXTRA_SUBJECT, "NowLock
App Support - Hamed")
                                                }
                                                runCatching {
startActivity(Intent.createChooser(emailIntent, "Send Email")) }
                                            },
                                            modifier = Modifier.fillMaxWidth().height(44.dp),
                                            colors =
ButtonDefaults.buttonColors(containerColor = blue),
                                            shape = RoundedCornerShape(14.dp)
                                        ) {
                                            Text("✉ hades.plagueis@gmail.com", color =
Color.White, fontSize = 13.sp)
                                        }
                                        Spacer(Modifier.height(10.dp))
                                        OutlinedButton(
                                            onClick = {
                                                showUpdateSubtitle = true
                                                scope.launch {
                                                    delay(6500)
                                                    showUpdateSubtitle = false
                                                }
                                            },
                                            modifier = Modifier.fillMaxWidth().height(44.dp),
                                            shape = RoundedCornerShape(14.dp),
                                            border = BorderStroke(1.dp, blue)
                                        ) {
                                            Text(
                                                s.aboutUpdateBtn,
                                                color = blue,
```

```
                                        fontSize = 13.sp,
                                        fontWeight = FontWeight.SemiBold
                                    )
                                }
                            }
                        }
                        }
                    }
                }
                }
                // Theme-Aware Bottom Subtitle Banner: "به سایت توسعه‌دهنده مراجعه کنید" +
Clickable GitHub Link
                AnimatedVisibility(
                    visible = showUpdateSubtitle,
                    enter = slideInVertically(initialOffsetY = { it }) + fadeIn(),
                    exit = slideOutVertically(targetOffsetY = { it }) + fadeOut(),
                    modifier = Modifier
                        .align(Alignment.BottomCenter)
                        .padding(horizontal = 20.dp, vertical = navBarBottomInset +
16.dp)
                ) {
                    Surface(
                        shape = RoundedCornerShape(16.dp),
                        color = if (isDark) Color(0xFF000000) else Color(0xFFFFFFFF),
                        border = BorderStroke(1.dp, if (isDark) Color(0x33FFFFFF) else
Color(0x22000000)),
                        shadowElevation = 10.dp
                    ) {
                        Column(
                            modifier = Modifier
                                .fillMaxWidth()
                                .clickable {
                                    runCatching {
                                        startActivity(Intent(Intent.ACTION_VIEW,
Uri.parse(GITHUB_RELEASES_PAGE_URL)))
                                    }
                                }
                                .padding(horizontal = 18.dp, vertical = 12.dp),
                            horizontalAlignment = Alignment.CenterHorizontally
                        ) {
                            Text(
```

```
                                    text = s.updateVisitSiteMsg,
                                    color = if (isDark) Color(0xFFFFFFFF) else
Color(0xFF000000),
                                    fontSize = 13.sp,
                                    fontWeight = FontWeight.SemiBold,
                                    textAlign = TextAlign.Center
                                )
                                Spacer(Modifier.height(4.dp))
                                Text(
                                    text = GITHUB_SHORT_DISPLAY_LINK,
                                    color = blue,
                                    fontSize = 13.sp,
                                    fontWeight = FontWeight.Bold,
                                    textDecoration = TextDecoration.Underline,
                                    textAlign = TextAlign.Center
                                )
                            }
                        }
                    }
                }
                // Direct In-Activity Lock Screen for NowLock itself (Zero task-switching,
zero relock loops!)
                val isSelfLocked by isSelfLockedState
                if (isSelfLocked) {
                    val config = LocalConfiguration.current
                    val isCompactScreen = config.screenHeightDp < 700
                    val primaryTextColor = Color(0xFFF8FAFC)
                    val secondaryTextColor = Color(0xFFD8DEE9)
                    val usePattern = (comboMode ==
LockComboMode.PATTERN_FINGERPRINT && AppRepo.isPatternSet()) ||
                        (!AppRepo.isPinSet() && AppRepo.isPatternSet())
                    val canFp = fpEnabled && canUseFingerprint(this@MainActivity)
                    var selfEnteredPin by remember { mutableStateOf("") }
                    var selfShowingFakeCrash by remember { mutableStateOf(false) }
                    var selfCooldownSec by remember {
mutableStateOf(AppRepo.getLockoutRemainingSeconds()) }
                    val defaultPrompt = if (usePattern) s.lockPromptPattern else
s.lockPromptPin
                    var selfStatusMsg by remember(appLang, usePattern) {
                        mutableStateOf(if (selfCooldownSec > 0)
s.formatCooldown(selfCooldownSec) else defaultPrompt)
                    }
```

```
                    var selfIsError by remember { mutableStateOf(selfCooldownSec > 0) }
                    var selfBioPulse by remember { mutableStateOf(false) }
                    LaunchedEffect(selfCooldownSec) {
                        if (selfCooldownSec > 0) {
                            selfIsError = true
                            selfStatusMsg = s.formatCooldown(selfCooldownSec)
                            delay(1000L)
                            val rem = AppRepo.getLockoutRemainingSeconds()
                            selfCooldownSec = rem
                            if (rem <= 0) {
                                selfIsError = false
                                selfStatusMsg = defaultPrompt
                            }
                        }
                    }
                    fun onSelfUnlockSuccess() {
                        AppRepo.resetFailedAttempts()
                        selfCooldownSec = 0
                        AppRepo.markUnlocked(packageName)
                        isSelfLockedState.value = false
                    }
                    fun onSelfWrongAttempt() {
                        selfIsError = true
                        AppRepo.triggerShortVibration(this@MainActivity, 65L)
                        val (_, penaltySec) =
AppRepo.registerWrongCredentialAttempt(this@MainActivity, packageName)
                        if (penaltySec > 0) {
                            selfCooldownSec = penaltySec
                            selfStatusMsg = s.formatCooldown(penaltySec)
                        } else {
                            val activeRem = AppRepo.getLockoutRemainingSeconds()
                            if (activeRem > 0) {
                                selfCooldownSec = activeRem
                                selfStatusMsg = s.formatCooldown(activeRem)
                            } else {
                                selfStatusMsg = s.lockWrongAttempt
                            }
                        }
                    }
                    LaunchedEffect(isSelfLocked) {
                        if (isSelfLocked && canFp) {
                            delay(160)
                            selfBioPulse = true
```

```
            triggerSelfFingerprintPrompt(
                s = s,
                onSuccess = { onSelfUnlockSuccess() },
                onFailedAttempt = { onSelfWrongAttempt() },
                onCancelOrError = { selfBioPulse = false }
            )
        }
    }
    FrostedAppColorGlassBackground(
        targetPkg = packageName,
        isDark = isDark,
        modifier = Modifier
            .fillMaxSize()
            .pointerInput(Unit) { detectTapGestures {} }
    ) {
        AdaptiveLockScreenBody(
            s = s,
            usePattern = usePattern,
            fpEnabled = canFp,
            isFingerprintPulsing = selfBioPulse,
            statusMessage = selfStatusMsg,
            isErrorState = selfIsError,
            enteredPinLength = selfEnteredPin.length,
            primaryTextColor = primaryTextColor,
            secondaryTextColor = secondaryTextColor,
            accentBlue = blue,
            onPatternAttempt = { patternStr, resetPatternCallback ->
                val rem = AppRepo.getLockoutRemainingSeconds()
                val isCorrect = AppRepo.verifyPattern(patternStr)
                val isDecoy = AppRepo.verifyDecoyPattern(patternStr)
                if (rem > 0) {
                    selfCooldownSec = rem
                    if (!isCorrect && !isDecoy) {
                        onSelfWrongAttempt()
                    } else {
                        selfIsError = true
                        AppRepo.triggerShortVibration(this@MainActivity, 50L)
                        selfStatusMsg = s.formatCooldown(selfCooldownSec)
                    }
                    scope.launch {
                        delay(320)
                        resetPatternCallback()
                    }
```

```
                                    } else if (isCorrect) {
                                        onSelfUnlockSuccess()
                                    } else if (isDecoy) {
                                        AppRepo.appendSecurityLog(this@MainActivity,
packageName, "DECOY_USED")
                                        resetPatternCallback()
                                        selfShowingFakeCrash = true
                                    } else {
                                        onSelfWrongAttempt()
                                        scope.launch {
                                            delay(350)
                                            resetPatternCallback()
                                            if (selfCooldownSec == 0) {
                                                selfIsError = false
                                                selfStatusMsg = defaultPrompt
                                            }
                                        }
                                    }
                                },
                                onDigitClick = { digit ->
                                    if (selfEnteredPin.length < 4) {
                                        if (selfCooldownSec <= 0) selfIsError = false
                                        selfEnteredPin += digit.toString()
                                        if (selfEnteredPin.length == 4) {
                                            val rem = AppRepo.getLockoutRemainingSeconds()
                                            val isCorrect = AppRepo.verifyPin(selfEnteredPin)
                                            val isDecoy = AppRepo.verifyDecoyPin(selfEnteredPin)
                                            if (rem > 0) {
                                                selfCooldownSec = rem
                                                if (!isCorrect && !isDecoy) {
                                                    onSelfWrongAttempt()
                                                } else {
                                                    selfIsError = true
                                                    AppRepo.triggerShortVibration(this@MainActivity,
50L)
                                                    selfStatusMsg = s.formatCooldown(selfCooldownSec)
                                                }
                                                scope.launch {
                                                    delay(260)
                                                    selfEnteredPin = ""
                                                }
                                            } else if (isCorrect) {
                                                onSelfUnlockSuccess()
```

```
                                    } else if (isDecoy) {
                                        AppRepo.appendSecurityLog(this@MainActivity,
packageName, "DECOY_USED")
                                        selfEnteredPin = ""
                                        selfShowingFakeCrash = true
                                    } else {
                                        onSelfWrongAttempt()
                                        scope.launch {
                                            delay(260)
                                            selfEnteredPin = ""
                                        }
                                    }
                                }
                            }
                        },
                        onBackspaceClick = {
                            if (selfEnteredPin.isNotEmpty()) {
                                selfIsError = false
                                selfEnteredPin = selfEnteredPin.dropLast(1)
                            }
                        },
                        onFingerprintClick = {
                            selfBioPulse = true
                            triggerSelfFingerprintPrompt(
                                s = s,
                                onSuccess = { onSelfUnlockSuccess() },
                                onFailedAttempt = { onSelfWrongAttempt() },
                                onCancelOrError = { selfBioPulse = false }
                            )
                        },
                        onExitClick = { finish() }
                    )
                    if (selfShowingFakeCrash) {
                        FakeAndroidSystemCrashOverlay(
                            targetPkg = packageName,
                            s = s,
                            onCloseApp = { finish() }
                        )
                    }
                }
            }
            // Recents Privacy Padlock Cover when paused in background switcher
            if (isPausedInRecents && recentsPrivacy) {
```

```
                        Box(
                            modifier = Modifier
                                .fillMaxSize()
                                .background(if (isDark) Color(0xFF181856) else
Color(0xFFF5F6FA)),
                            contentAlignment = Alignment.Center
                        ) {
                            Image(
                                painter = painterResource(id = R.drawable.ic_nowlock_logo),
                                contentDescription = "NowLock Privacy Shield",
                                modifier = Modifier.size(92.dp).clip(RoundedCornerShape(22.dp))
                            )
                        }
                    }
                    if (showPrivacyFullModal) {
                        PrivacyPolicyFullModal(
                            s = s,
                            isFa = isFa,
                            isDark = isDark,
                            txtColor = txt,
                            subColor = sub,
                            accentBlue = blue,
                            onDismiss = { showPrivacyFullModal = false }
                        )
                    }
                    // Translucent Glassy Top Status Bar & Bottom Navigation Bar Overlays
(ONLY active in NowLock main UI, NOT on Lock Screen!)
                    if (!isSelfLocked) {
                        NotchTranslucentStatusBarOverlay(isDark = isDark)
                        GlassyNavigationBarOverlay(isDark = isDark, modifier =
Modifier.align(Alignment.BottomCenter))
                    }
                    if (showPinDialog) {
                        PinSetupDialog(
                            s = s,
                            onDismiss = { showPinDialog = false; pendingLockPkg = null },
                            onSavePin = { pin ->
                                AppRepo.setPin(pin)
                                AppRepo.setComboMode(LockComboMode.PIN_FINGERPRINT)
                                comboMode = LockComboMode.PIN_FINGERPRINT
                                passwordSetState = true
                                pendingLockPkg?.let { pkg ->
                                    AppRepo.toggleAppLock(pkg, true)
```

```
                                    if (pkg == packageName)
AppRepo.markUnlocked(packageName)
                                    appList = appList.map { if (it.packageName == pkg)
it.copy(isLocked = true) else it }
                                }
                                pendingLockPkg = null
                                showPinDialog = false
                                NowLockMonitorService.startServiceSafely(this@MainActivity)
                            }
                        )
                    }
                    if (showPatternDialog) {
                        PatternSetupDialog(
                            s = s,
                            onDismiss = { showPatternDialog = false; pendingLockPkg = null },
                            onSavePattern = { pat ->
                                AppRepo.setPattern(pat)

AppRepo.setComboMode(LockComboMode.PATTERN_FINGERPRINT)
                                comboMode = LockComboMode.PATTERN_FINGERPRINT
                                passwordSetState = true
                                pendingLockPkg?.let { pkg ->
                                    AppRepo.toggleAppLock(pkg, true)
                                    if (pkg == packageName)
AppRepo.markUnlocked(packageName)
                                    appList = appList.map { if (it.packageName == pkg)
it.copy(isLocked = true) else it }
                                }
                                pendingLockPkg = null
                                showPatternDialog = false
                                NowLockMonitorService.startServiceSafely(this@MainActivity)
                            }
                        )
                    }
                    if (showDecoyPinDialog) {
                        PinSetupDialog(
                            s = s,
                            isDecoySetup = true,
                            onDismiss = { showDecoyPinDialog = false },
                            onSavePin = { decoyPin ->
                                AppRepo.setDecoyPin(decoyPin)
                                decoyEnabled = true
                                decoyPinSetState = true
```

```
                        showDecoyPinDialog = false
                        Toast.makeText(this@MainActivity, s.btnChangeDecoyPin,
Toast.LENGTH_SHORT).show()
                    }
                )
            }
            if (showDecoyPatternDialog) {
                PatternSetupDialog(
                    s = s,
                    isDecoySetup = true,
                    onDismiss = { showDecoyPatternDialog = false },
                    onSavePattern = { decoyPat ->
                        AppRepo.setDecoyPattern(decoyPat)
                        decoyEnabled = true
                        decoyPatternSetState = true
                        showDecoyPatternDialog = false
                        Toast.makeText(this@MainActivity, s.btnChangeDecoyPattern,
Toast.LENGTH_SHORT).show()
                    }
                )
            }
            if (showSecQSetupDialog) {
                SecurityQuestionSetupDialog(
                    s = s,
                    accentBlue = blue,
                    onDismiss = { showSecQSetupDialog = false },
                    onSaveQuestion = { qIdx, ans ->
                        AppRepo.saveSecurityQuestion(qIdx, ans)
                        secQuestionSetState = true
                        showSecQSetupDialog = false
                        Toast.makeText(this@MainActivity, s.secQSavedToast,
Toast.LENGTH_LONG).show()
                    }
                )
            }
            if (showSecQRecoverDialog) {
                SecurityQuestionRecoveryDialog(
                    s = s,
                    accentBlue = blue,
                    onDismiss = { showSecQRecoverDialog = false },
                    onVerifiedReset = {
                        AppRepo.resetPrimaryCredentialsBySecurityQuestion()
                        passwordSetState = false
```

```
                                showSecQRecoverDialog = false
                                Toast.makeText(this@MainActivity, s.secQResetSuccessToast,
Toast.LENGTH_LONG).show()
                                if (comboMode == LockComboMode.PATTERN_FINGERPRINT) {
                                    showPatternDialog = true
                                } else {
                                    showPinDialog = true
                                }
                            }
                        )
                    }
                }
            }
        }
    }
    private fun loadRealLauncherApps(): List<RealAppItem> {
        val pm = packageManager
        val mainIntent = Intent(Intent.ACTION_MAIN, null).apply {
            addCategory(Intent.CATEGORY_LAUNCHER)
        }
        val resolveInfos = pm.queryIntentActivities(mainIntent, 0)
        val blockedKeywords = listOf(
            "plugin", "effect", "filter", "sticker", "wallpaper",
            "auto_generated_rro", "overlay", "provider", "com.android.cts"
        )
        val lockedSet = AppRepo.getLockedPackages()
        val items = resolveInfos
            .mapNotNull { ri ->
                val pkg = ri.activityInfo?.packageName ?: return@mapNotNull null
                if (blockedKeywords.any { pkg.lowercase().contains(it) }) return@mapNotNull
null
                val isSelf = (pkg == packageName)
                val label = if (isSelf) "NowLock" else (ri.loadLabel(pm)?.toString() ?: pkg)
                val targetPx = if (AppRepo.isLowEndHardware) 84 else 120
                val iconDrawable = ri.loadIcon(pm) ?: pm.defaultActivityIcon
                val imgBmp = drawableToHdImageBitmap(iconDrawable, targetPx)
                RealAppItem(
                    label = label,
                    packageName = pkg,
                    iconImage = imgBmp,
                    isLocked = lockedSet.contains(pkg),
                    isSelfApp = isSelf
```

```
                )
            }
            .distinctBy { it.packageName }
            .toMutableList()
        if (items.none { it.packageName == packageName }) {
            runCatching {
                val targetPx = if (AppRepo.isLowEndHardware) 84 else 120
                val ai = pm.getApplicationInfo(packageName, 0)
                val imgBmp = drawableToHdImageBitmap(pm.getApplicationIcon(ai), targetPx)
                items.add(
                    0,
                    RealAppItem(
                        label = "NowLock",
                        packageName = packageName,
                        iconImage = imgBmp,
                        isLocked = lockedSet.contains(packageName),
                        isSelfApp = true
                    )
                )
            }
        }
        return items.sortedWith(
            compareByDescending<RealAppItem> { it.isSelfApp }
                .thenByDescending { it.isLocked }
                .thenBy { it.label.lowercase() }
        )
    }
}
@Composable
fun PrivacyPolicyFullModal(
    s: UiStrings,
    isFa: Boolean,
    isDark: Boolean,
    txtColor: Color,
    subColor: Color,
    accentBlue: Color,
    onDismiss: () -> Unit
) {
    val isAmoled = AppRepo.isAmoledEffective()
    BackHandler(onBack = onDismiss)
    Box(
        modifier = Modifier
            .fillMaxSize()
```

```
                .background(Color(0x99000000))
                .pointerInput(Unit) { detectTapGestures { onDismiss() } }

.windowInsetsPadding(WindowInsets.systemBars.union(WindowInsets.displayCutout))
                .padding(horizontal = 16.dp, vertical = 14.dp),
            contentAlignment = Alignment.Center
        ) {
            Surface(
                modifier = Modifier
                    .widthIn(max = 680.dp)
                    .fillMaxSize()
                    .pointerInput(Unit) { detectTapGestures {} },
                shape = RoundedCornerShape(24.dp),
                color = if (isAmoled) Color(0xFF0A0A0F) else if (isDark) Color(0xFF1F1D52) else
Color(0xFFFFFFFF),
                border = BorderStroke(1.dp, if (isDark) Color(0x28FFFFFF) else
Color(0x18000000))
            ) {
                Column(
                    modifier = Modifier
                        .fillMaxSize()
                        .padding(20.dp),
                    verticalArrangement = Arrangement.SpaceBetween
                ) {
                    Column(
                        modifier = Modifier
                            .weight(1f)
                            .verticalScroll(rememberScrollState()),
                        verticalArrangement = Arrangement.spacedBy(14.dp)
                    ) {
                        Text(
                            text = s.aboutPrivacyTitle,
                            color = txtColor,
                            fontSize = 18.sp,
                            fontWeight = FontWeight.Bold
                        )
                        HorizontalDivider(color = subColor.copy(alpha = 0.2f))
                        s.privacyModalSections.forEach { (heading, body) ->
                            Text(heading, color = accentBlue, fontSize = 14.sp, fontWeight =
FontWeight.Bold)
                            Text(body, color = subColor, fontSize = 12.5.sp, lineHeight = 20.sp)
                        }
                    }
```

```
            Spacer(Modifier.height(12.dp))
            Button(
                onClick = onDismiss,
                modifier = Modifier.fillMaxWidth().height(46.dp),
                shape = RoundedCornerShape(14.dp),
                colors = ButtonDefaults.buttonColors(containerColor = accentBlue)
            ) {
                Text(s.btnClose, color = Color.White, fontWeight = FontWeight.Bold)
            }
        }
    }
}
@Composable
fun MinimalSectionCard(
    title: String,
    cardColor: Color,
    borderColor: Color,
    txtColor: Color,
    content: @Composable ColumnScope.() -> Unit
) {
    Surface(
        shape = RoundedCornerShape(20.dp),
        color = cardColor,
        border = BorderStroke(1.dp, borderColor),
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(
                text = title,
                color = txtColor,
                fontWeight = FontWeight.Bold,
                fontSize = 13.5.sp
            )
            Spacer(Modifier.height(10.dp))
            content()
        }
    }
}
@Composable
fun SettingsToggleRow(
    title: String,
    subtitle: String,
```

```
        checked: Boolean,
        txtColor: Color,
        subColor: Color,
        onCheckedChange: (Boolean) -> Unit
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(vertical = 8.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(modifier = Modifier.weight(1f).padding(end = 12.dp)) {
                Text(title, color = txtColor, fontSize = 13.sp, fontWeight = FontWeight.SemiBold)
                Text(subtitle, color = subColor, fontSize = 11.sp)
            }
            Switch(checked = checked, onCheckedChange = onCheckedChange)
        }
    }
    @Composable
    fun PinSetupDialog(
        s: UiStrings,
        isDecoySetup: Boolean = false,
        onDismiss: () -> Unit,
        onSavePin: (String) -> Unit
    ) {
        val context = LocalContext.current
        val hasOldPin = remember { if (isDecoySetup) AppRepo.isDecoyPinSet() else
AppRepo.isPinSet() }
        // step 0 = verify old PIN, step 1 = enter new PIN, step 2 = confirm new PIN
        var step by remember { mutableStateOf(if (hasOldPin) 0 else 1) }
        var pinInput by remember { mutableStateOf("") }
        var firstNewPin by remember { mutableStateOf("") }
        var errorMsg by remember { mutableStateOf<String?>(null) }
        val titleText = when (step) {
            0 -> s.dlgPinVerifyOldTitle
            1 -> s.dlgPinTitle
            else -> s.dlgPinConfirmTitle
        }
        val subText = errorMsg ?: when (step) {
            0 -> s.dlgPinVerifyOldSub
            1 -> s.dlgPinSub
            else -> s.dlgPinConfirmSub
```

```
        }
        fun handleAction() {
            if (pinInput.length != 4) return
            when (step) {
                0 -> {
                    val oldPinValid = if (isDecoySetup) AppRepo.verifyDecoyPinRaw(pinInput) else
AppRepo.verifyPin(pinInput)
                    if (oldPinValid) {
                        errorMsg = null
                        pinInput = ""
                        step = 1
                    } else {
                        AppRepo.triggerShortVibration(context, 60L)
                        errorMsg = s.lockWrongAttempt
                        pinInput = ""
                    }
                }
                1 -> {
                    if (isDecoySetup && AppRepo.verifyPin(pinInput)) {
                        AppRepo.triggerShortVibration(context, 60L)
                        errorMsg = s.dlgMismatchError
                        pinInput = ""
                    } else {
                        firstNewPin = pinInput
                        errorMsg = null
                        pinInput = ""
                        step = 2
                    }
                }
                2 -> {
                    if (pinInput == firstNewPin) {
                        AppRepo.resetFailedAttempts()
                        onSavePin(pinInput)
                    } else {
                        AppRepo.triggerShortVibration(context, 60L)
                        errorMsg = s.dlgMismatchError
                        firstNewPin = ""
                        pinInput = ""
                        step = 1
                    }
                }
            }
        }
```

```
        AlertDialog(
            onDismissRequest = onDismiss,
            title = {
                Text(titleText, fontWeight = FontWeight.Bold, fontSize = 16.sp)
            },
            text = {
                Column(horizontalAlignment = Alignment.CenterHorizontally, modifier =
Modifier.fillMaxWidth()) {
                    Text(
                        text = subText,
                        color = if (errorMsg != null) Color(0xFFEF4444) else Color.Unspecified,
                        fontSize = 12.sp,
                        textAlign = TextAlign.Center
                    )
                    Spacer(Modifier.height(12.dp))
                    Text(
                        text = pinInput.padEnd(4, '○'),
                        fontSize = 26.sp,
                        fontWeight = FontWeight.Bold,
                        color = if (errorMsg != null) Color(0xFFEF4444) else Color(0xFF5D56F5)
                    )
                    Spacer(Modifier.height(12.dp))
                    CompositionLocalProvider(LocalLayoutDirection provides LayoutDirection.Ltr) {
                        val rows = listOf(listOf(1, 2, 3), listOf(4, 5, 6), listOf(7, 8, 9))
                        rows.forEach { r ->
                            Row(horizontalArrangement = Arrangement.spacedBy(10.dp), modifier =
Modifier.padding(vertical = 4.dp)) {
                                r.forEach { d ->
                                    OutlinedButton(
                                        onClick = {
                                            if (pinInput.length < 4) {
                                                errorMsg = null
                                                pinInput += d.toString()
                                            }
                                        },
                                        modifier = Modifier.size(52.dp),
                                        shape = CircleShape,
                                        contentPadding = PaddingValues(0.dp)
                                    ) {
                                        Text(d.toString(), fontSize = 18.sp, fontWeight = FontWeight.Bold)
                                    }
                                }
                            }
```

```
                    }
                    Row(horizontalArrangement = Arrangement.spacedBy(10.dp), modifier =
Modifier.padding(top = 4.dp)) {
                        OutlinedButton(
                            onClick = {
                                if (pinInput.isNotEmpty()) {
                                    errorMsg = null
                                    pinInput = pinInput.dropLast(1)
                                }
                            },
                            modifier = Modifier.size(52.dp),
                            shape = CircleShape,
                            contentPadding = PaddingValues(0.dp)
                        ) {
                            Text("⌫", fontSize = 16.sp)
                        }
                        OutlinedButton(
                            onClick = {
                                if (pinInput.length < 4) {
                                    errorMsg = null
                                    pinInput += "0"
                                }
                            },
                            modifier = Modifier.size(52.dp),
                            shape = CircleShape,
                            contentPadding = PaddingValues(0.dp)
                        ) {
                            Text("0", fontSize = 18.sp, fontWeight = FontWeight.Bold)
                        }
                        Spacer(Modifier.size(52.dp))
                    }
                }
            }
        },
        confirmButton = {
            Button(
                enabled = pinInput.length == 4,
                onClick = { handleAction() }
            ) {
                Text(if (step == 2) s.btnSave else s.btnNext)
            }
        },
        dismissButton = {
```

```
                TextButton(onClick = onDismiss) {
                    Text(s.btnCancel)
                }
            }
        )
    }
    // 3-Step Continuous Touch-Drag Pattern Setup Dialog (Verify Old -> Draw New ->
Confirm New)
    @Composable
    fun PatternSetupDialog(
        s: UiStrings,
        isDecoySetup: Boolean = false,
        onDismiss: () -> Unit,
        onSavePattern: (String) -> Unit
    ) {
        val context = LocalContext.current
        val hasOldPattern = remember { if (isDecoySetup) AppRepo.isDecoyPatternSet() else
AppRepo.isPatternSet() }
        var step by remember { mutableStateOf(if (hasOldPattern) 0 else 1) }
        val selectedNodes = remember { mutableStateListOf<Int>() }
        var firstNewPattern by remember { mutableStateOf("") }
        var errorMsg by remember { mutableStateOf<String?>(null) }
        var currentTouchPos by remember { mutableStateOf<Offset?>(null) }
        var boxSize by remember { mutableStateOf(IntSize.Zero) }
        val titleText = when (step) {
            0 -> s.dlgPatternVerifyOldTitle
            1 -> s.dlgPatternTitle
            else -> s.dlgPatternConfirmTitle
        }
        val subText = errorMsg ?: when (step) {
            0 -> s.dlgPatternVerifyOldSub
            1 -> "${s.dlgPatternSub} (${selectedNodes.size})"
            else -> "${s.dlgPatternConfirmSub} (${selectedNodes.size})"
        }
        fun nodeCenter(index: Int): Offset {
            val row = (index - 1) / 3
            val col = (index - 1) % 3
            val cellW = boxSize.width / 3f
            val cellH = boxSize.height / 3f
            return Offset(cellW * (col + 0.5f), cellH * (row + 0.5f))
        }
        fun checkHitNode(pos: Offset): Int? {
            if (boxSize.width == 0) return null
```

```
            val hitRadius = (boxSize.width / 6f) * 0.68f
            for (i in 1..9) {
                val c = nodeCenter(i)
                if (hypot(pos.x - c.x, pos.y - c.y) <= hitRadius) {
                    return i
                }
            }
            return null
        }
        fun handlePatternAction() {
            if (selectedNodes.size < 4) return
            val patStr = selectedNodes.joinToString("-")
            when (step) {
                0 -> {
                    val oldPatValid = if (isDecoySetup) AppRepo.verifyDecoyPatternRaw(patStr)
else AppRepo.verifyPattern(patStr)
                    if (oldPatValid) {
                        errorMsg = null
                        selectedNodes.clear()
                        currentTouchPos = null
                        step = 1
                    } else {
                        AppRepo.triggerShortVibration(context, 60L)
                        errorMsg = s.lockWrongAttempt
                        selectedNodes.clear()
                        currentTouchPos = null
                    }
                }
                1 -> {
                    if (isDecoySetup && AppRepo.verifyPattern(patStr)) {
                        AppRepo.triggerShortVibration(context, 60L)
                        errorMsg = s.dlgMismatchError
                        selectedNodes.clear()
                        currentTouchPos = null
                    } else {
                        firstNewPattern = patStr
                        errorMsg = null
                        selectedNodes.clear()
                        currentTouchPos = null
                        step = 2
                    }
                }
                2 -> {
```

```
                    if (patStr == firstNewPattern) {
                        AppRepo.resetFailedAttempts()
                        onSavePattern(patStr)
                    } else {
                        AppRepo.triggerShortVibration(context, 60L)
                        errorMsg = s.dlgMismatchError
                        firstNewPattern = ""
                        selectedNodes.clear()
                        currentTouchPos = null
                        step = 1
                    }
                }
            }
        }
        AlertDialog(
            onDismissRequest = onDismiss,
            title = {
                Text(titleText, fontWeight = FontWeight.Bold, fontSize = 16.sp)
            },
            text = {
                Column(horizontalAlignment = Alignment.CenterHorizontally, modifier =
Modifier.fillMaxWidth()) {
                    Text(
                        text = subText,
                        color = if (errorMsg != null) Color(0xFFEF4444) else Color.Unspecified,
                        fontSize = 12.sp,
                        textAlign = TextAlign.Center
                    )
                    Spacer(Modifier.height(14.dp))
                    CompositionLocalProvider(LocalLayoutDirection provides LayoutDirection.Ltr) {
                        Box(
                            modifier = Modifier
                                .size(230.dp)
                                .onSizeChanged { boxSize = it }
                                .pointerInput(Unit) {
                                    detectDragGestures(
                                        onDragStart = { offset ->
                                            errorMsg = null
                                            selectedNodes.clear()
                                            checkHitNode(offset)?.let { node ->
                                                if (!selectedNodes.contains(node)) selectedNodes.add(node)
                                            }
                                            currentTouchPos = offset
```

```
                        },
                        onDrag = { change, _ ->
                            change.consume()
                            currentTouchPos = change.position
                            checkHitNode(change.position)?.let { node ->
                                if (!selectedNodes.contains(node)) {
                                    selectedNodes.add(node)
                                }
                            }
                        },
                        onDragEnd = {
                            currentTouchPos = null
                            if (selectedNodes.size < 4) {
                                selectedNodes.clear()
                            }
                        },
                        onDragCancel = {
                            currentTouchPos = null
                        }
                    )
                }
            ) {
                val lineColor = if (errorMsg != null) Color(0xFFEF4444) else
Color(0xFF5D56F5)
                val dotColor = Color(0xFF64748B)
                ComposeCanvas(modifier = Modifier.fillMaxSize()) {
                    for (i in 0 until selectedNodes.size - 1) {
                        val start = nodeCenter(selectedNodes[i])
                        val end = nodeCenter(selectedNodes[i + 1])
                        drawLine(
                            color = lineColor,
                            start = start,
                            end = end,
                            strokeWidth = 4.5.dp.toPx(),
                            cap = StrokeCap.Round
                        )
                    }
                    if (selectedNodes.isNotEmpty() && currentTouchPos != null) {
                        drawLine(
                            color = lineColor.copy(alpha = 0.65f),
                            start = nodeCenter(selectedNodes.last()),
                            end = currentTouchPos!!,
                            strokeWidth = 3.5.dp.toPx(),
```

```
                                cap = StrokeCap.Round
                            )
                        }
                        for (i in 1..9) {
                            val center = nodeCenter(i)
                            val isSelected = selectedNodes.contains(i)
                            if (isSelected) {
                                drawCircle(
                                    color = lineColor.copy(alpha = 0.24f),
                                    radius = 20.dp.toPx(),
                                    center = center
                                )
                                drawCircle(
                                    color = lineColor,
                                    radius = 8.5.dp.toPx(),
                                    center = center
                                )
                            } else {
                                drawCircle(
                                    color = dotColor.copy(alpha = 0.55f),
                                    radius = 6.dp.toPx(),
                                    center = center
                                )
                            }
                        }
                    }
                }
            }
            TextButton(onClick = { selectedNodes.clear(); currentTouchPos = null; errorMsg
= null }) {
                Text(s.btnClear)
            }
        }
    },
    confirmButton = {
        Button(
            enabled = selectedNodes.size >= 4,
            onClick = { handlePatternAction() }
        ) {
            Text(if (step == 2) s.btnSave else s.btnNext)
        }
    },
    dismissButton = {
```

```
            TextButton(onClick = onDismiss) {
                Text(s.btnCancel)
            }
        }
    )
}

@Composable
fun SecurityQuestionSetupDialog(
    s: UiStrings,
    accentBlue: Color,
    onDismiss: () -> Unit,
    onSaveQuestion: (Int, String) -> Unit
) {
    var selectedIdx by remember { mutableStateOf(0) }
    var answerInput by remember { mutableStateOf("") }
    AlertDialog(
        onDismissRequest = onDismiss,
        title = {
            Text(s.secQSetupTitle, fontWeight = FontWeight.Bold, fontSize = 15.5.sp)
        },
        text = {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .verticalScroll(rememberScrollState()),
                verticalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                Text(
                    text = s.secQSetupSub,
                    fontSize = 11.5.sp,
                    lineHeight = 17.sp
                )
                Spacer(Modifier.height(4.dp))
                s.secQuestionsList.forEachIndexed { idx, qText ->
                    val isSelected = selectedIdx == idx
                    Surface(
                        shape = RoundedCornerShape(12.dp),
                        color = if (isSelected) accentBlue.copy(alpha = 0.16f) else
Color.Transparent,
                        border = BorderStroke(
                            width = if (isSelected) 1.5.dp else 1.dp,
                            color = if (isSelected) accentBlue else Color(0x44888888)
```

```
                    ),
                    modifier = Modifier
                        .fillMaxWidth()
                        .clickable { selectedIdx = idx }
                ) {
                    Text(
                        text = qText,
                        fontSize = 12.sp,
                        fontWeight = if (isSelected) FontWeight.Bold else FontWeight.Normal,
                        lineHeight = 18.sp,
                        modifier = Modifier.padding(horizontal = 12.dp, vertical = 10.dp)
                    )
                }
            }
            Spacer(Modifier.height(6.dp))
            OutlinedTextField(
                value = answerInput,
                onValueChange = { answerInput = it },
                placeholder = { Text(s.secQAnswerPlaceholder, fontSize = 12.sp) },
                singleLine = true,
                modifier = Modifier.fillMaxWidth(),
                shape = RoundedCornerShape(12.dp)
            )
        }
    },
    confirmButton = {
        Button(
            enabled = answerInput.trim().length >= 2,
            onClick = { onSaveQuestion(selectedIdx, answerInput.trim()) },
            colors = ButtonDefaults.buttonColors(containerColor = accentBlue)
        ) {
            Text(s.btnSave, color = Color.White)
        }
    },
    dismissButton = {
        TextButton(onClick = onDismiss) {
            Text(s.btnCancel)
        }
    }
  )
}

@Composable
```

```
        fun SecurityQuestionRecoveryDialog(
            s: UiStrings,
            accentBlue: Color,
            onDismiss: () -> Unit,
            onVerifiedReset: () -> Unit
        ) {
            val context = LocalContext.current
            val questionIdx = remember { AppRepo.getSecurityQuestionIndex() }
            val questionText = s.secQuestionsList.getOrElse(questionIdx) {
s.secQuestionsList.first() }
            var answerInput by remember { mutableStateOf("") }
            var hasError by remember { mutableStateOf(false) }
            AlertDialog(
                onDismissRequest = onDismiss,
                title = {
                    Text(s.secQRecoverTitle, fontWeight = FontWeight.Bold, fontSize = 15.5.sp)
                },
                text = {
                    Column(
                        modifier = Modifier.fillMaxWidth(),
                        verticalArrangement = Arrangement.spacedBy(10.dp)
                    ) {
                        Text(
                            text = s.secQRecoverSub,
                            fontSize = 11.5.sp,
                            lineHeight = 17.sp
                        )
                        Surface(
                            shape = RoundedCornerShape(12.dp),
                            color = accentBlue.copy(alpha = 0.12f),
                            border = BorderStroke(1.dp, accentBlue.copy(alpha = 0.4f)),
                            modifier = Modifier.fillMaxWidth()
                        ) {
                            Text(
                                text = questionText,
                                fontSize = 12.5.sp,
                                fontWeight = FontWeight.Bold,
                                lineHeight = 19.sp,
                                modifier = Modifier.padding(12.dp)
                            )
                        }
                        OutlinedTextField(
                            value = answerInput,
```

```
                onValueChange = {
                    answerInput = it
                    hasError = false
                },
                placeholder = { Text(s.secQAnswerPlaceholder, fontSize = 12.sp) },
                isError = hasError,
                singleLine = true,
                modifier = Modifier.fillMaxWidth(),
                shape = RoundedCornerShape(12.dp)
            )
            if (hasError) {
                Text(
                    text = s.secQWrongError,
                    color = Color(0xFFEF4444),
                    fontSize = 11.5.sp,
                    fontWeight = FontWeight.SemiBold
                )
            }
        }
    },
    confirmButton = {
        Button(
            enabled = answerInput.trim().isNotEmpty(),
            onClick = {
                if (AppRepo.verifySecurityQuestionAnswer(answerInput)) {
                    onVerifiedReset()
                } else {
                    AppRepo.triggerShortVibration(context, 65L)
                    hasError = true
                }
            },
            colors = ButtonDefaults.buttonColors(containerColor = accentBlue)
        ) {
            Text(s.btnNext, color = Color.White)
        }
    },
    dismissButton = {
        TextButton(onClick = onDismiss) {
            Text(s.btnCancel)
        }
    }
  )
}
```

```
        ''')
        print("NowLock v7.5 project generated cleanly!")
    - name: Compile Optimized Release APK with JDK 17, Android 16 (API 36) & Gradle 8.7
      run: |
        export JAVA_HOME="${JAVA_HOME_17_X64:-/usr/lib/jvm/temurin-17-jdk-amd64}"
        export PATH="$JAVA_HOME/bin:$PATH"
        yes | $ANDROID_HOME/cmdline-tools/latest/bin/sdkmanager --licenses > /dev/null 2>&1 || true
        if [ ! -f /tmp/gradle-8.7/bin/gradle ]; then
          curl -sL https://services.gradle.org/distributions/gradle-8.7-bin.zip -o /tmp/gradle-8.7.zip
          unzip -q -o /tmp/gradle-8.7.zip -d /tmp
        fi
        /tmp/gradle-8.7/bin/gradle assembleRelease --no-daemon --no-build-cache --stacktrace
        cp app/build/outputs/apk/release/app-release.apk NowLock.apk
    - name: Publish Direct Download APK to GitHub Releases
      env:
        GH_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      run: |
        gh release delete v9.0.2-nowlock --repo "${{ github.repository }}" --cleanup-tag --yes 2>/dev/null || true
        cat << 'RELNOTES' > /tmp/release_notes.md
        ## 🛡️ NowLock v9.0.2 — Next-Gen Privacy & App Security for Android

        **NowLock** is a fast, ultra-lightweight, 100% offline Android privacy shield engineered with **Kotlin**, **Jetpack Compose**, and **Material 3**. Designed to protect sensitive apps with zero foreground lag and minimal battery impact across phones, tablets, and foldables.

        ### ✨ Highlights in v9.0.2
        - 🔑 **Interactive Security Question Recovery**: Added an in-app password recovery flow in Settings featuring lighthearted predefined questions and encrypted one-way answer storage—allowing you to safely reset your primary PIN and Pattern if forgotten while keeping your answer permanently hidden after setup.
        - 🕵️ **Unified 3-Step Decoy Credential Flow**: Managing your Fake Crash Decoy PIN and Decoy Pattern now follows the same 3-step verification flow (verify current credential ➔ enter new ➔ confirm new) as your primary lock credentials.
        - 🎨 **Refined 2×2 Theme Grid & True AMOLED Mode**: Reorganized the theme selector into a clean 2×2 grid (`System`, `Light`, `Dark`, `AMOLED Black`) and eliminated the brief navigation bar flash when resuming from Recent Apps.
        - 🚨 **Smart Security Incident Center**: Unified real-time tracking for failed unlock attempts, decoy triggers, lockouts, permission changes, and Device Admin tamper attempts—dynamically translated across all 5 supported languages (`English`, `آذری`, `فارسی`, `مازندرانی`, `لری``).
```

🔒 Privacy & Architecture

- **100% Offline & Local-First**: Zero internet permissions, zero ads, and zero cloud telemetry. All credentials and security answers are hashed locally with `SHA-256`.

- **Seamless Upgrade**: Signed with NowLock's persistent release key so you can install directly over previous versions without losing your settings or locked app list.

RELNOTES

gh release create v9.0.2-nowlock NowLock.apk --repo "${{ github.repository }}" --target "${{ github.sha }}" --title "🛡️ NowLock v9.0.2 — Smart Security Recovery, AMOLED Polish & Active Defense" --notes-file /tmp/release_notes.md